

Data Visualization in python

Name Rizwan ullah

Student : GCUF

line_chart

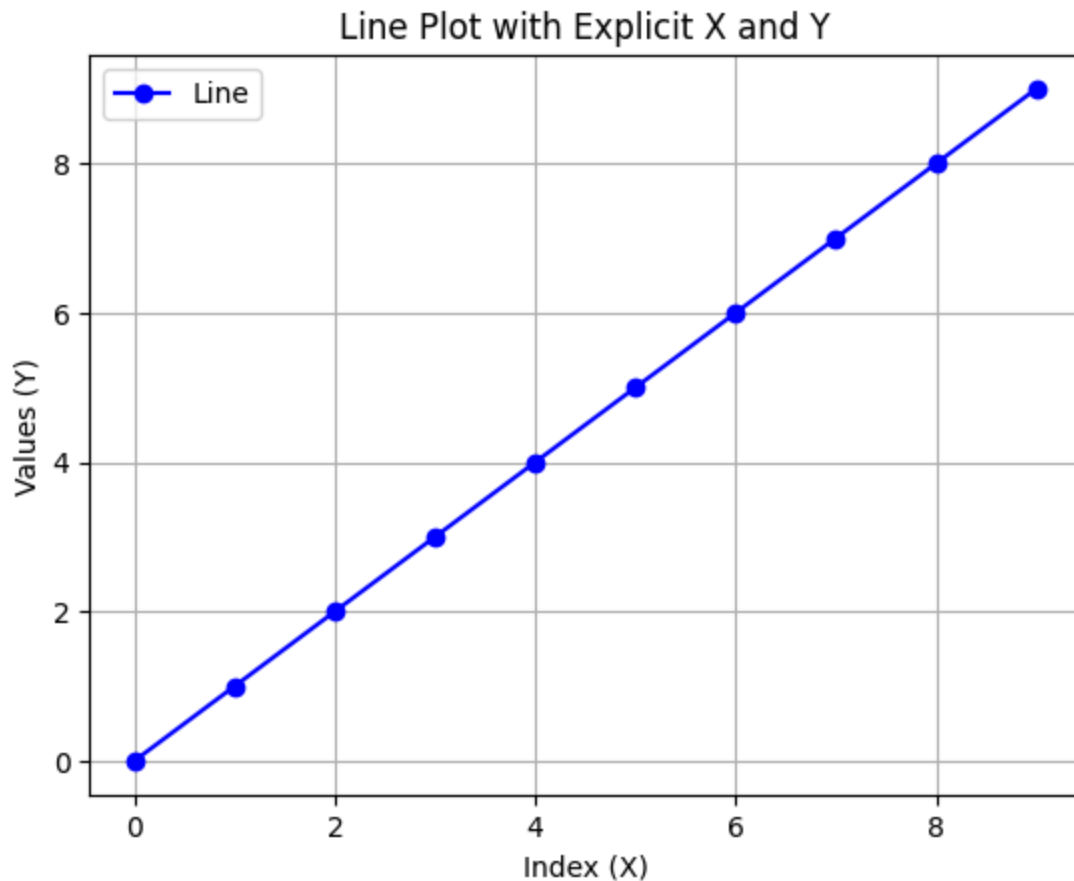
```
In [8]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

data = np.arange(10)
print("Array:", data)

x = np.arange(len(data))    # x-axis indices [0,1,2,...,9]
y = data                    # y-axis values

plt.plot(x, y, marker="o", color="blue", label="Line")
plt.legend()
plt.title("Line Plot with Explicit X and Y")
plt.xlabel("Index (X)")
plt.ylabel("Values (Y)")
plt.grid(True)
plt.show()
```

Array: [0 1 2 3 4 5 6 7 8 9]

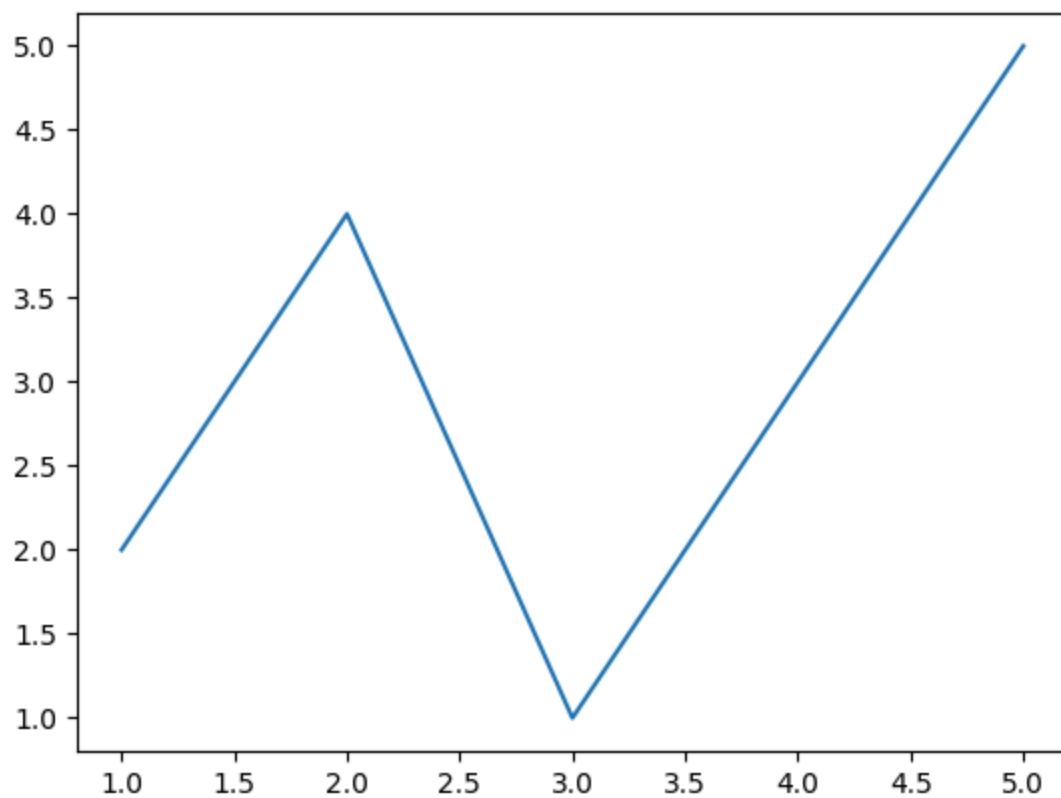
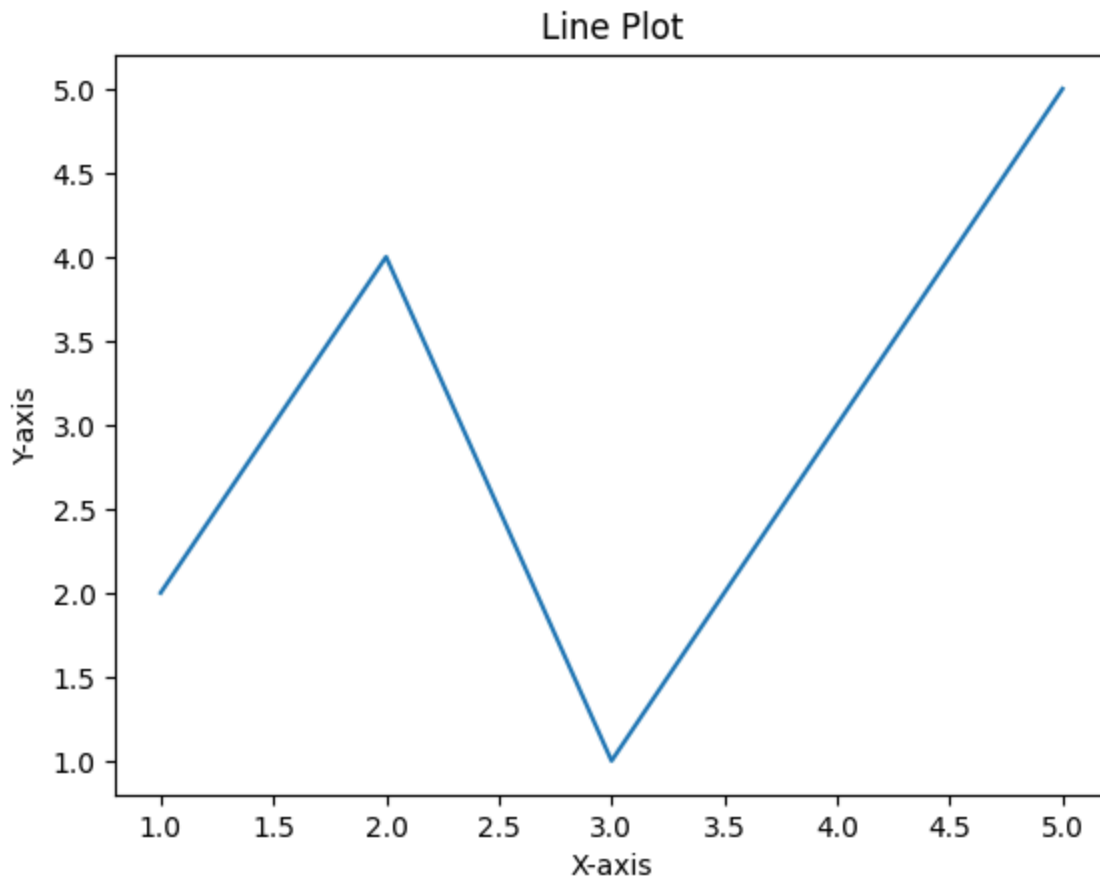


```
In [1]: import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import numpy as np

# Sample data use for :Trend dikhane ke Liye, Comparisons ke Liye, Continuous data k
x = [1, 2, 3, 4, 5]
y = [2, 4, 1, 3, 5]

# Create a line plot
plt.plot(x, y)
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.title("Line Plot")
linestyle="solid"
plt.show()

# Or using Seaborn
sns.lineplot(x=x, y=y)
plt.show()
```

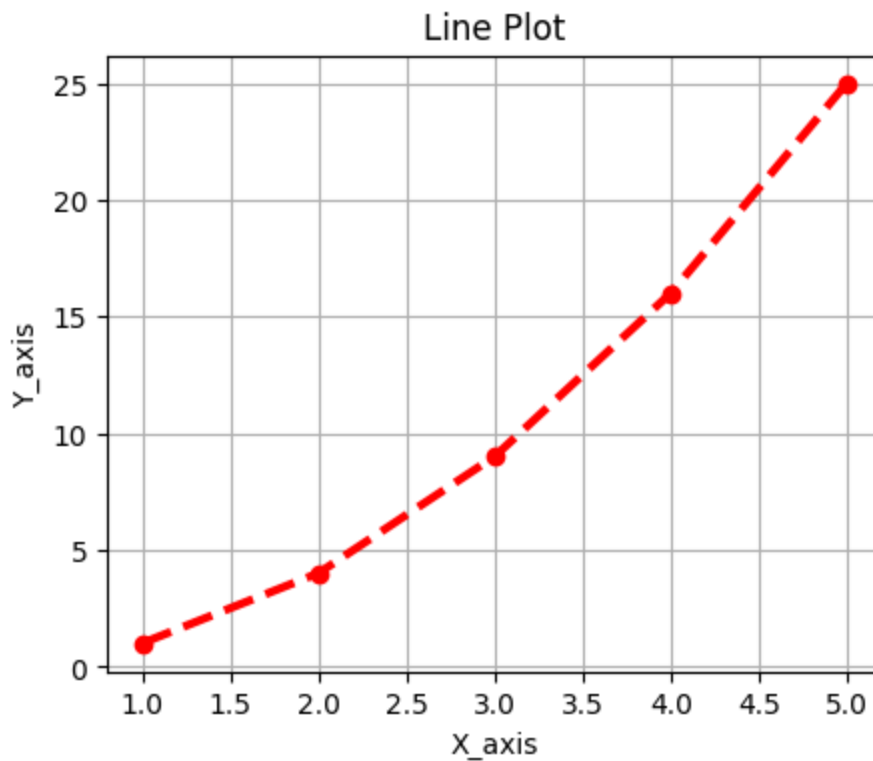


```
In [2]: import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import numpy as np
```

```

x= [1,2,3,4,5]
y= [1,4,9,16,25]
plt.figure(figsize=(5,4))
plt.plot(x, y ,color="red",linestyle="--",marker="o",linewidth=3)
plt.grid()
plt.xlabel("X_axis")
plt.ylabel("Y_axis")
plt.title("Line Plot")
plt.show()

```



Subplot

```

In [3]: import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import numpy as np
x = [1,2,3,4,5]
y1 = [1,4,9,16,25]
y2 = [1,2,3,4,5]
plt.figure(figsize=(9,5))
plt.subplot(2,2,1)
plt.plot(x,y1,color="green")
plt.title("plot1")

plt.subplot(2,2,2)
plt.plot(x,y1,color="red")
plt.title("plot2")

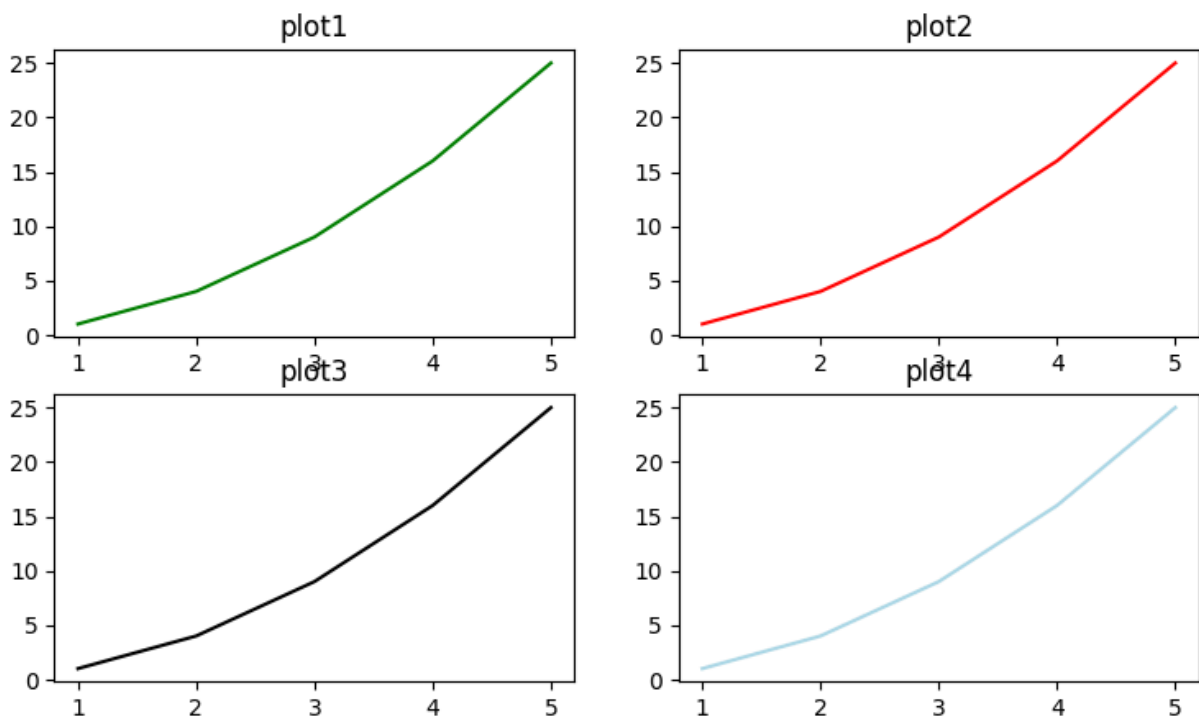
plt.subplot(2,2,3)
plt.plot(x,y1,color="black")

```

```
plt.title("plot3")

plt.subplot(2,2,4)
plt.plot(x,y1,color="lightblue")
plt.title("plot4")
```

Out[3]: Text(0.5, 1.0, 'plot4')



multiple line chart

In [4]: `import matplotlib.pyplot as plt`

```
# Sample data
x = [1, 2, 3, 4, 5]
y1 = [2, 3, 5, 7, 11]
y2 = [1, 4, 6, 8, 10]
y3 = [2, 2, 3, 3, 5]

# Plotting multiple lines
plt.plot(x, y1, label='Line 1', color='red', marker='o')
plt.plot(x, y2, label='Line 2', color='blue', linestyle='--')
plt.plot(x, y3, label='Line 3', color='green', linestyle='-.')

# Adding title and labels
plt.title('Multiple Line Chart Example')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')

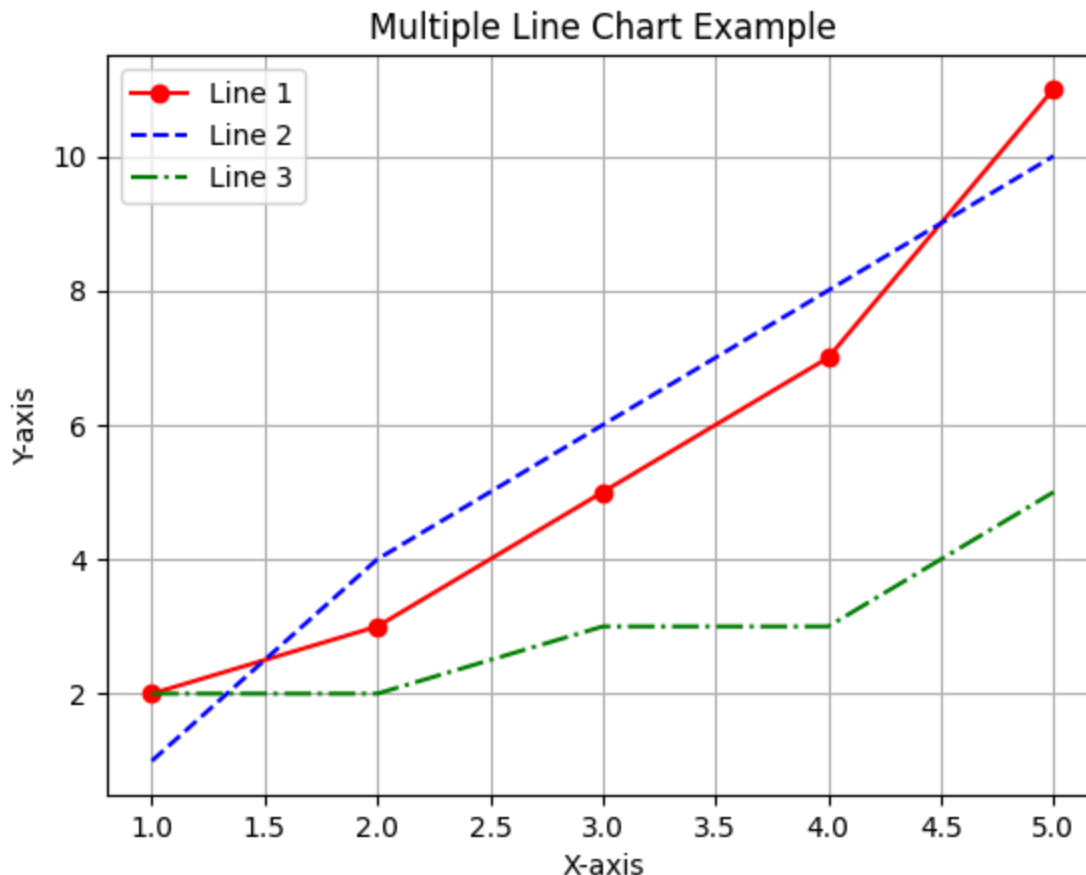
# Show Legend
plt.legend()

# Show grid
```

```
plt.grid(True)
```

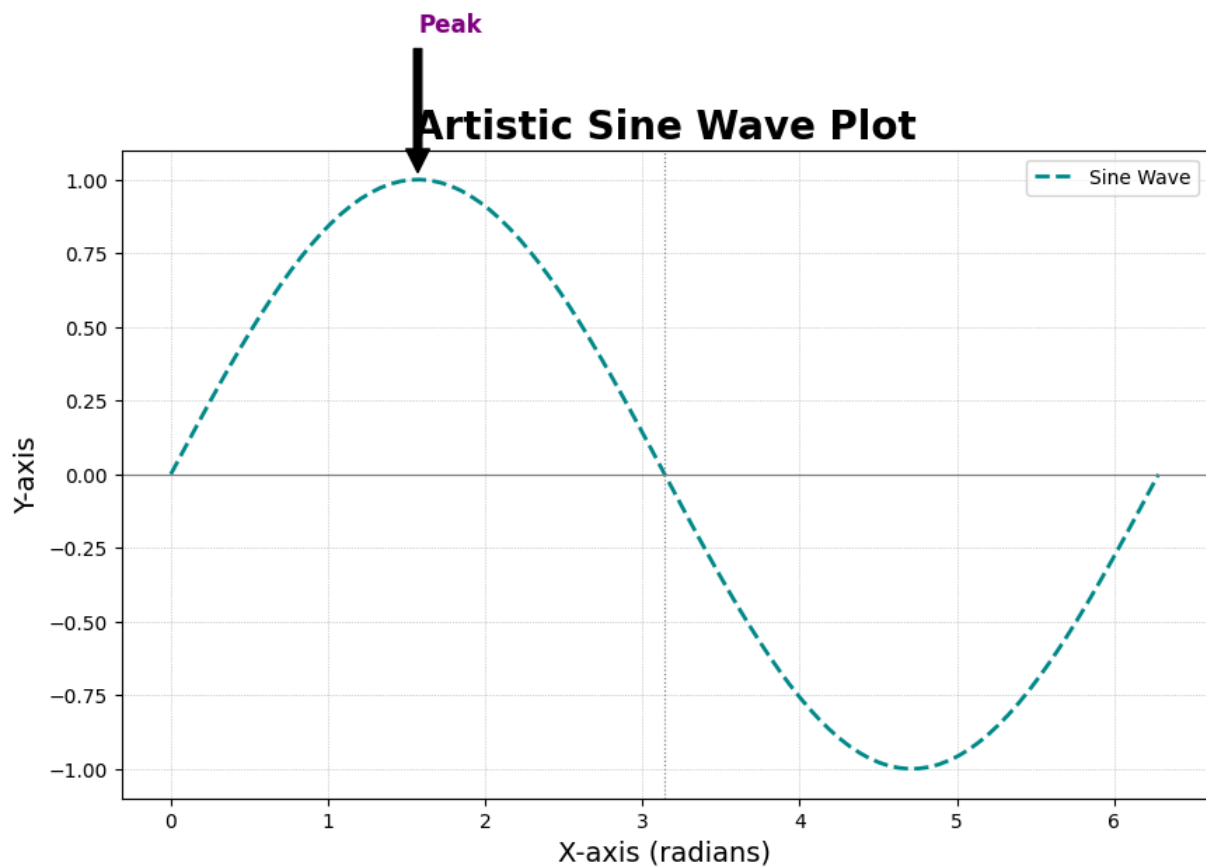
```
# Display plot
```

```
plt.show()
```



sine wave plot

```
In [1]: import numpy as np
import matplotlib.pyplot as plt
x = np.linspace(0, 2 * np.pi, 100)
y = np.sin(x)
plt.figure(figsize=(10, 6))
plt.plot(x, y, color='darkcyan', linestyle='--', linewidth=2, label='Sine Wave')
plt.title('Artistic Sine Wave Plot', fontsize=20, fontweight='bold')
plt.xlabel('X-axis (radians)', fontsize=14)
plt.ylabel('Y-axis', fontsize=14)
plt.axhline(0, color='grey', linewidth=0.8)
plt.axvline(np.pi, color='grey', linewidth=0.8, linestyle=':')
plt.legend(loc='upper right')
plt.grid(True, which='both', linestyle=':', linewidth=0.5)
plt.annotate('Peak', xy=(np.pi / 2, 1), xytext=(np.pi / 2, 1.5),
arrowprops=dict(facecolor='black', shrink=0.05),
fontsize=12, fontweight='bold', color='purple')
plt.savefig('sine_wave_plot.png')
```



Contour laine

```
In [5]: import matplotlib.pyplot as plt
import numpy as np

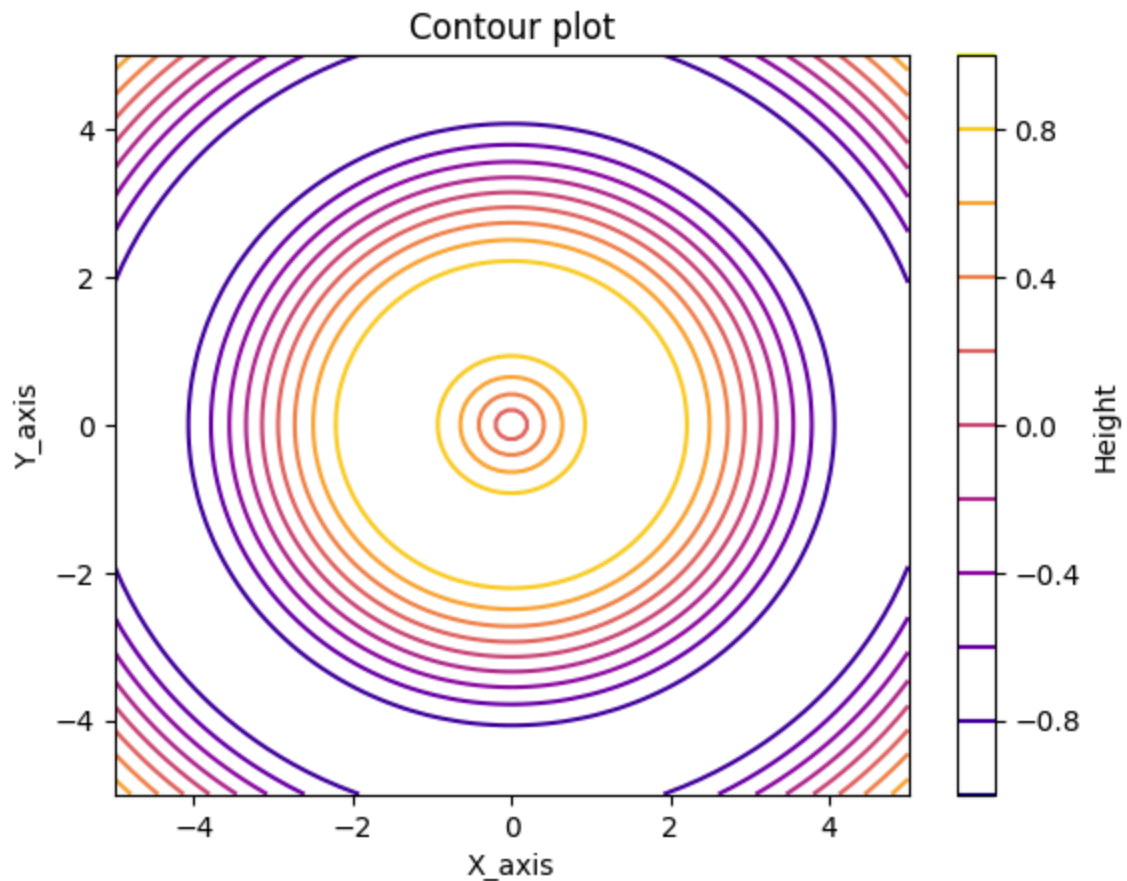
# Correct function name is linspace, not linespace
x = np.linspace(-5, 5, 100)
y = np.linspace(-5, 5, 100)

# Correct function name is meshgrid, not npmeshgrid
X, Y = np.meshgrid(x, y)

Z = np.sin(np.sqrt(X**2 + Y**2))

# Typo in 'camp', correct keyword is 'cmap' for color map
plt.contour(X, Y, Z, levels=10, cmap="plasma")

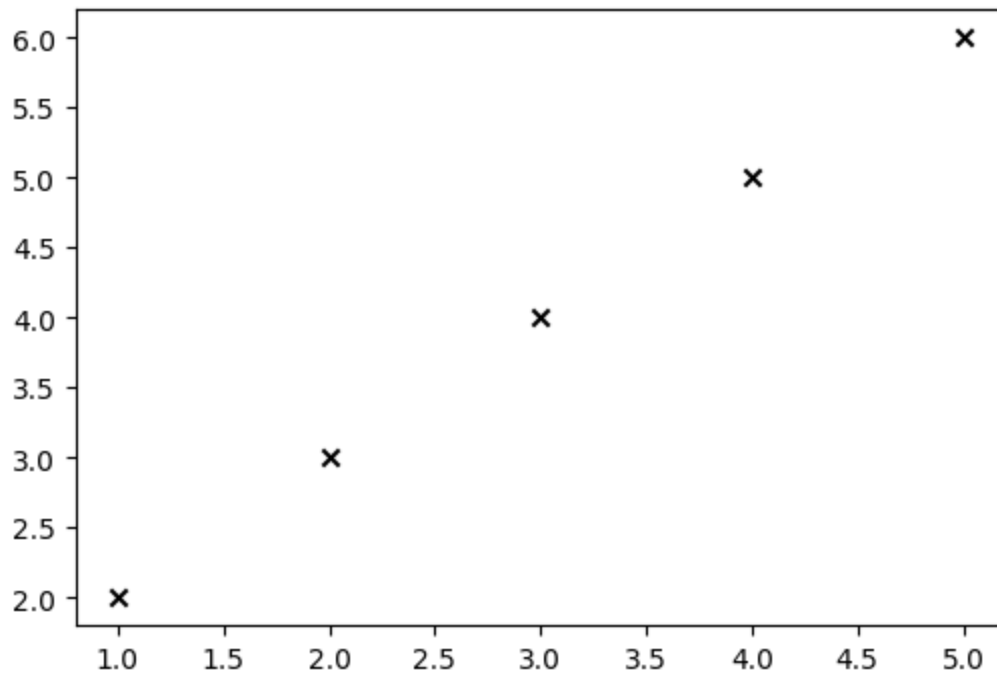
plt.colorbar(label="Height")
plt.title("Contour plot")
plt.xlabel("X_axis")
plt.ylabel("Y_axis")
plt.show()
```



SCATTAR PLOT

```
In [6]: import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import numpy as np
x = [1,2,3,4,5]
y = [2,3,4,5,6]
plt.figure(figsize=(6,4))
plt.scatter(x, y, color="black", marker="x")
```

Out[6]: <matplotlib.collections.PathCollection at 0x21e601b5400>

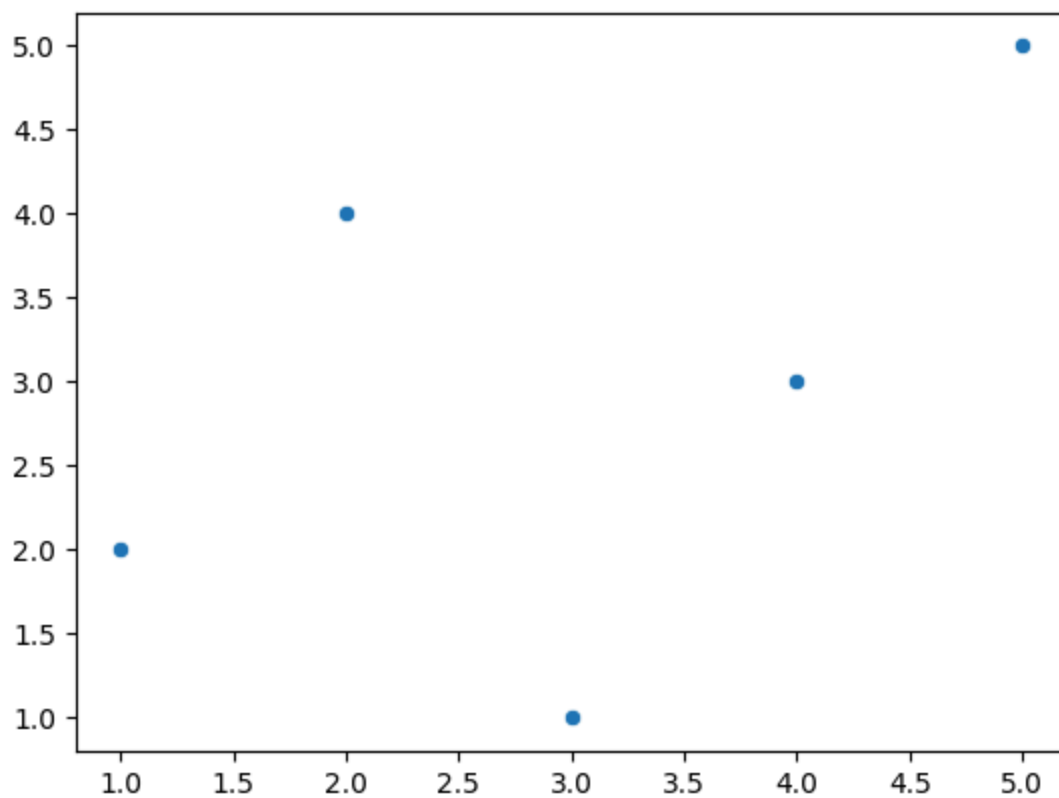
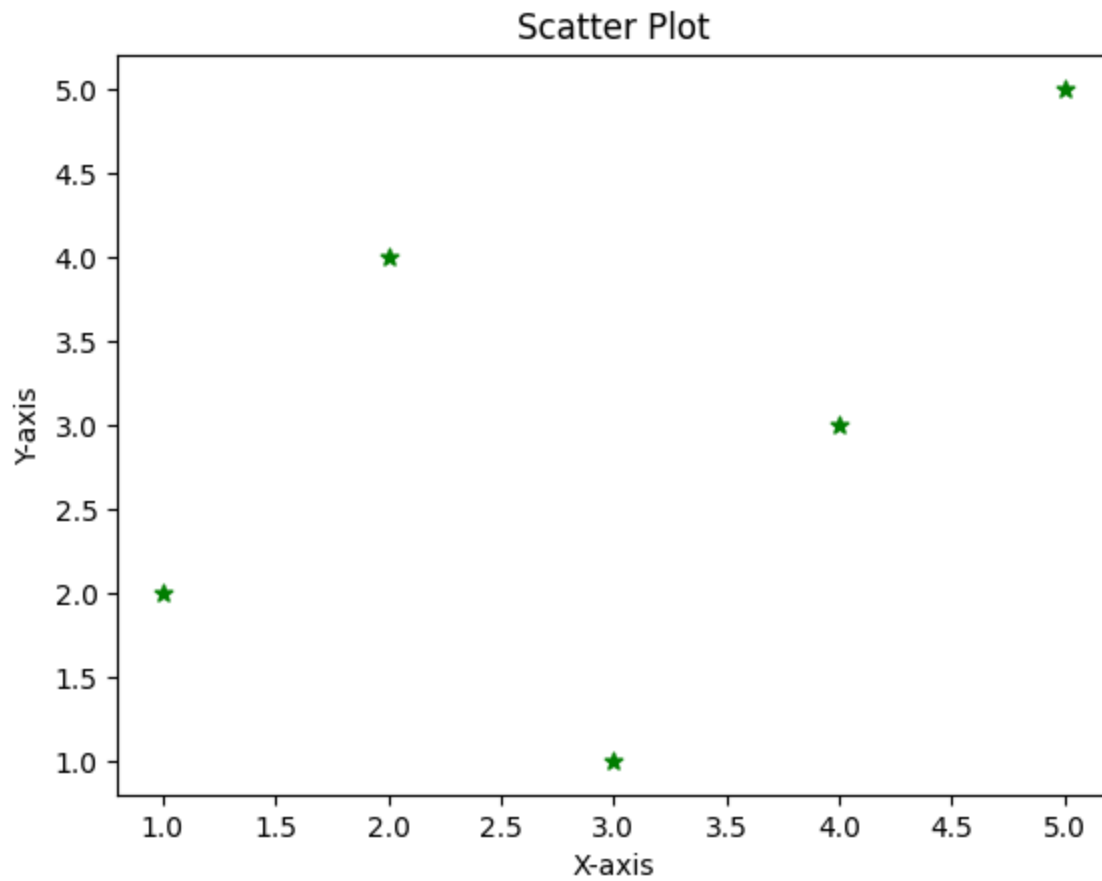


```
In [7]: import matplotlib.pyplot as plt
import seaborn as sns

# Sample data use for: Correlation check karne ke Liye, Outliers (ajeeb data points)
# Patterns ya trends dhoondhne ke Liye, Relationship dekhne ke Liye
x = [1, 2, 3, 4, 5]
y = [2, 4, 1, 3, 5]

# Create a scatter plot
plt.scatter(x, y, s=40, color="green", marker="*") # s is the size of the points
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.title("Scatter Plot")
plt.show()

# Or using Seaborn
sns.scatterplot(x=x, y=y)
plt.show()
```

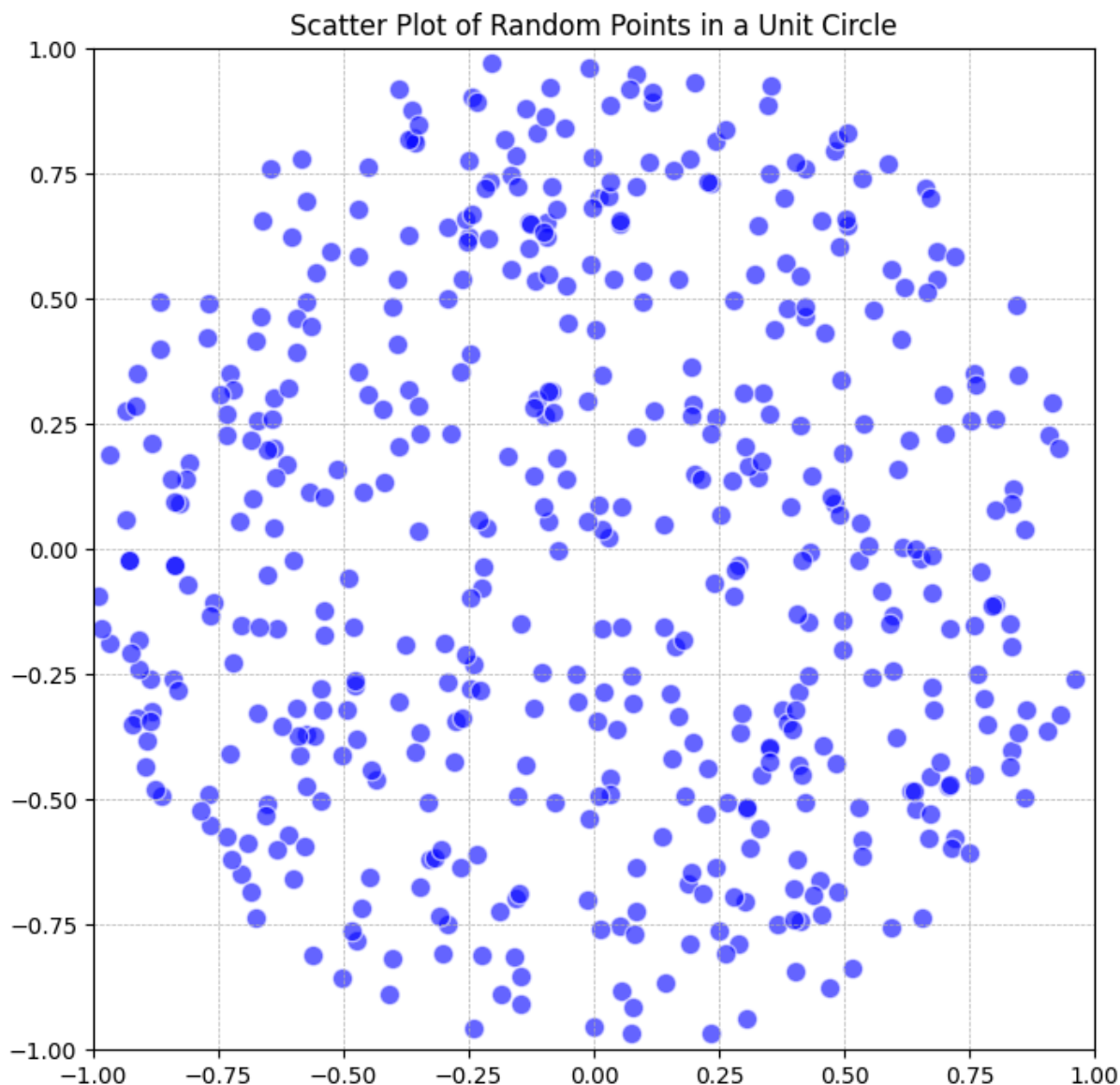


Scatter plot unit Circle

```
In [4]: import numpy as np
import matplotlib.pyplot as plt
## Generate random points within a unit circle
num_points = 500
angles = np.random.uniform(0, 2 * np.pi, num_points) # Random angles
radii = np.sqrt(np.random.uniform(0, 1, num_points)) # Random radii
x_points = radii * np.cos(angles)
y_points = radii * np.sin(angles)
## Plot the points
plt.figure(figsize=(8, 8))
plt.scatter(x_points, y_points, c='blue', alpha=0.6, edgecolors='w', s=80)
# Blue points with white edges and transparency
plt.title("Scatter Plot of Random Points in a Unit Circle")
plt.xlim(-1, 1)
plt.ylim(-1, 1)
plt.gca().set_aspect("equal", adjustable="box")

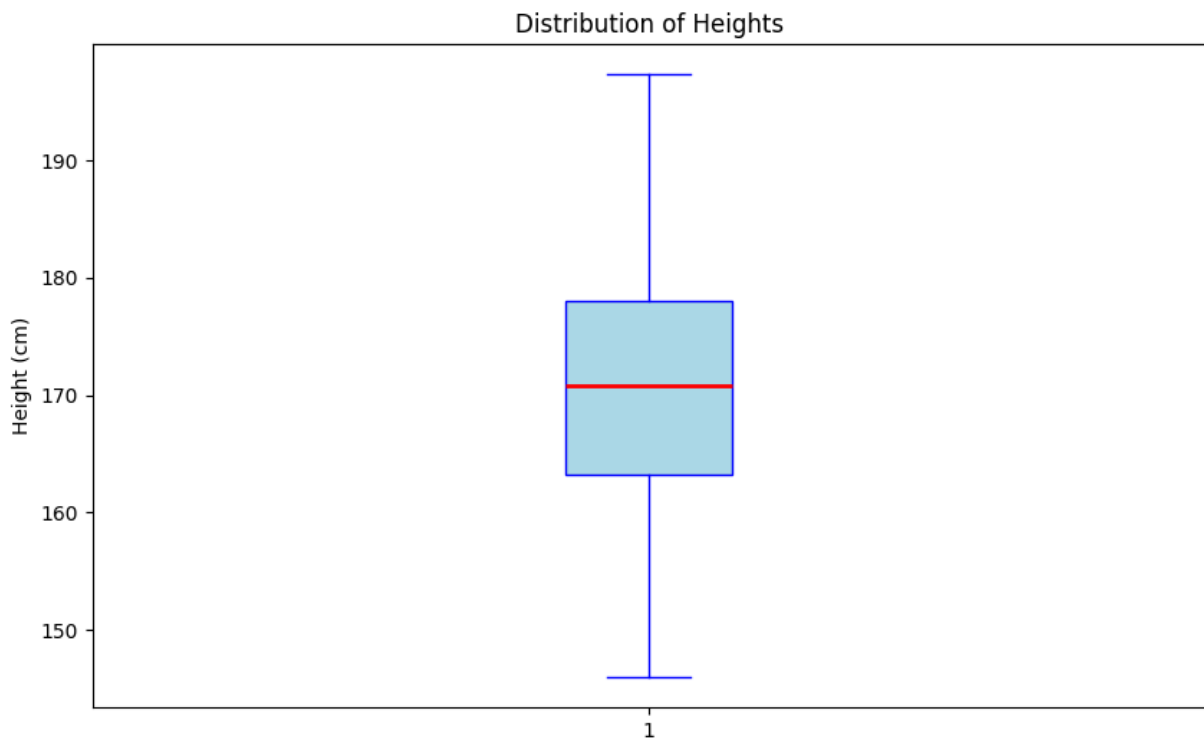
plt.grid(True, which='both', linestyle='--', linewidth=0.5)

plt.savefig("scatter_plot_unit_circle.png")
```



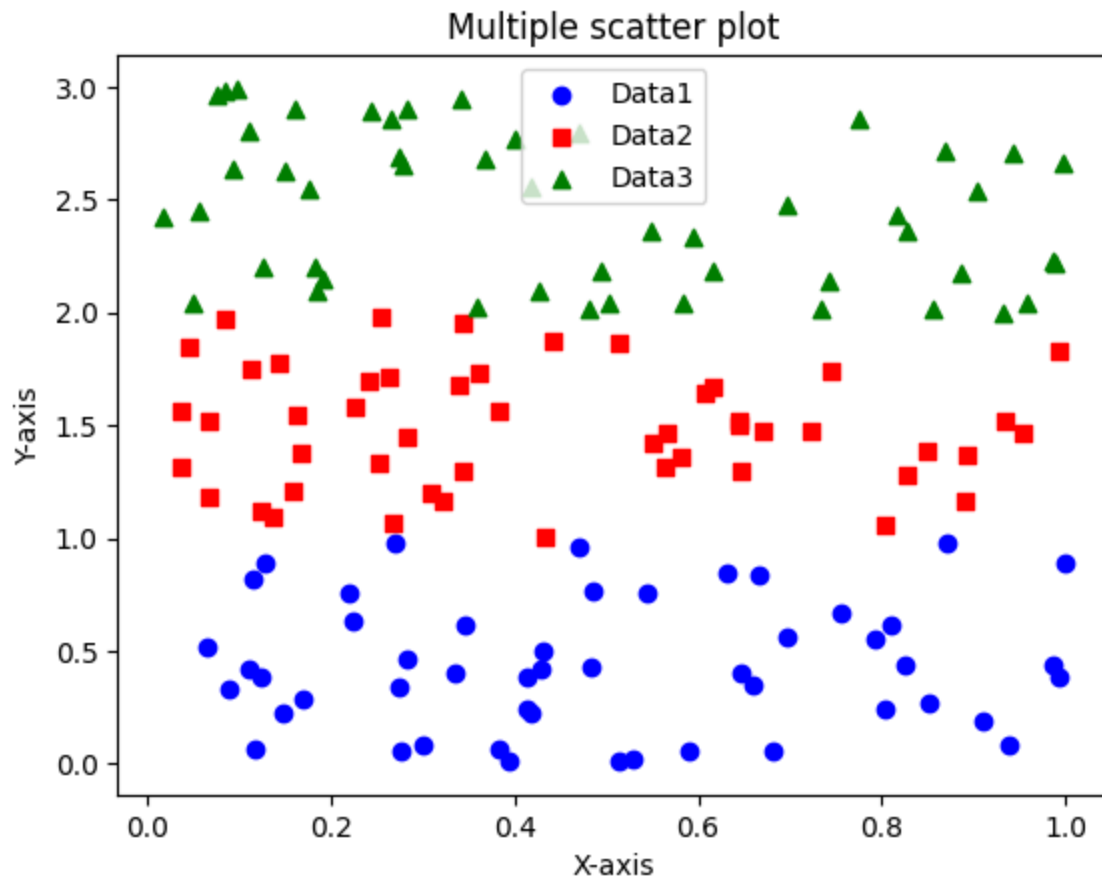
Box plot Height Distribution

```
In [5]: import numpy as np
import matplotlib.pyplot as plt
#Generating random height data
heights = np.random.normal(loc=170, scale=10, size=50)
#Creating the box plot
plt.figure(figsize=(10,6))
plt.boxplot(heights, patch_artist=True,
boxprops=dict(facecolor='lightblue', color='blue'),
medianprops=dict(color='red', linewidth=2),
whiskerprops=dict(color='blue'),
capprops=dict(color='blue'),
flierprops=dict(marker='o', color='red', alpha=0.5))
plt.title("Distribution of Heights")
plt.ylabel("Height (cm)")
plt.savefig("box_plot_heights_distribution.png")
```

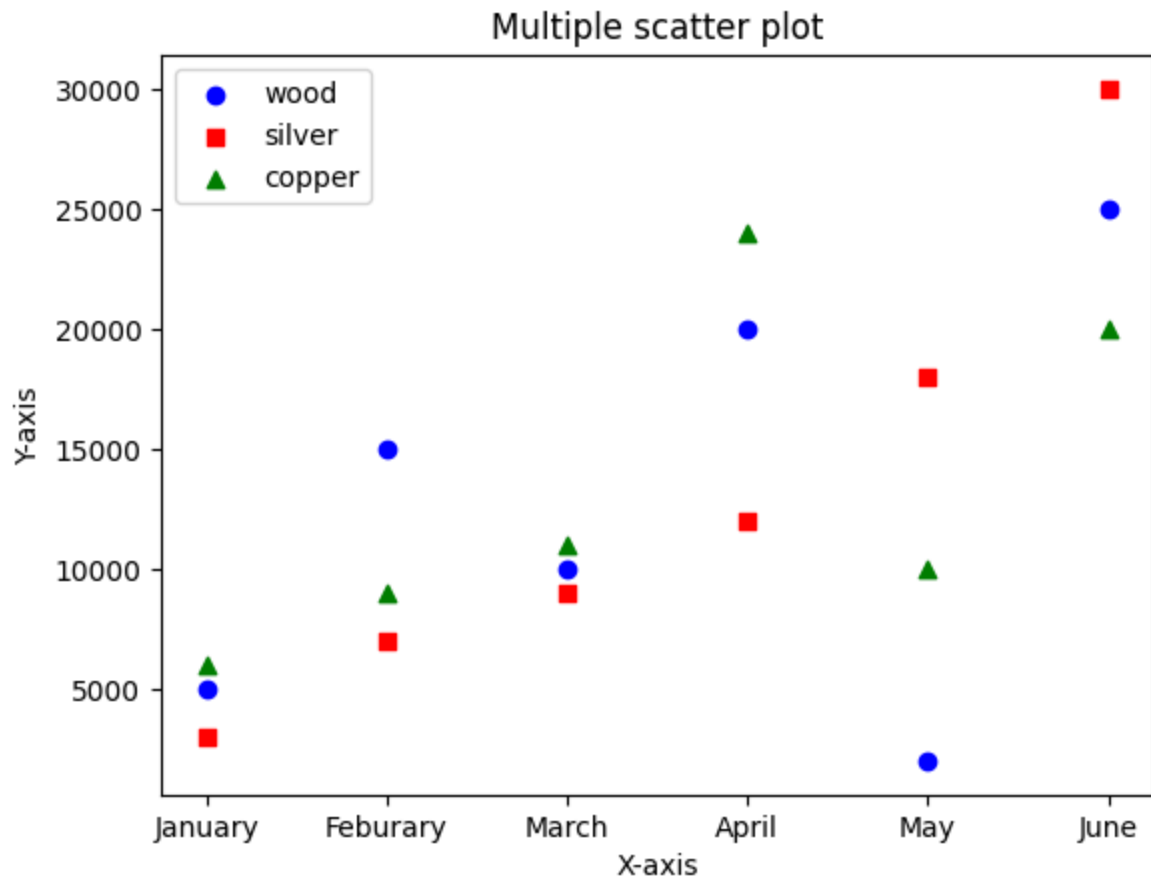


Multiple Scatter plot

```
In [8]: import matplotlib.pyplot as plt
import numpy as np
x1=np.random.rand(50)
y1=np.random.rand(50)
x2=np.random.rand(50)
y2=np.random.rand(50)+1 #shifted for visibility
x3=np.random.rand(50)
y3=np.random.rand(50)+2 #shifted for visibility
plt.scatter(x1,y1,label='Data1',marker='o',color='blue')
plt.scatter(x2,y2,label='Data2',marker='s',color='red')
plt.scatter(x3,y3,label='Data3',marker='^',color='green')
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.title("Multiple scatter plot")
plt.legend()
plt.show()
```

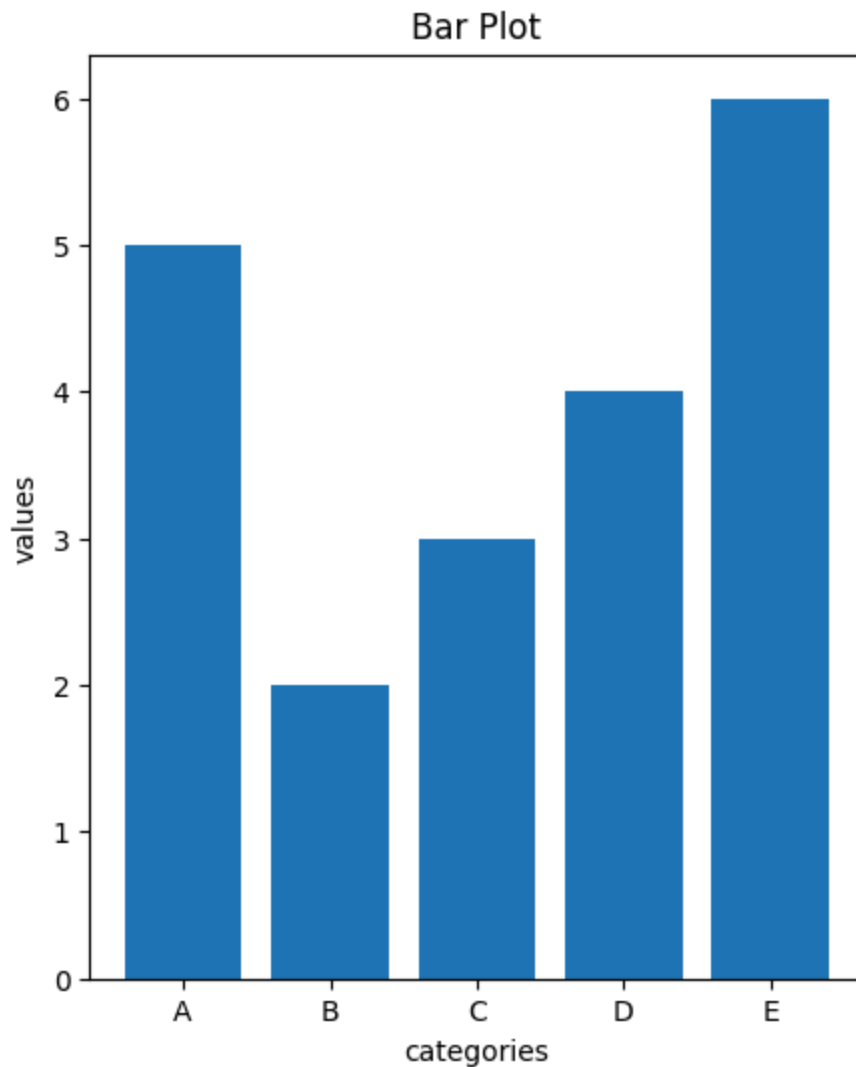


```
In [9]: import matplotlib.pyplot as plt
import numpy as np
Months = ['January', 'February', 'March', 'April', 'May', 'June']
Sale1 = [5000, 15000, 10000, 20000, 2000, 25000]
Sale2 = [3000, 7000, 9000, 12000, 18000, 30000]
Sale3 = [6000, 9000, 11000, 24000, 10000, 20000]
plt.scatter(Months, Sale1, label='wood', marker='o', color='blue')
plt.scatter(Months, Sale2, label='silver', marker='s', color='red')
plt.scatter(Months, Sale3, label='copper', marker='^', color='green')
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.title("Multiple scatter plot")
plt.legend()
plt.show()
```



Bar Chart

```
In [10]: import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import numpy as np
categories=["A","B","C","D","E"]
values=[5,2,3,4,6]
plt.figure(figsize=(5,6))
plt.bar(categories,values)
plt.xlabel("categories")
plt.ylabel("values")
plt.title("Bar Plot")
plt.show()
```

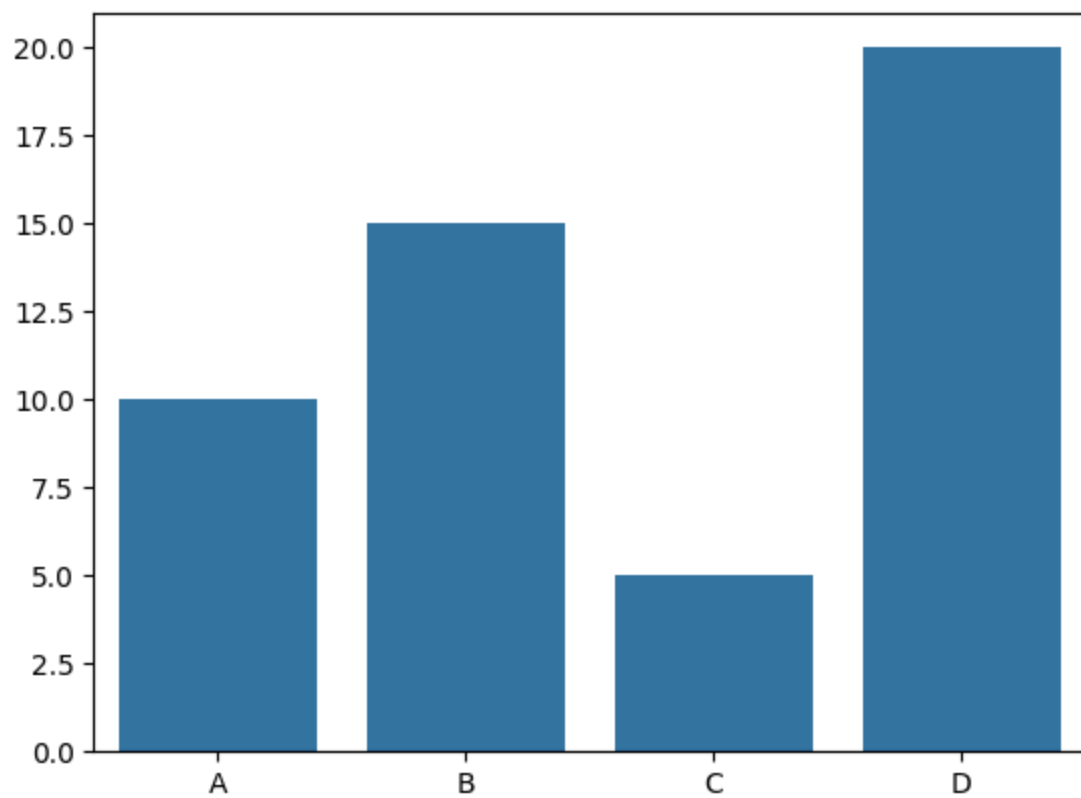
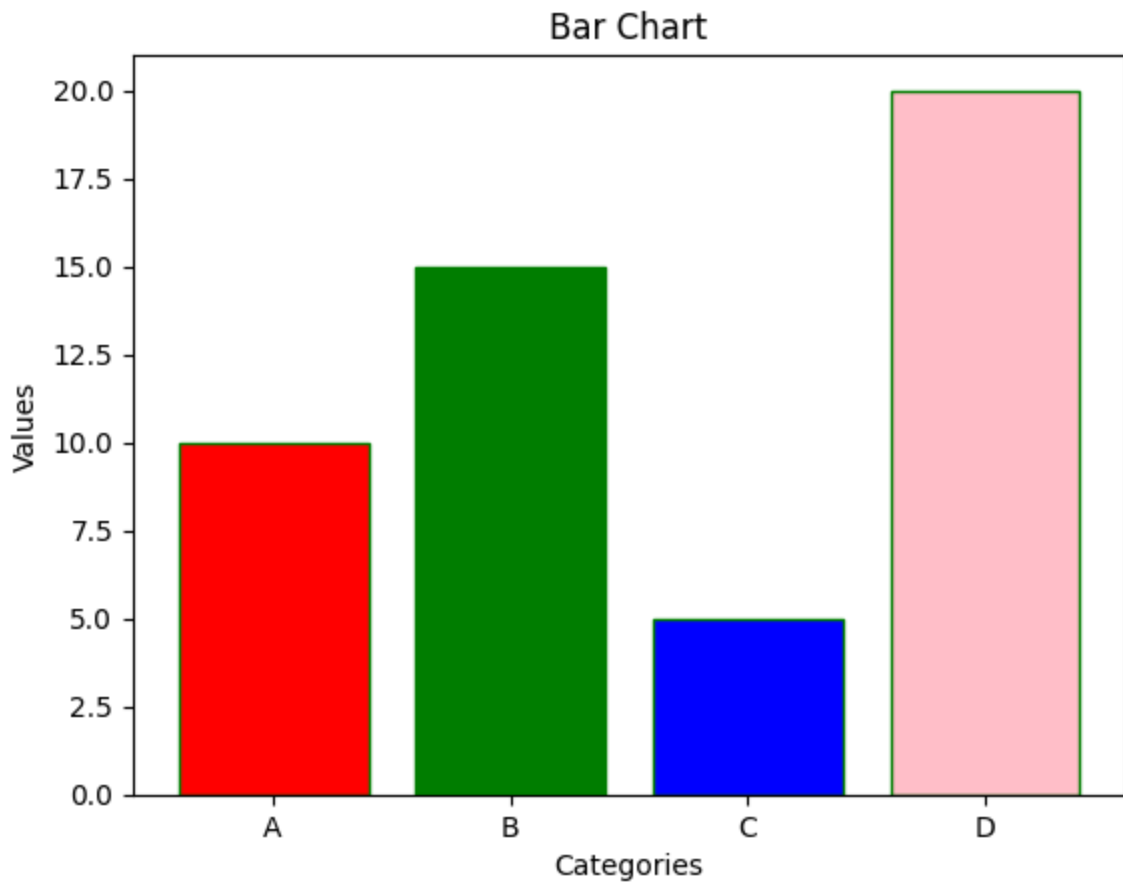


```
In [11]: import matplotlib.pyplot as plt
import seaborn as sns

# Sample data use for: Frequency dikhani ho, Compare karna ho, Categorical data ke li
categories = ['A', 'B', 'C', 'D']
values = [10, 15, 5, 20]
bar_colors=['red', 'green', 'blue', 'pink']

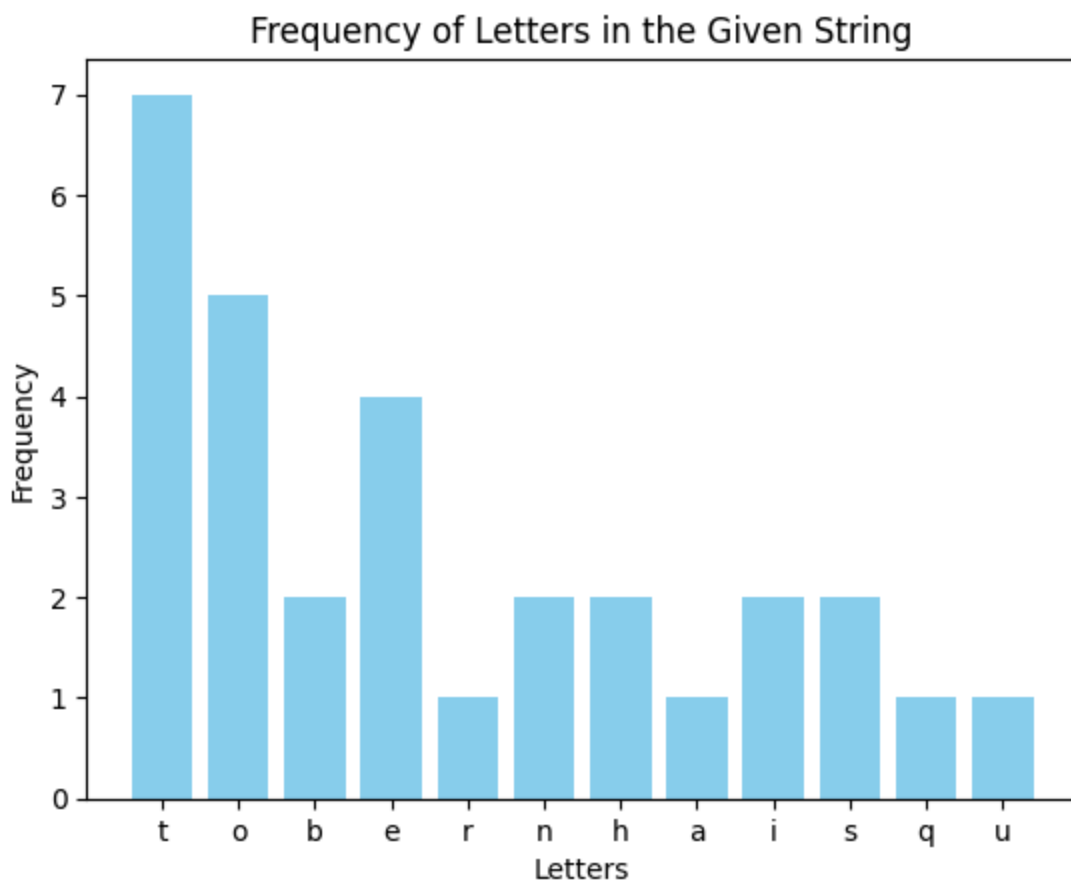
# Create a bar chart
plt.bar(categories, values, color=bar_colors, edgecolor='green')
plt.xlabel("Categories")
plt.ylabel("Values")
plt.title("Bar Chart")
plt.show()

# Or using Seaborn
sns.barplot(x=categories, y=values)
plt.show()
```



Bar Chart Letter Frequency

```
In [3]: import collections
import matplotlib.pyplot as plt
import numpy as np
import string
data = "To be, or not to be, that is the question."
data = data.lower()
data = ''.join(filter(lambda x: x in string.ascii_lowercase, data))
counter = collections.Counter(data)
letters, counts = zip(*counter.items())
indices = np.arange(len(letters))
plt.bar(indices, counts, color='skyblue')
plt.xticks(indices, letters)
plt.xlabel('Letters')
plt.ylabel('Frequency')
plt.title('Frequency of Letters in the Given String')
plt.savefig("bar_chart_letter_frequency.png")
```



Multiple Bar Chart

```
In [12]: import matplotlib.pyplot as plt
import numpy as np

# Sample data
groups = ['Group A', 'Group B', 'Group C']
categories = ['Category 1', 'Category 2', 'Category 3']
values_group_a = [23, 12, 34]
```

```

values_group_b = [15, 28, 20]
values_group_c = [31, 18, 25]

# Set the width of each bar
bar_width = 0.25

# Calculate the position of each bar on the x-axis
x_pos_group_a = np.arange(len(categories))
x_pos_group_b = [x + bar_width for x in x_pos_group_a]
x_pos_group_c = [x + bar_width for x in x_pos_group_b]

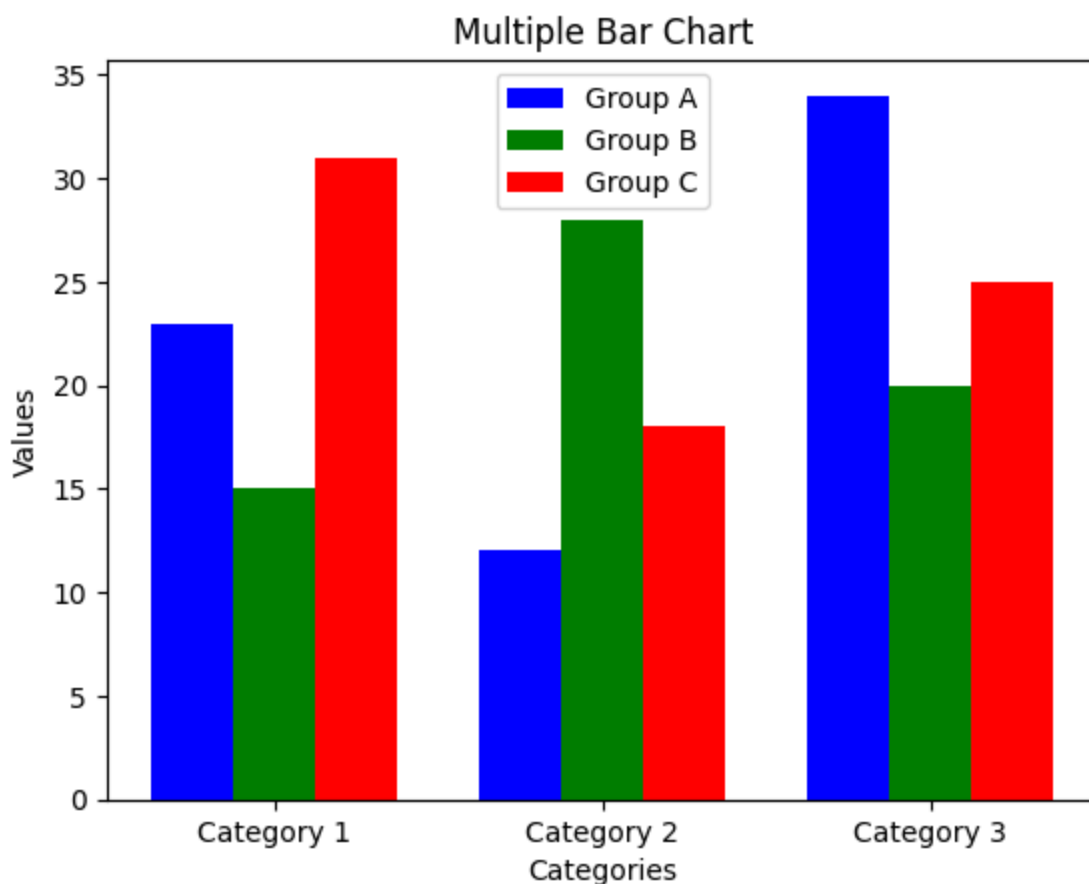
# Create the bar chart
plt.bar(x_pos_group_a, values_group_a, color='blue', width=bar_width, label='Group A')
plt.bar(x_pos_group_b, values_group_b, color='green', width=bar_width, label='Group B')
plt.bar(x_pos_group_c, values_group_c, color='red', width=bar_width, label='Group C')

# Set the x-axis tick positions and labels
plt.xticks([x + bar_width for x in range(len(categories))], categories)

# Add labels, title, and legend
plt.xlabel("Categories")
plt.ylabel("Values")
plt.title("Multiple Bar Chart")
plt.legend()

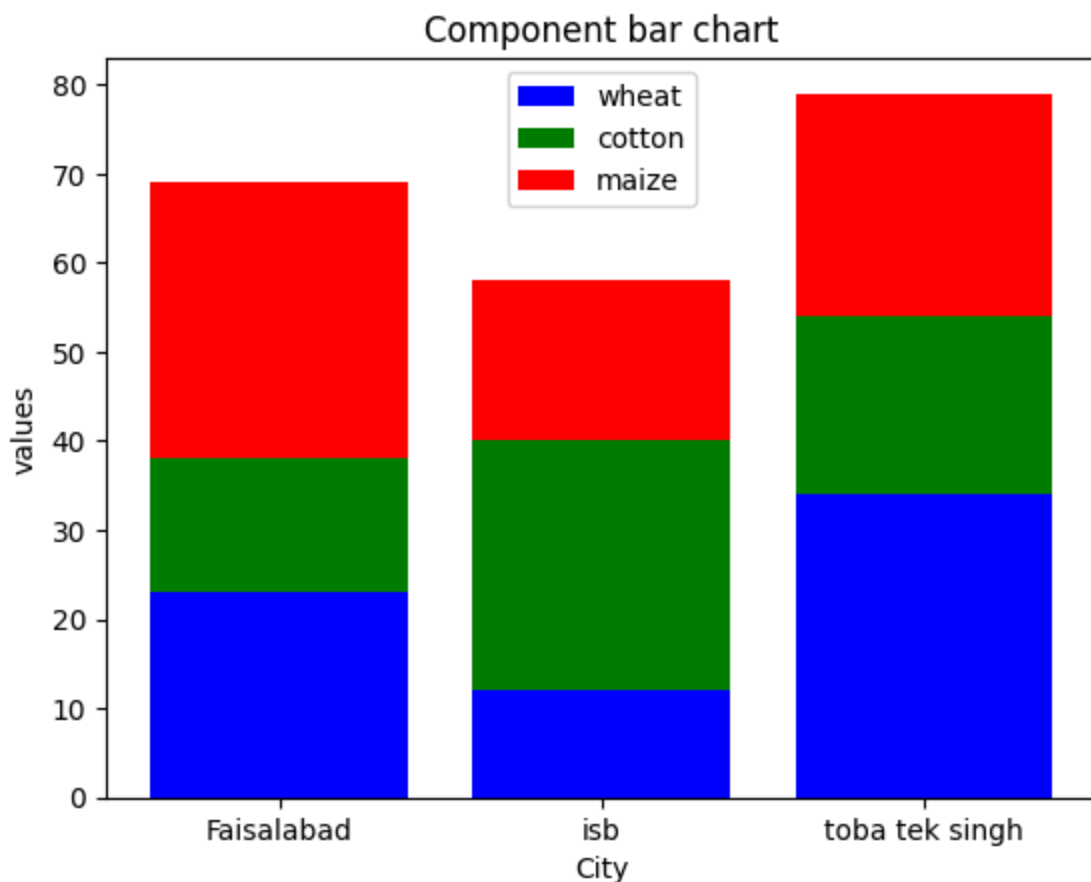
# Show the plot
plt.show()

```



Component Bar Chart

```
In [13]: import matplotlib.pyplot as plt
import numpy as np
#sample data
City=['Faisalabad','isb','toba tek singh']
wheat=[23,12,34]
cotton=[15,28,20]
maize=[31,18,25]
#create a component bar chart
plt.bar(City,wheat,color='blue',label='wheat')
plt.bar(City,cotton,bottom=wheat,color='green',label='cotton')
plt.bar(City,maize,bottom=np.array(wheat)+np.array(cotton),color='red',label='maize')
#add Label ,title and Legend
plt.xlabel("City")
plt.ylabel("values")
plt.title("Component bar chart")
plt.legend()
plt.show()
```



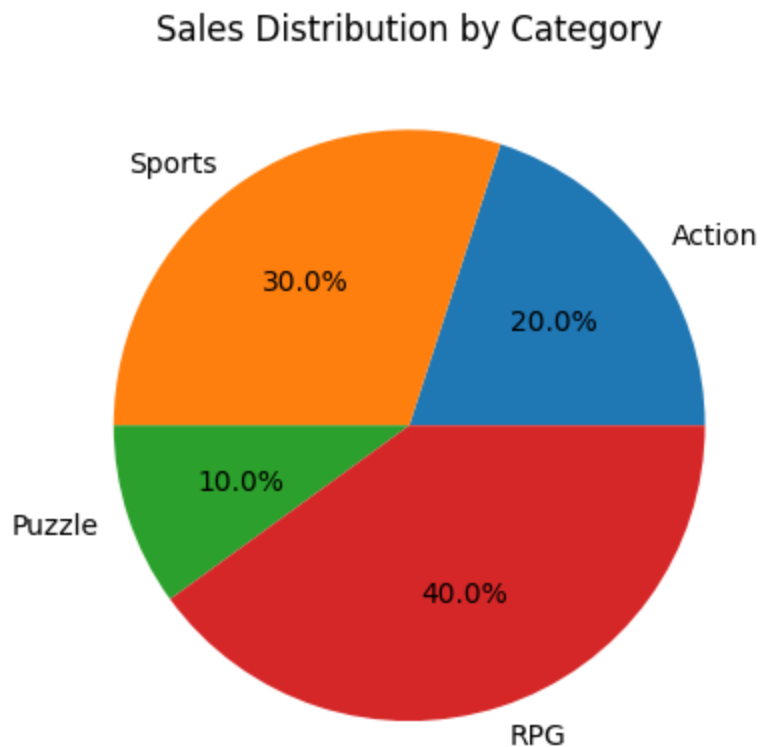
Pie Chart

```
In [14]: import pandas as pd
import matplotlib.pyplot as plt
```

```
# Create your own data
data = {
    'Category': ['Action', 'Sports', 'Puzzle', 'RPG'],
    'Sales': [100, 150, 50, 200]
}

file = pd.DataFrame(data)

# Plot
plt.pie(file['Sales'], labels=file['Category'], autopct='%1.1f%%')
plt.title('Sales Distribution by Category')
plt.show()
```



explode one slice in pie chart

```
In [2]: import matplotlib.pyplot as plt

labels = ["A", "B", "C", "D"]
sizes = [10, 20, 30, 40]
colors = ["gold", "yellowgreen", "lightcoral", "lightskyblue"]
explode = [0.2, 0, 0, 0]

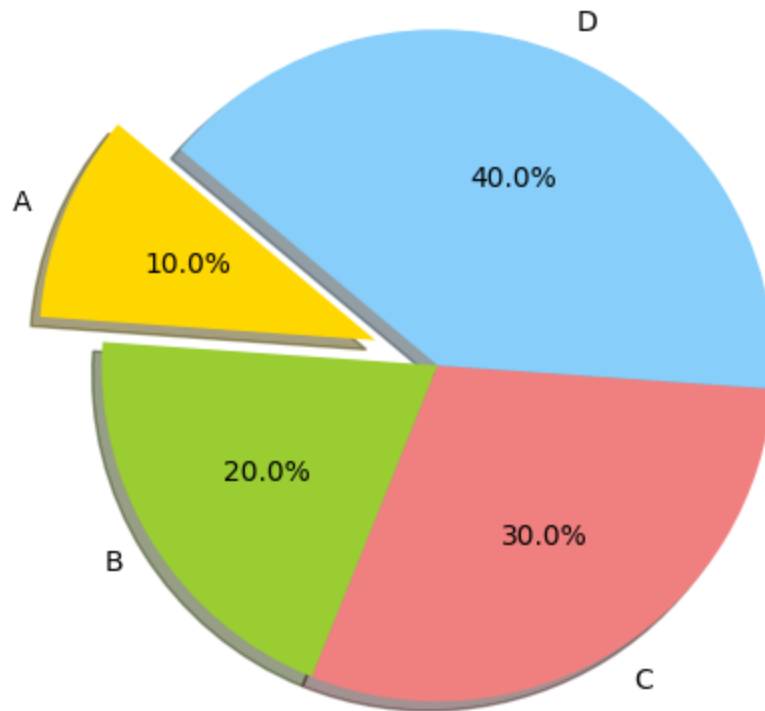
plt.pie(
    sizes,
    labels=labels,
    colors=colors,
    explode=explode,
    autopct='%1.1f%%',
```

```

        shadow=True,
        startangle=140
    )

plt.axis('equal')
plt.show()

```



```

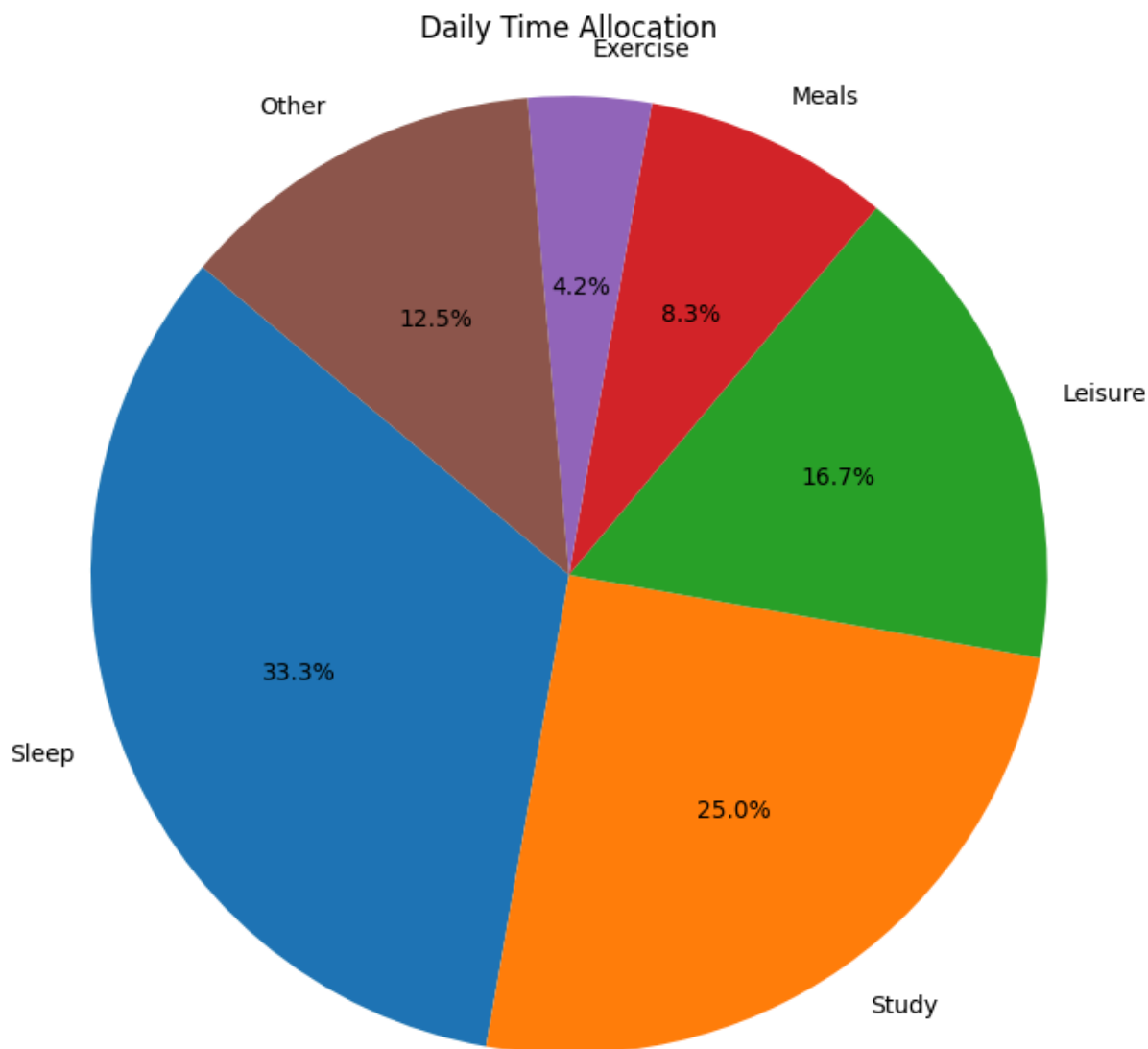
In [15]: import matplotlib.pyplot as plt

# Data
activities = ['Sleep', 'Study', 'Leisure', 'Meals', 'Exercise', 'Other']
hours = [8, 6, 4, 2, 1, 3]

# Create pie chart
plt.figure(figsize=(8, 8))
plt.pie(hours, labels=activities, autopct='%1.1f%%', startangle=140)
plt.title('Daily Time Allocation')
plt.axis('equal') # Equal aspect ratio ensures that pie is drawn as a circle.

# Show plot
plt.show()

```

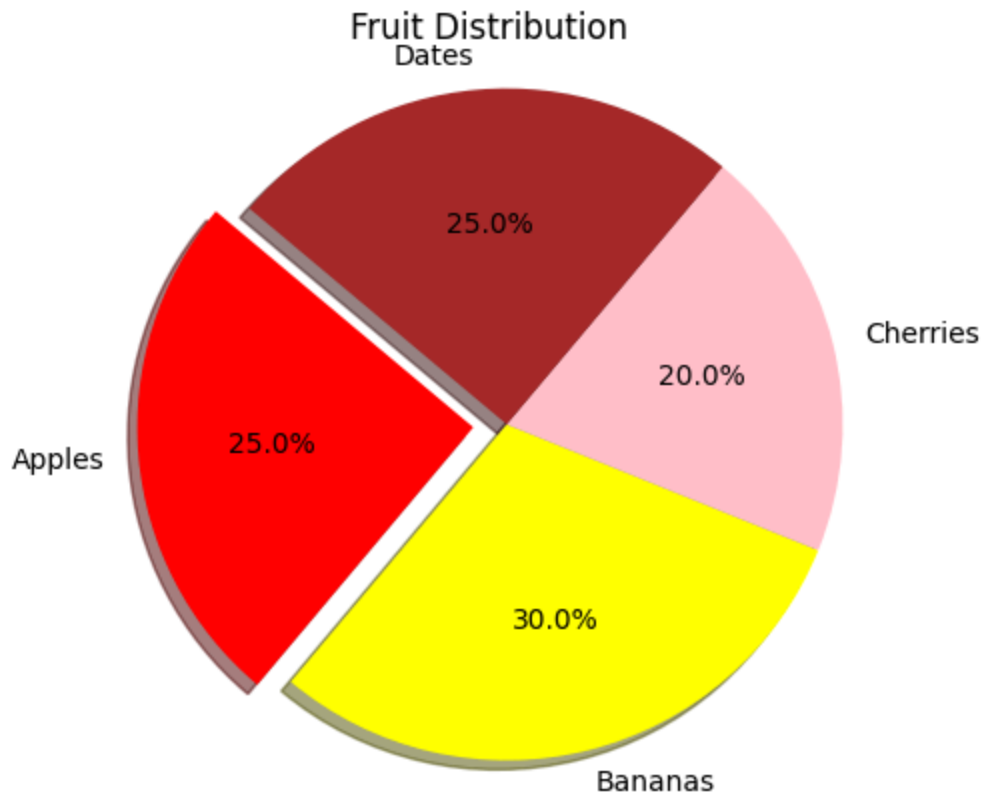


```
In [16]: import matplotlib.pyplot as plt

# Data to plot
labels = ['Apples', 'Bananas', 'Cherries', 'Dates']
sizes = [25, 30, 20, 25] # percentage values
colors = ['red', 'yellow', 'pink', 'brown']
explode = (0.1, 0, 0, 0) # "explode" the 1st slice

# Create pie chart
plt.pie(sizes, labels=labels, colors=colors, explode=explode,
        autopct='%1.1f%%', shadow=True, startangle=140)

plt.title('Fruit Distribution')
plt.axis('equal') # Equal aspect ratio ensures the pie chart is circular.
plt.show()
```



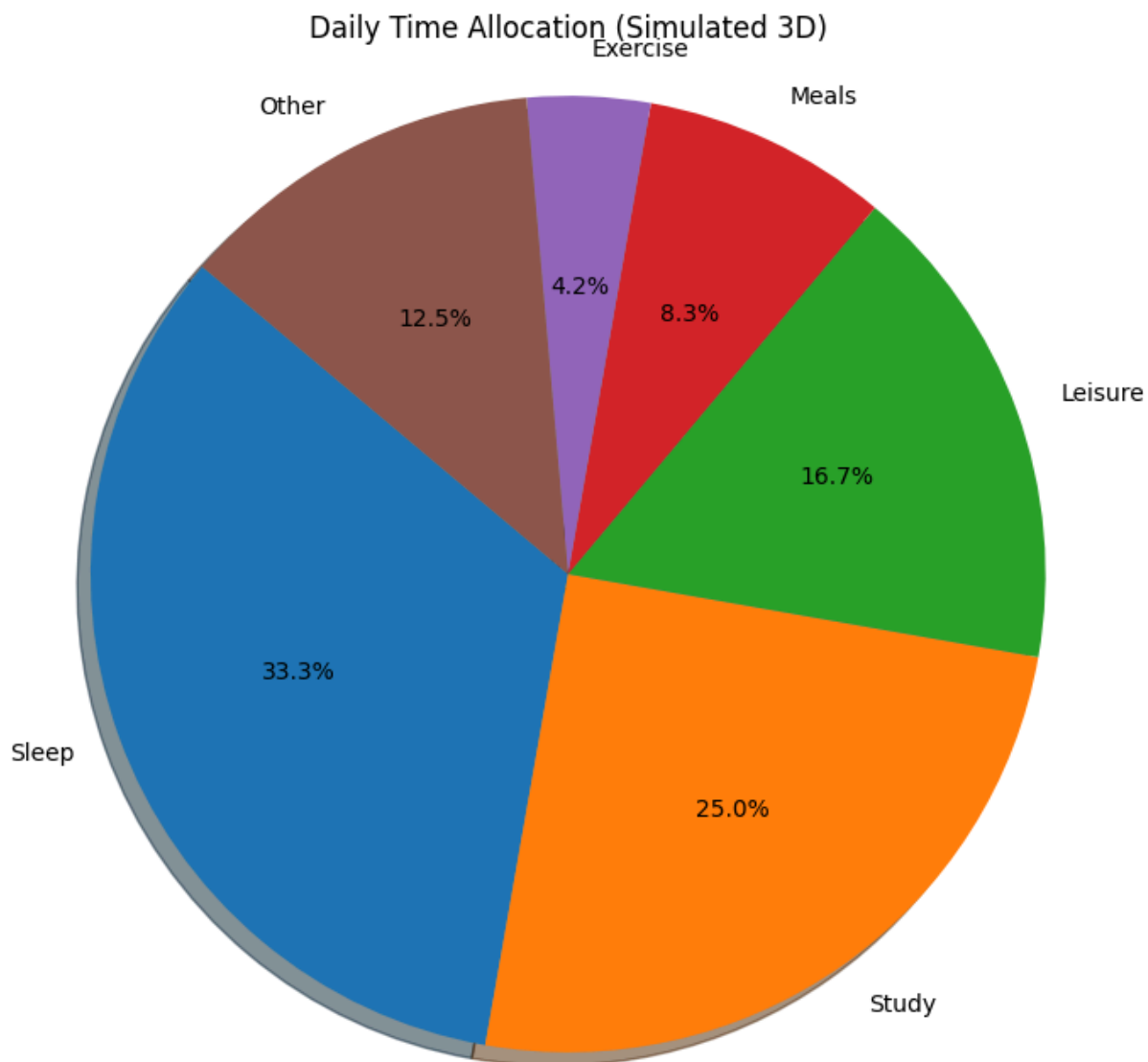
3D pie

```
In [17]: import matplotlib.pyplot as plt

# Data
activities = ['Sleep', 'Study', 'Leisure', 'Meals', 'Exercise', 'Other']
hours = [8, 6, 4, 2, 1, 3]

# Create pie chart with shadow to simulate 3D
plt.figure(figsize=(8, 8))
plt.pie(hours, labels=activities, autopct='%1.1f%%', startangle=140, shadow=True)
plt.title('Daily Time Allocation (Simulated 3D)')
plt.axis('equal')

plt.show()
```



```
In [18]: import plotly.express as px

# Data
activities = ['Sleep', 'Study', 'Leisure', 'Meals', 'Exercise', 'Other']
hours = [8, 6, 4, 2, 1, 3]

# Create pie chart
fig = px.pie(names=activities, values=hours, title='Daily Time Allocation (3D Style',
             hole=0.2) # hole=0.2 gives it a 3D-like appearance
fig.update_traces(textinfo='percent+label', pull=[0.05]*6)

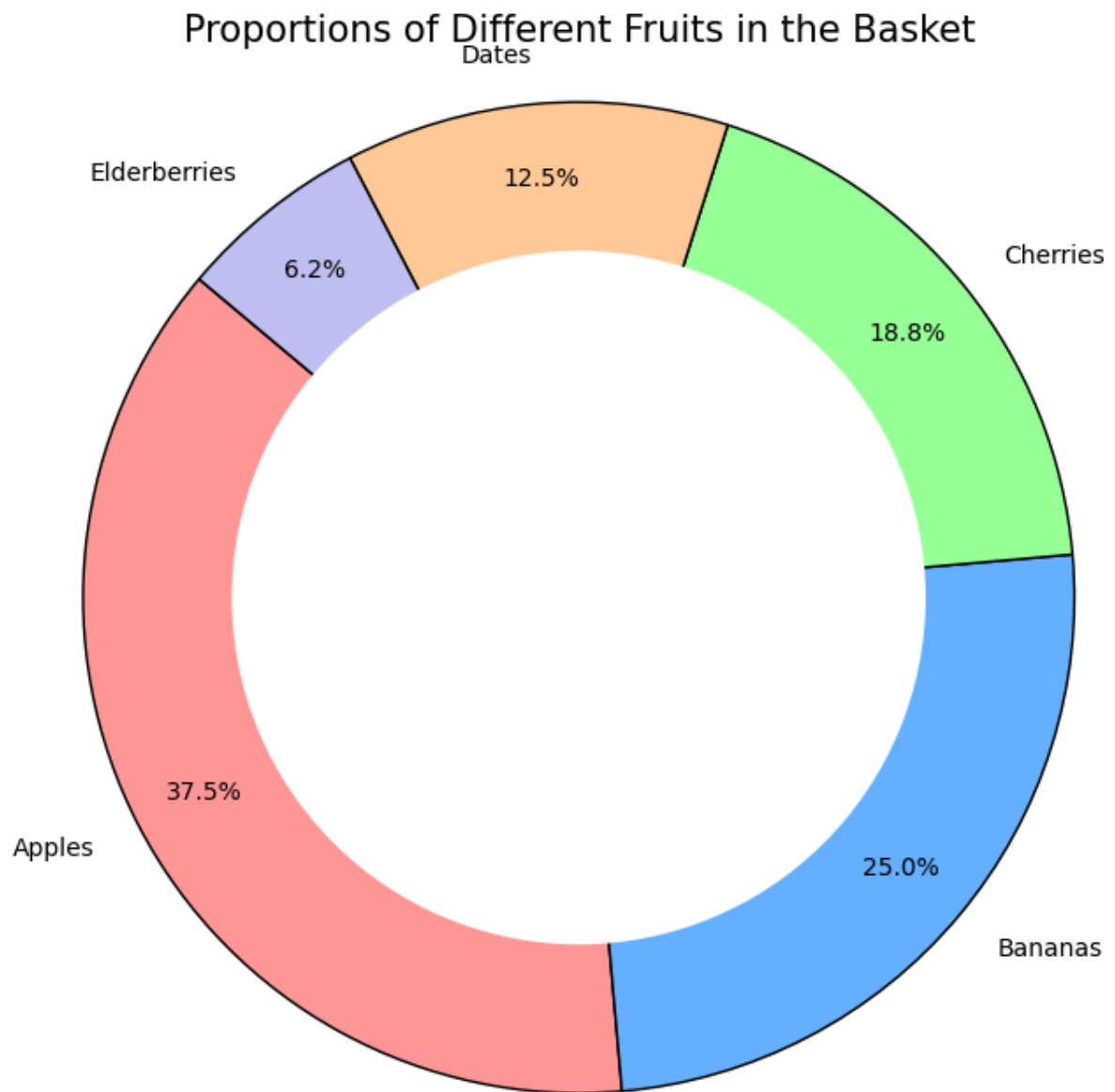
fig.show()
```

Daily Time Allocation (3D Style)



Pie Dounot Chart

```
In [2]: import matplotlib.pyplot as plt
fruits = ["Apples", "Bananas", "Cherries", "Dates", "Elderberries"]
quantities = [30, 20, 15, 10, 5]
colors = ["#ff9999", "#66b3ff", "#99ff99", "#ffcc99", "#c2c2f0"]
plt.figure(figsize=(8, 8))
plt.pie(quantities, labels=fruits, colors=colors, autopct="%1.1f%%",
startangle=140, pctdistance=0.85, wedgeprops={'edgecolor': 'black'})
centre_circle = plt.Circle((0, 0), 0.70, fc="white")
fig = plt.gcf()
fig.gca().add_artist(centre_circle)
plt.title("Proportions of Different Fruits in the Basket", fontsize=15)
plt.axis("equal")
plt.savefig("fruit_basket_proportions.png")
```



Box plot

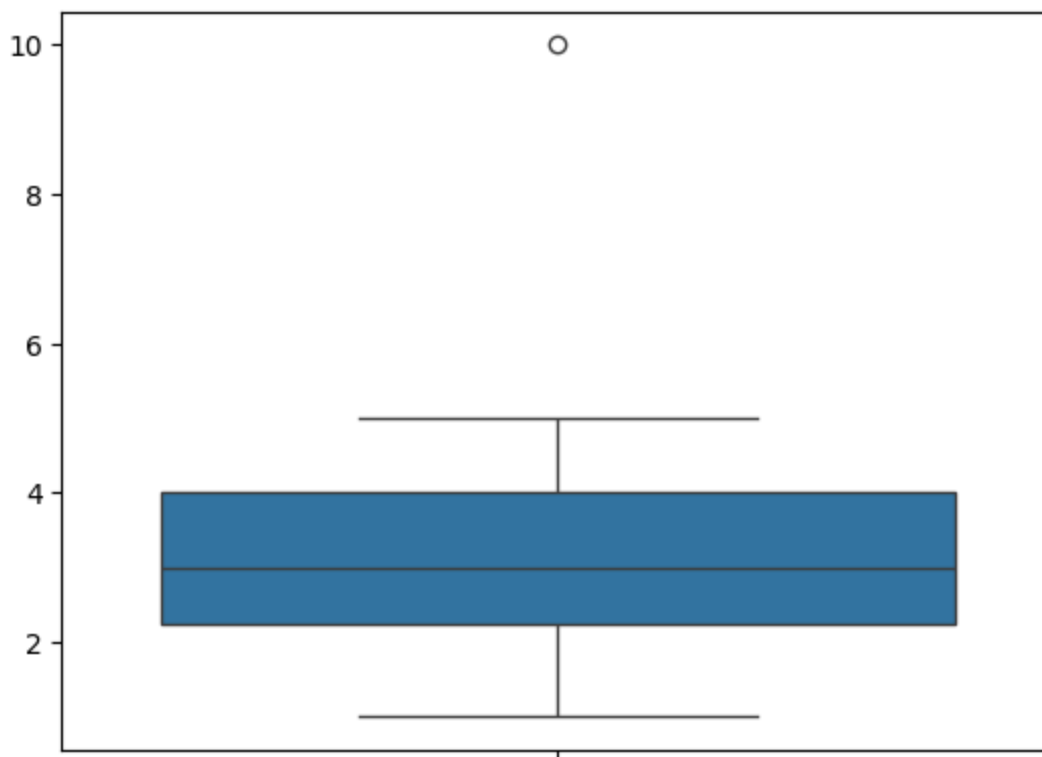
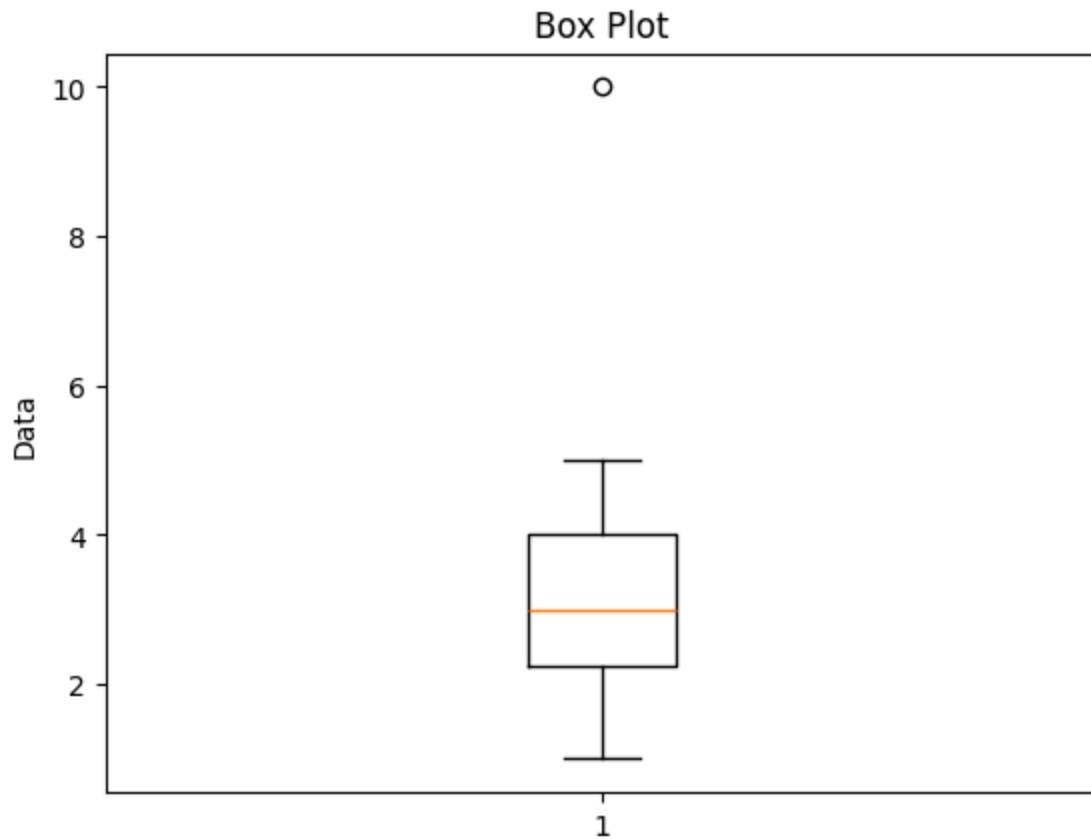
```
In [19]: import matplotlib.pyplot as plt
import seaborn as sns

# Sample data
data = [1, 2, 2, 3, 3, 3, 4, 4, 5, 10] # 10 is an outlier

# Create a box plot
plt.boxplot(data)
plt.ylabel("Data")
plt.title("Box Plot")
plt.show()

# Or using Seaborn
```

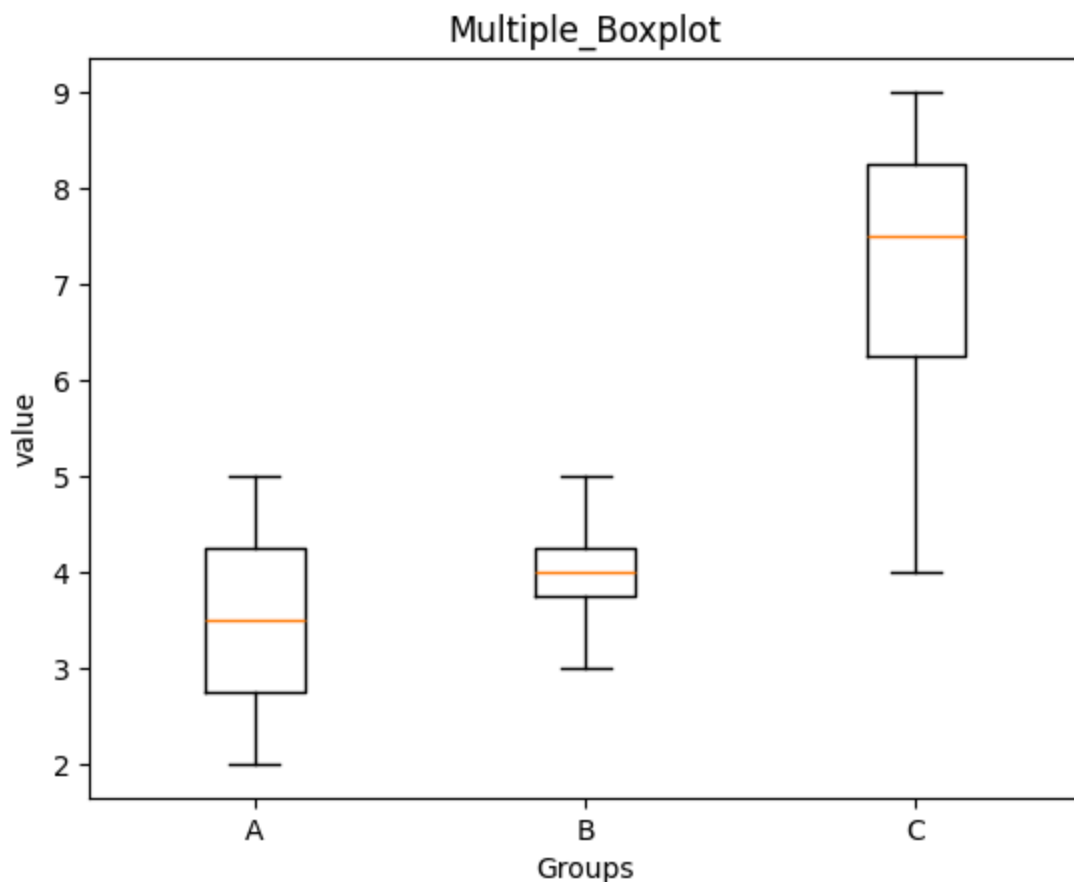
```
sns.boxplot(y=data) # Use 'y' for vertical orientation  
plt.show()
```



multiple box plot

```
In [20]: import matplotlib.pyplot as plt
import seaborn as sns
data_a = [2,3,4,5]
data_b = [3,4,4,5]
data_c = [4,7,8,9]
labels = ['A','B','C']
data = [data_a, data_b, data_c]

plt.boxplot(data, tick_labels=labels)# delta che da kam tick lafzl dee da sirf jptr
plt.title('Multiple_Boxplot')
plt.xlabel('Groups')
plt.ylabel('value')
plt.show()
```



Multiple Boxplot in Seaborn

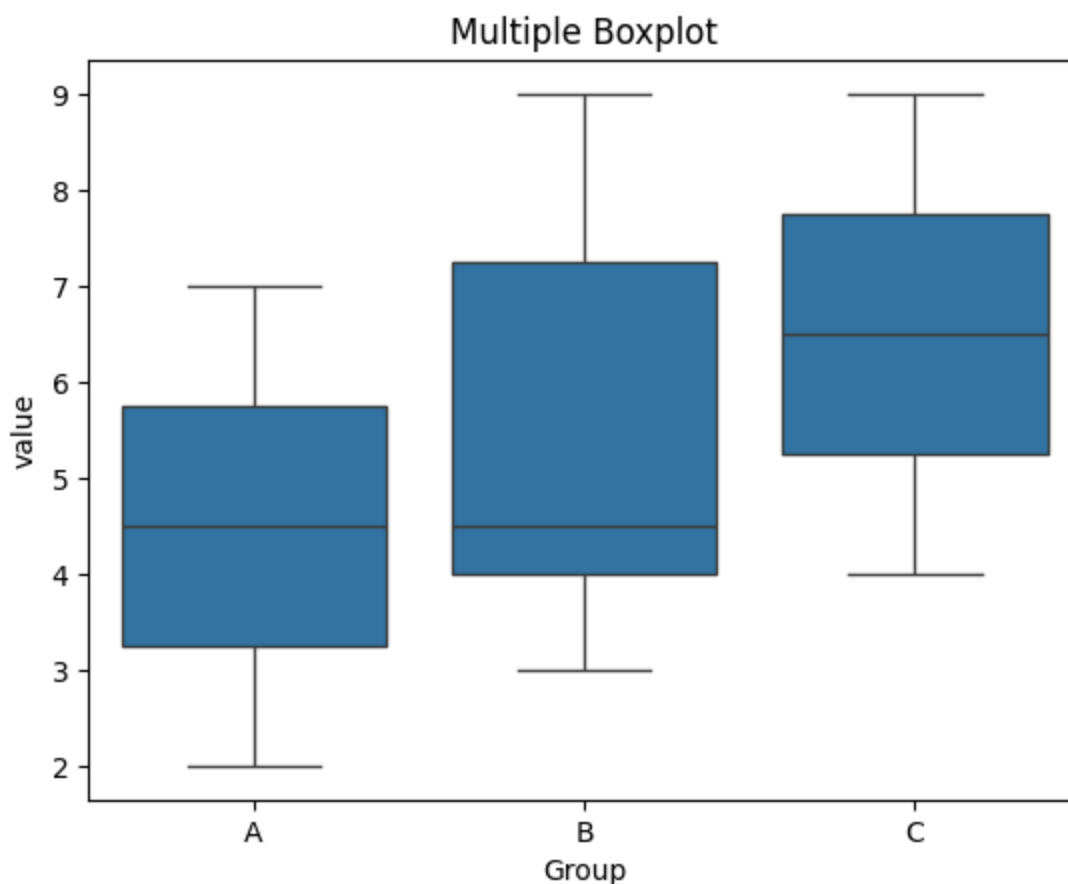
```
In [21]: import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

data_a = [2, 3, 4, 5, 6, 7]
data_b = [3, 4, 4, 5, 8, 9]
data_c = [4, 7, 8, 9, 5, 6]

# Create a DataFrame
data = pd.DataFrame({
```

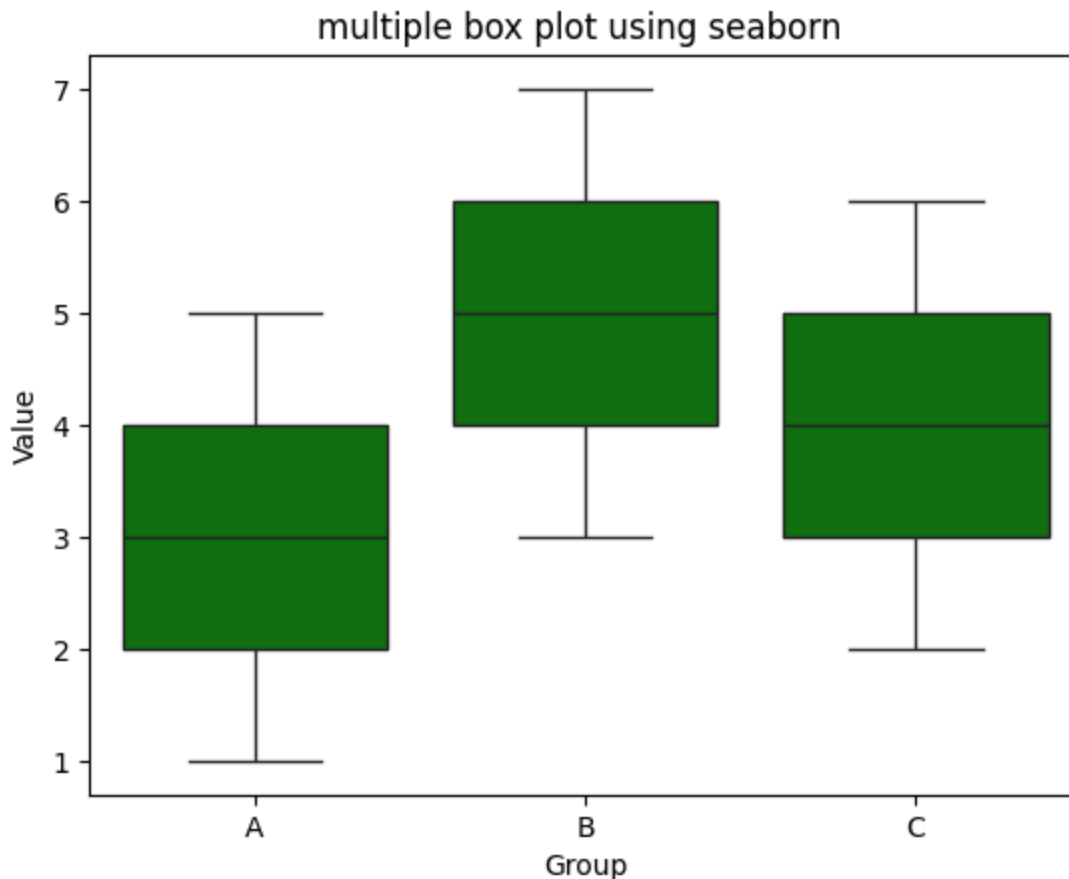
```
'Group': ['A'] * len(data_a) + ['B'] * len(data_b) + ['C'] * len(data_c),
'value': data_a + data_b + data_c
})

# Create the boxplot
sns.boxplot(x='Group', y='value', data=data)
plt.title('Multiple Boxplot')
plt.show()
```



multiple boxplot use eaborn

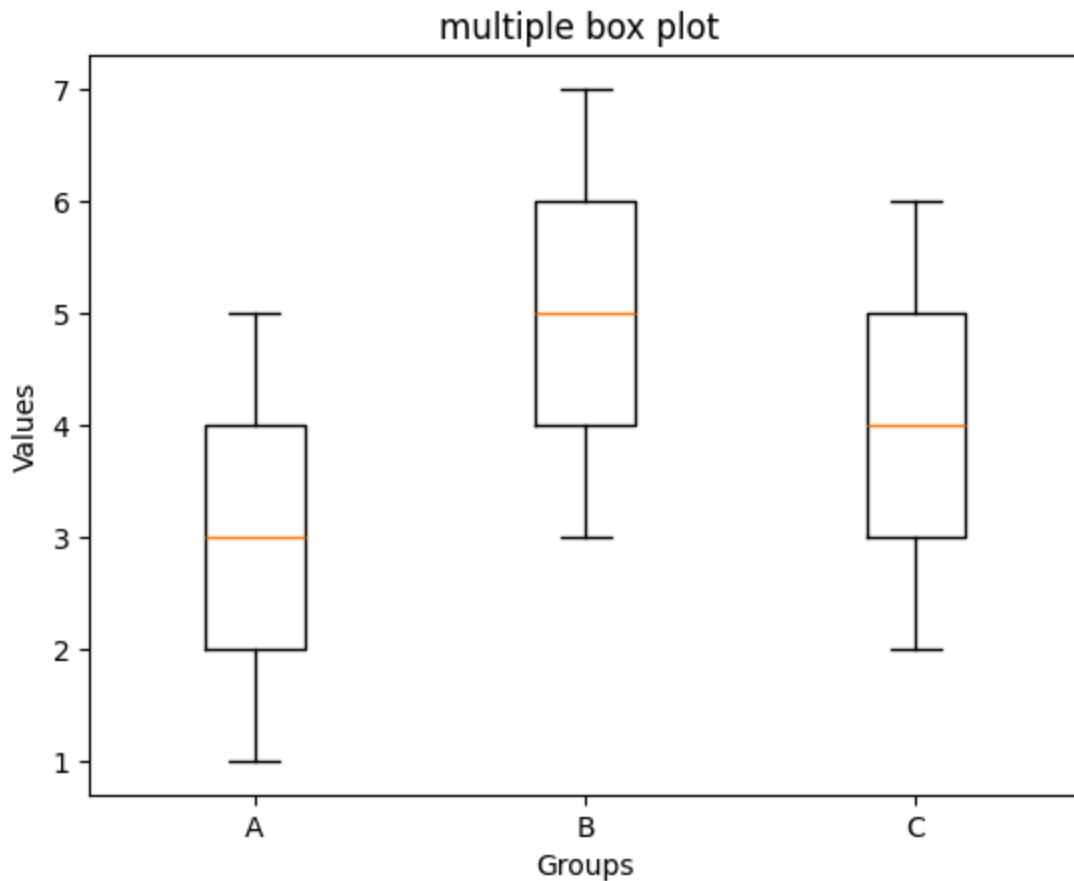
```
In [22]: import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
#sample data
data_a=[1,2,3,4,5];data_b=[3,4,5,6,7]
data_c=[2,3,4,5,6]
#Create a pandas data frame
data=pd.DataFrame({
    'Group':['A']*len(data_a)+['B']*len(data_b)+['C']*len(data_c), 'Value':data_a+da
})
#create the box plot
sns.boxplot(x='Group',y='Value',data=data,color='green')
plt.title("multiple box plot using seaborn ")
plt.show()
```



```
In [23]: import matplotlib.pyplot as plt
import seaborn as sns
#sample data
data_a=[1,2,3,4,5];data_b=[3,4,5,6,7]
data_c=[2,3,4,5,6]
#combine data into a list of lists
data=[data_a,data_b,data_c]
#group label
labels=['A','B','C']
#create the box plot
plt.boxplot(data,labels=labels)
#ass titles and labels
plt.title("multiple box plot ")
plt.xlabel('Groups')
plt.ylabel('Values')
plt.show()
```

C:\Users\klh\AppData\Local\Temp\ipykernel_11720\1345803500.py:11: MatplotlibDeprecationWarning:

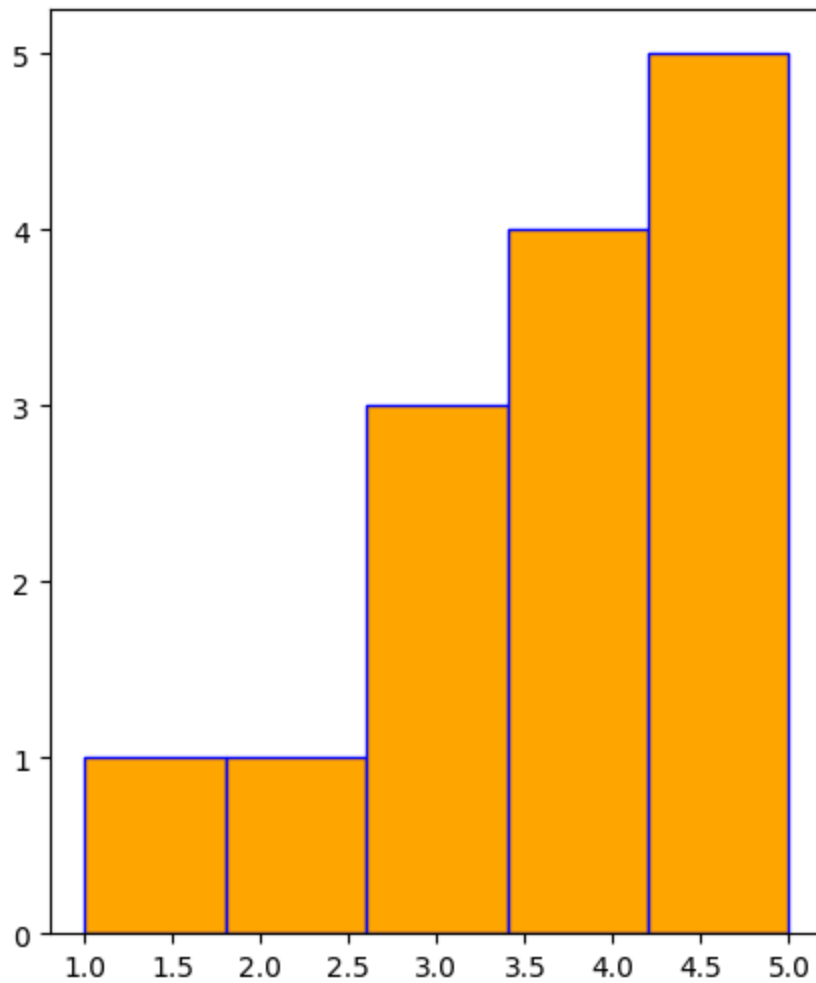
The 'labels' parameter of boxplot() has been renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be dropped in 3.11.



Simple_Histogram

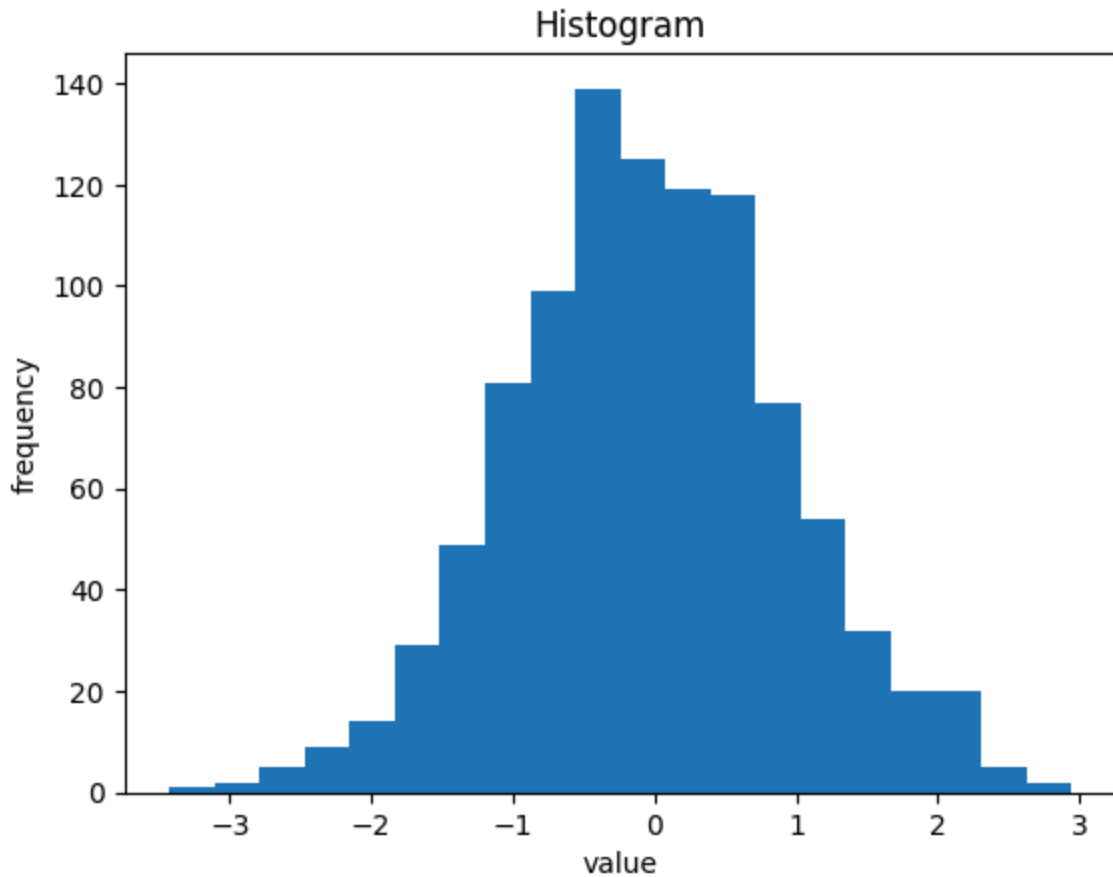
```
In [24]: data = [1,2,3,3,3,4,4,4,4,5,5,5,5,5]
plt.figure(figsize=(5,6))
plt.hist(data,bins=5,color="orange",edgecolor="blue")
```

```
Out[24]: (array([1., 1., 3., 4., 5.]),
array([1. , 1.8, 2.6, 3.4, 4.2, 5. ]),
<BarContainer object of 5 artists>)
```



```
In [25]: import matplotlib.pyplot as plt
import numpy as np

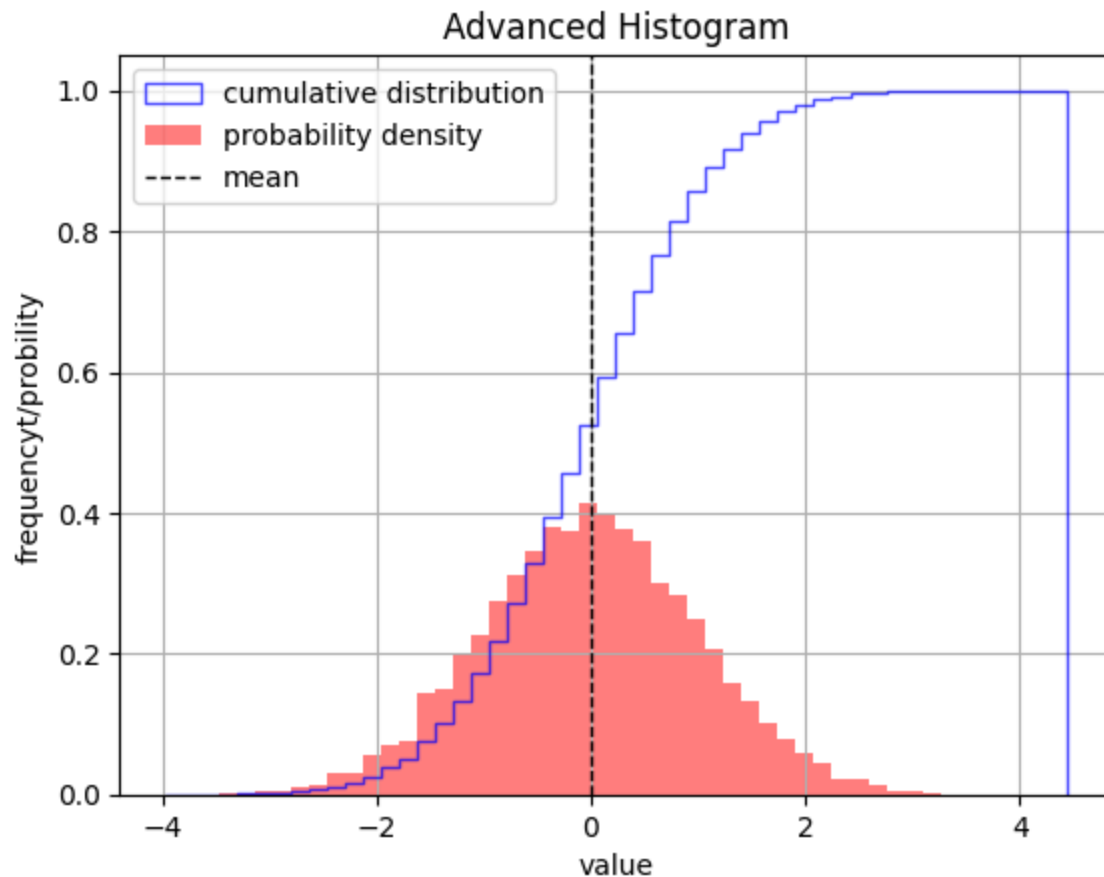
data=np.random.randn(1000)
plt.hist(data,bins=20)
plt.xlabel('value')
plt.ylabel('frequency')
plt.title('Histogram')
plt.show()#display the histogram
```



Advance_Histogram

```
In [26]: import matplotlib.pyplot as plt
import numpy as np

data=np.random.randn(10000)
plt.hist(data,bins=50,density=True,cumulative=True,
         histtype='step',color='blue',alpha=0.7,
         label='cumulative distribution')
plt.hist(data,bins=50,density=True,
         histtype='bar',color='red',alpha=0.5,
         label='probability density')
#add a vertical line at the mean
plt.axvline(data.mean(),color='k',linestyle='dashed',
            linewidth=1,label='mean')
plt.xlabel('value')
plt.ylabel('frequency/probability')
plt.title('Advanced Histogram')
plt.legend()
plt.grid(True)
plt.show()
```

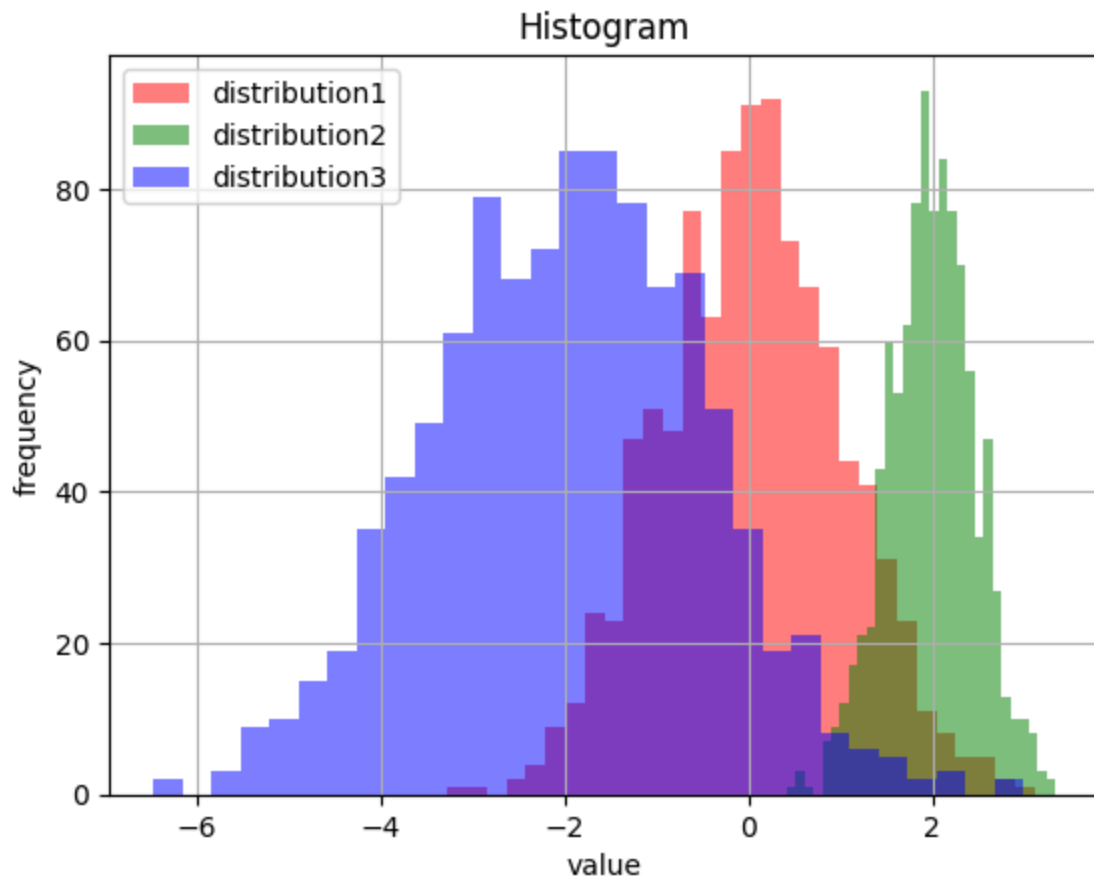


Verstyle_Histogram_multiple

```
In [27]: #verstyle histogram
import matplotlib.pyplot as plt
import numpy as np
data1=np.random.normal(loc=0,scale=1,size=1000)
data2=np.random.normal(loc=2,scale=0.5,size=1000)
data3=np.random.normal(loc=-2,scale=1.5,size=1000)

plt.hist(data1,bins=30,color='red',alpha=0.5,label='distribution1')
plt.hist(data2,bins=30,color='green',alpha=0.5,label='distribution2')
plt.hist(data3,bins=30,color='blue',alpha=0.5,label='distribution3')

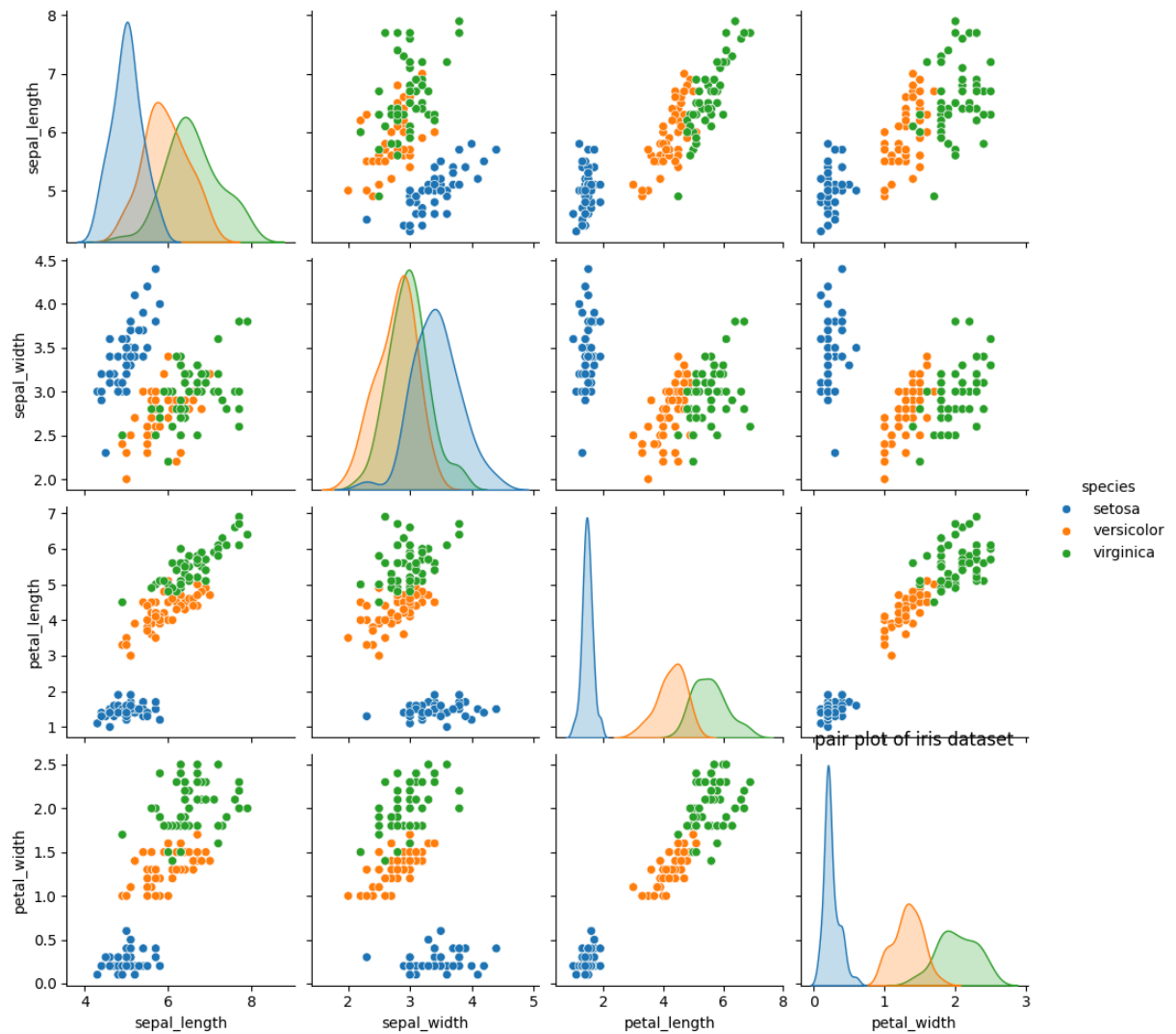
plt.xlabel('value')
plt.ylabel('frequency')
plt.title('Histogram')
plt.legend()
plt.grid(True)
plt.show() #display the histogram
```



pair plot

```
In [28]: import matplotlib.pyplot as plt
import seaborn as sns
data=sns.load_dataset('iris')
##create pair plot

sns.pairplot(data,hue='species')
plt.title('pair plot of iris dataset')
plt.show()
```



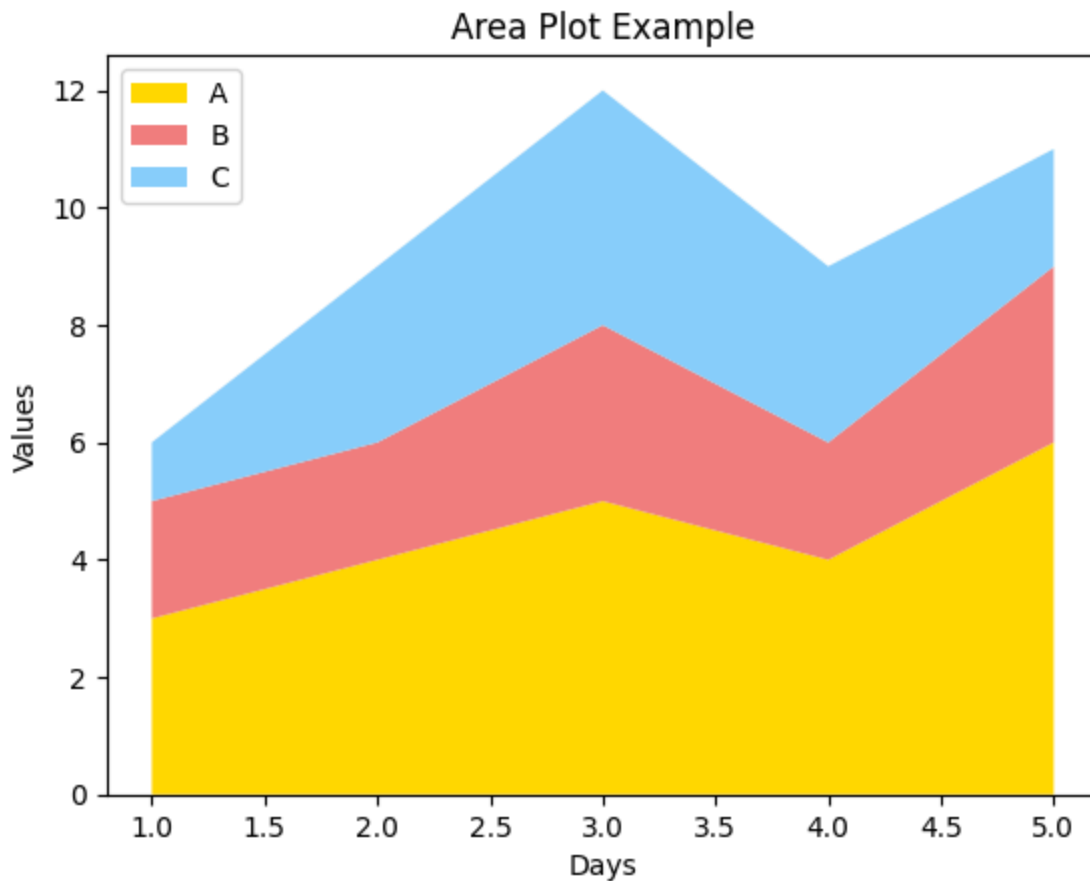
Area Plor

```
In [3]: import matplotlib.pyplot as plt

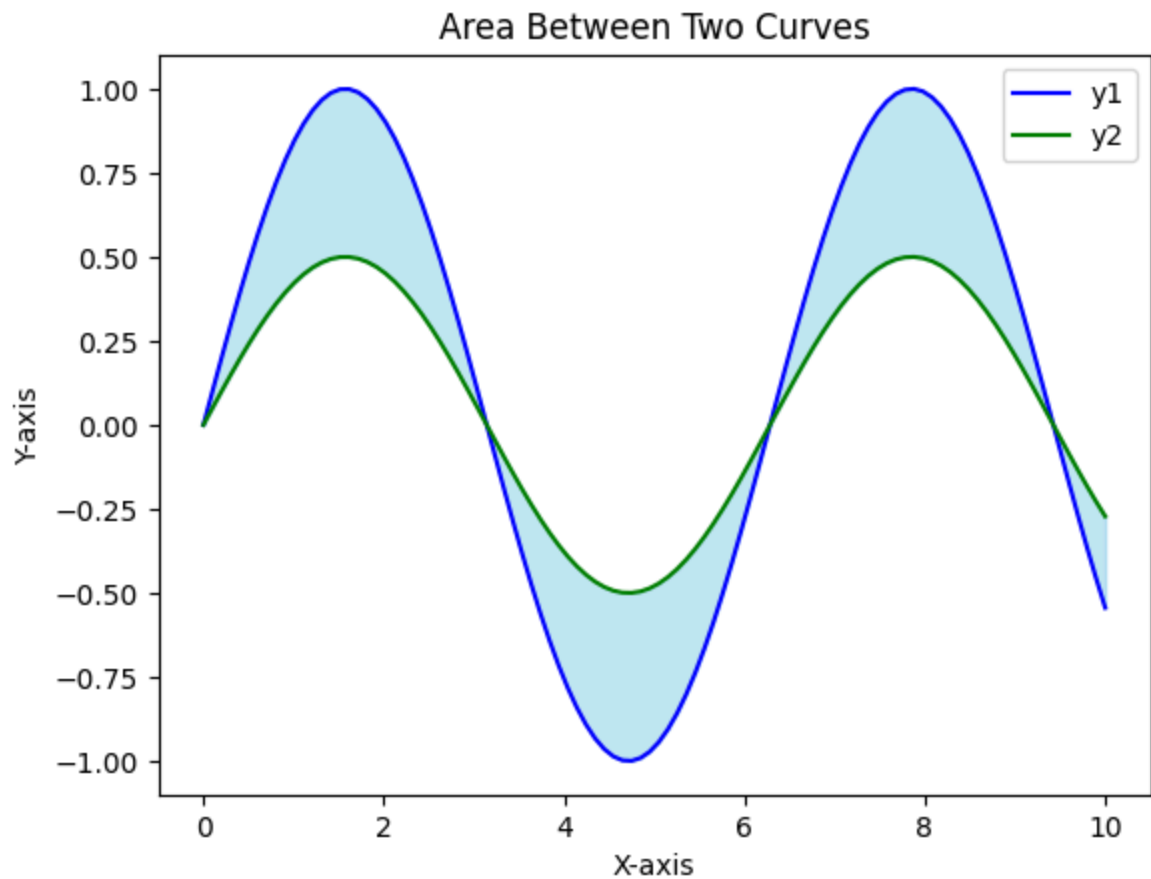
# Sample data
days = [1, 2, 3, 4, 5]
A = [3, 4, 5, 4, 6]
B = [2, 2, 3, 2, 3]
C = [1, 3, 4, 3, 2]

# Area plot
plt.stackplot(days, A, B, C, labels=['A', 'B', 'C'], colors=['gold', 'lightcoral', 'lightblue'])

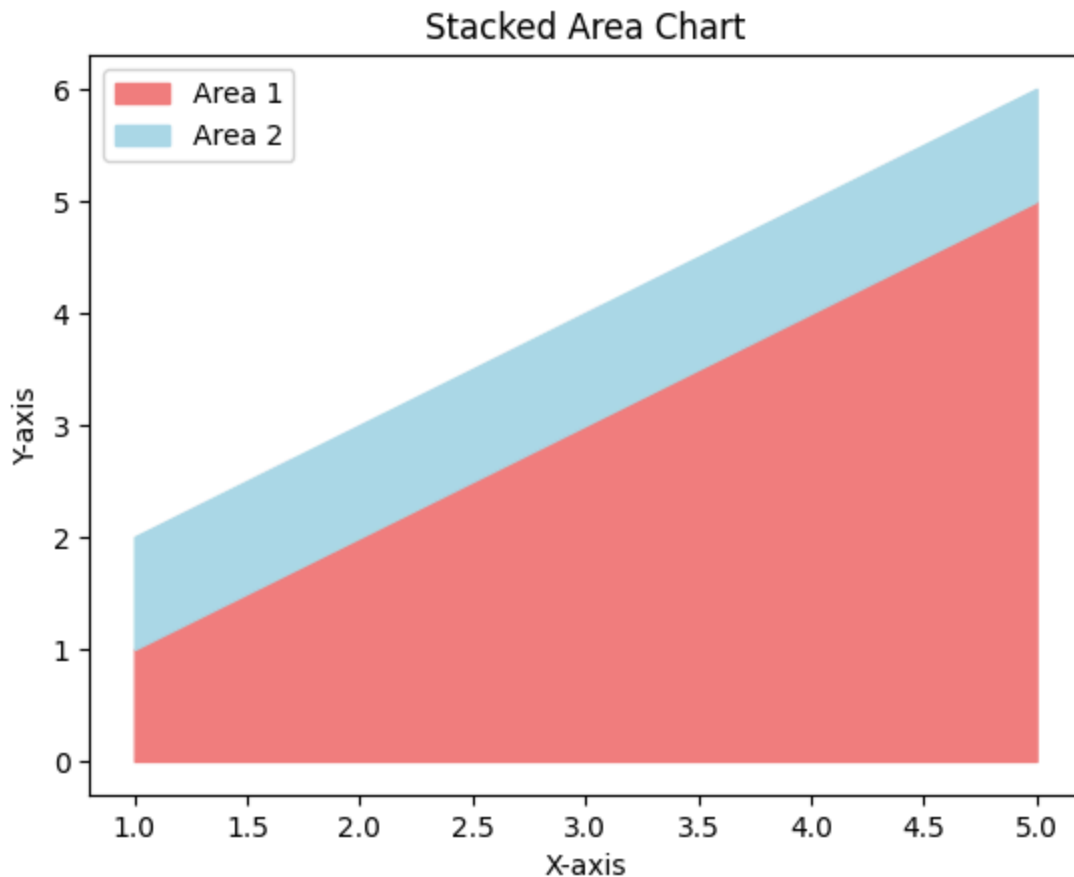
plt.legend(loc='upper left')
plt.title("Area Plot Example")
plt.xlabel("Days")
plt.ylabel("Values")
plt.show()
```



```
In [4]: import matplotlib.pyplot as plt
import numpy as np
# Data
x = np.linspace(0, 10, 100)
y1 = np.sin(x)
y2 = 0.5 * np.sin(x)
# Fill the area between y1 and y2
plt.fill_between(x, y1, y2, color="skyblue", alpha=0.5)
plt.plot(x, y1, label="y1", color="blue")
plt.plot(x, y2, label="y2", color="green")
# Labels and Legend
plt.title("Area Between Two Curves")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.legend()
plt.show()
```

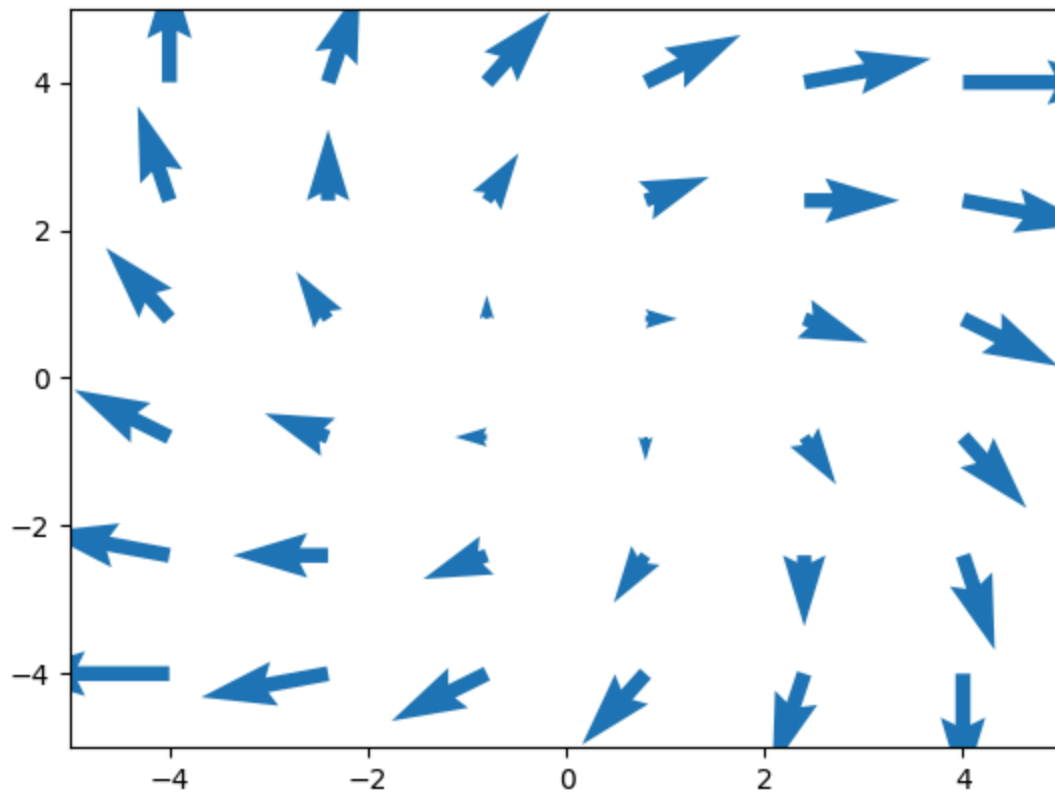


```
In [5]: # Data for stacked area chart
x = np.arange(1, 6)
y1 = [1, 2, 3, 4, 5]
y2 = [2, 3, 4, 5, 6]
# Stacked areas
plt.fill_between(x, y1, color="lightcoral", label="Area 1")
plt.fill_between(x, y2, y1, color="lightblue", label="Area 2")
# Labels and Legend
plt.title("Stacked Area Chart")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.legend()
plt.show()
```

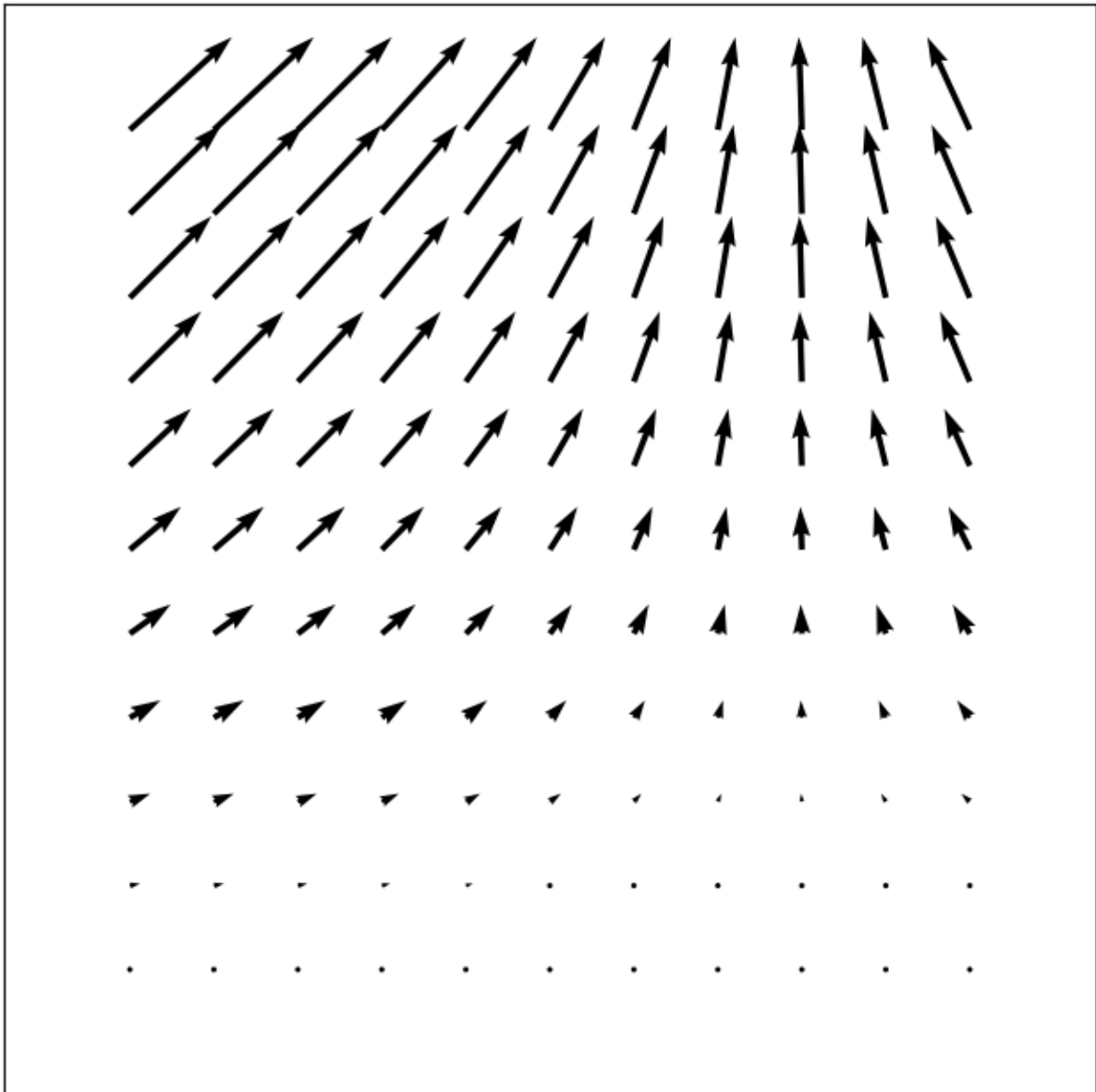


Quiver Plot

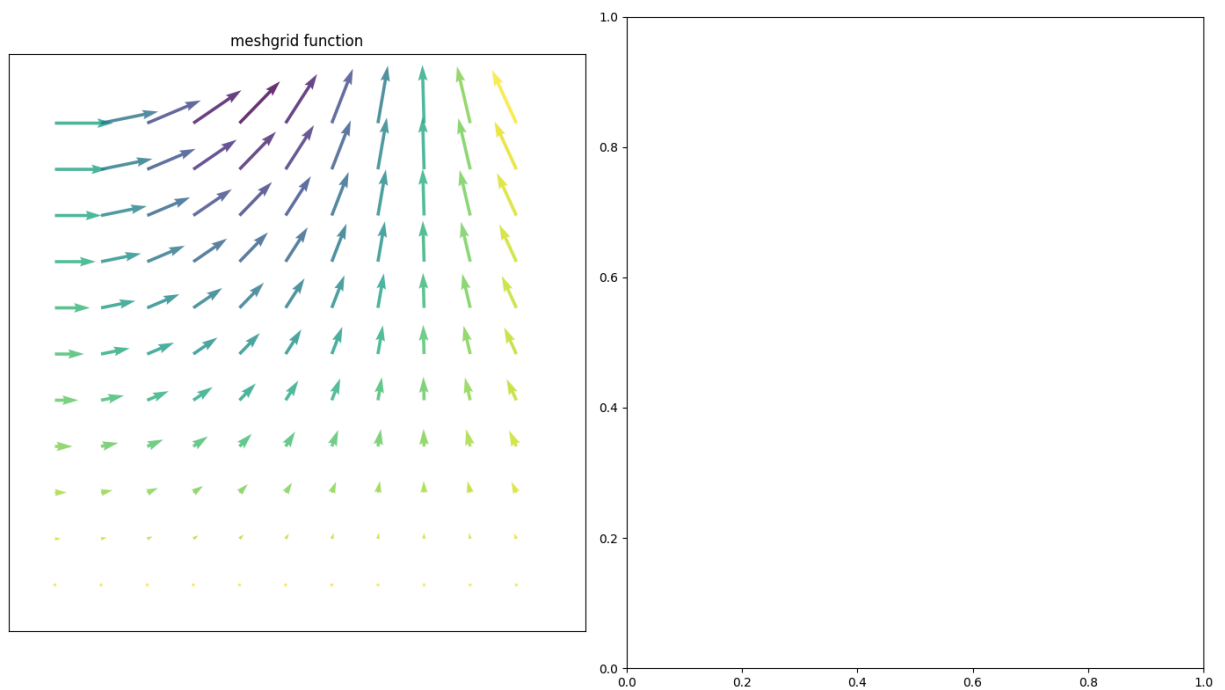
```
In [6]: import numpy as np
import matplotlib.pyplot as plt
# Define the grid
x = np.linspace(-4, 4, 6)
y = np.linspace(-4, 4, 6)
X, Y = np.meshgrid(x, y)
# Define the vector components
U = X + Y
V = Y - X
# Create the quiver plot
plt.quiver(X, Y, U, V, color="C0", angles='xy', scale_units='xy', scale=5, width=.05)
plt.xlim(-5, 5)
plt.ylim(-5, 5)
plt.show()
```



```
In [8]: import numpy as np
import matplotlib.pyplot as plt
x = np.arange(0, 2.2, 0.2)
y = np.arange(0, 2.2, 0.2)
X, Y = np.meshgrid(x, y)
u = np.cos(X) * Y
v = np.sin(Y) * Y
fig, ax = plt.subplots(figsize=(14, 8))
ax.quiver(X, Y, u, v)
ax.xaxis.set_ticks([])
ax.yaxis.set_ticks([])
ax.axis([-0.3, 2.3, -0.3, 2.3])
ax.set_aspect('equal')
plt.show()
```



```
In [9]: import numpy as np
import matplotlib.pyplot as plt
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 8))
x = np.arange(0, 2.2, 0.2)
y = np.arange(0, 2.2, 0.2)
X, Y = np.meshgrid(x, y)
u = np.cos(X) * Y
v = np.sin(y) * Y
color = np.sqrt(((v - 2) / 2) ** 2 + ((u - 2) / 2) ** 2)
ax1.quiver(X, Y, u, v, color, alpha=0.8)
ax1.xaxis.set_ticks([])
ax1.yaxis.set_ticks([])
ax1.axis([-0.2, 2.3, -0.2, 2.3])
ax1.set_aspect('equal')
ax1.set_title('meshgrid function')
plt.tight_layout()
plt.show()
```



```
In [10]: # Import libraries
import numpy as np
import matplotlib.pyplot as plt

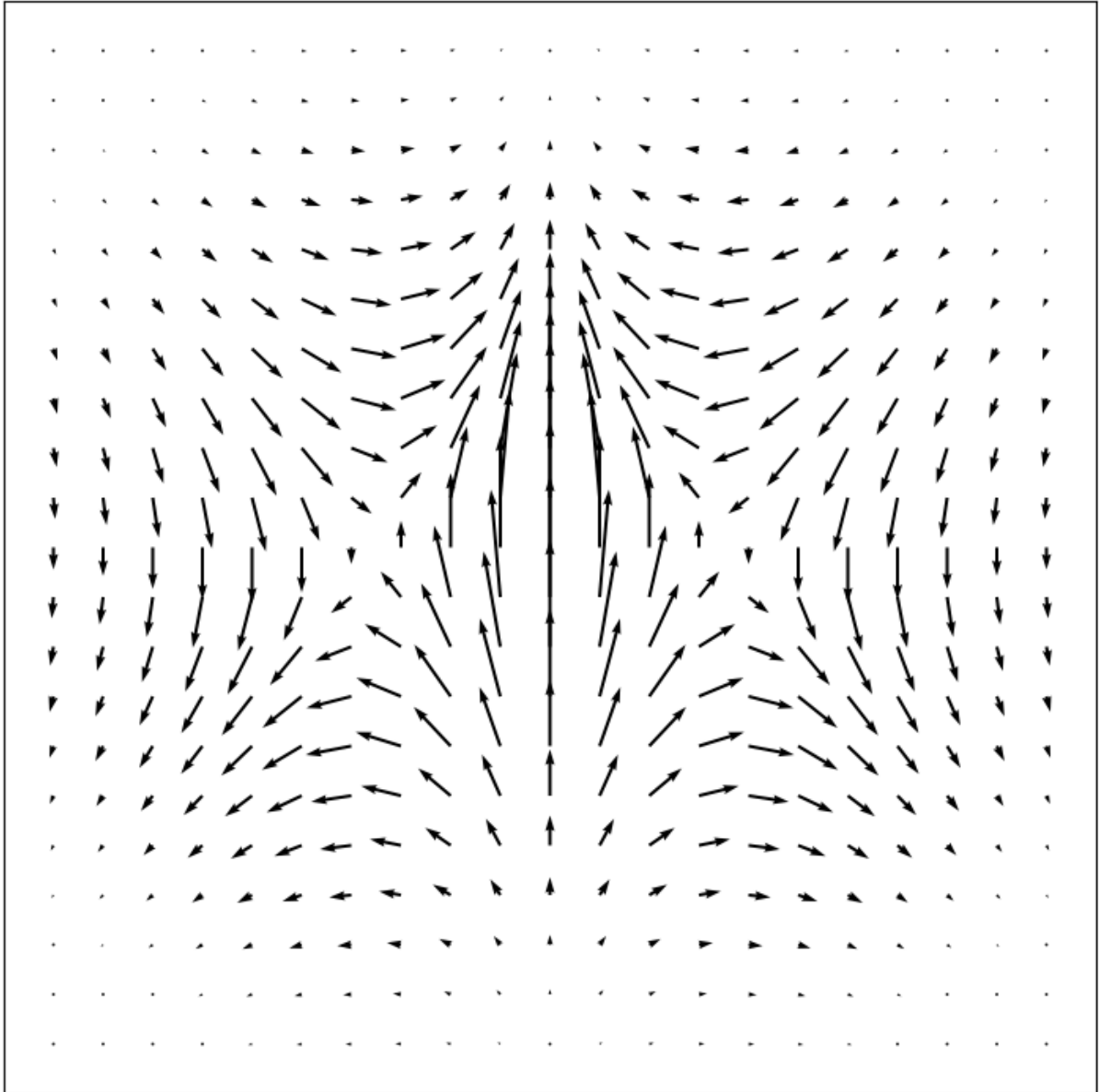
# Creating arrows
x = np.arange(-2, 2.2, 0.2)
y = np.arange(-2, 2.2, 0.2)

# Creating gradient
X, Y = np.meshgrid(x, y)
z = X * np.exp(-X**2-Y**2)
dx, dy = np.gradient(z)

# Creating plot
fig, ax = plt.subplots(figsize =(9, 9))
ax.quiver(X, Y, dx, dy)

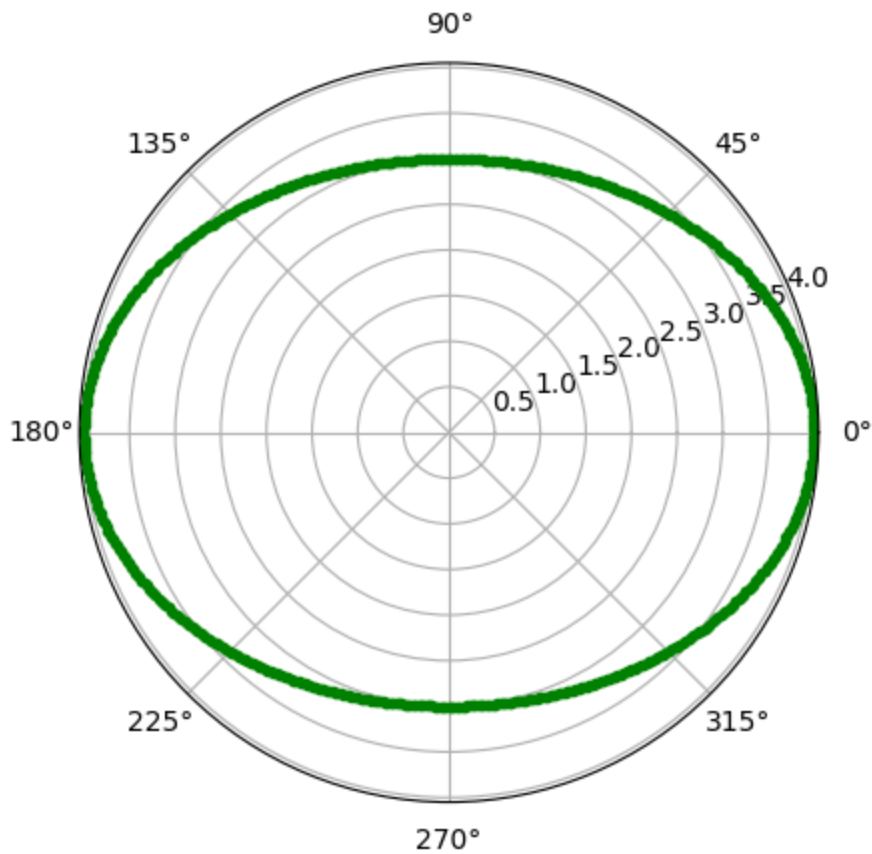
ax.xaxis.set_ticks([])
ax.yaxis.set_ticks([])
ax.set_aspect('equal')

# show plot
plt.show()
```

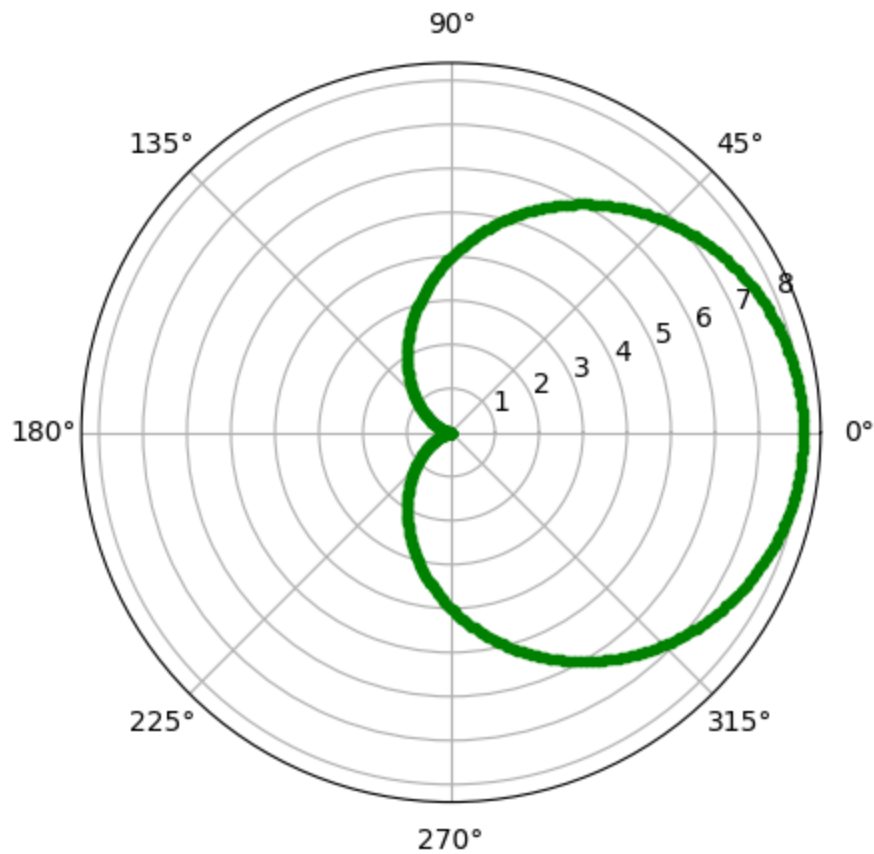


Polar Plot

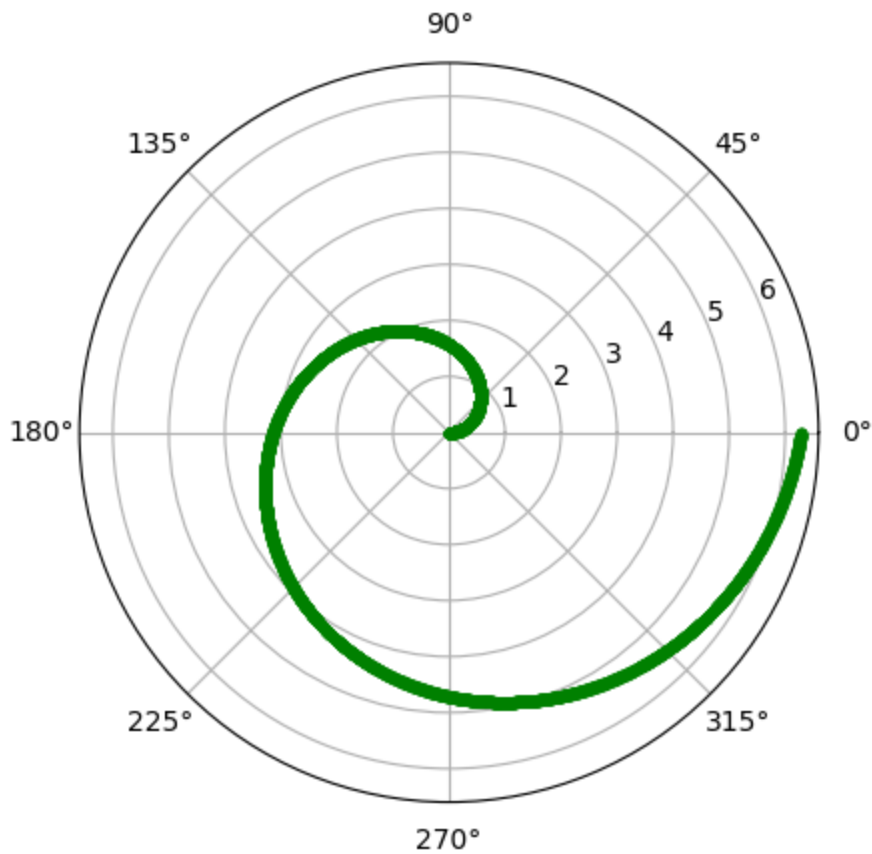
```
In [11]: import numpy as np
import matplotlib.pyplot as plt
import math
plt.axes(projection = 'polar')
a = 4
b = 3
rads = np.arange(0, (2 * np.pi), 0.01)
for rad in rads:
    r = (a*b)/math.sqrt((a*np.sin(rad))**2 + (b*np.cos(rad))**2)
    plt.polar(rad, r, 'g.')
plt.show()
```



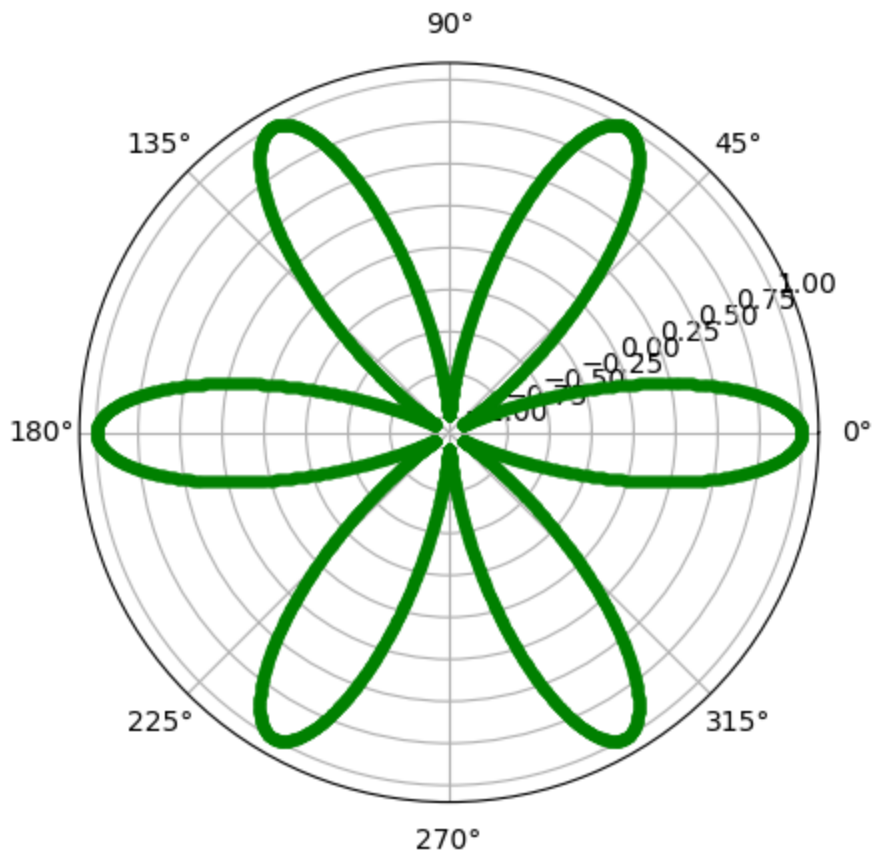
```
In [12]: import numpy as np
import matplotlib.pyplot as plt
import math
plt.axes(projection = 'polar')
a=4
rads = np.arange(0, (2 * np.pi), 0.01)
for rad in rads:
    r = a + (a*np.cos(rad))
    plt.polar(rad,r,'g.')
plt.show()
```



```
In [13]: import numpy as np
import matplotlib.pyplot as plt
plt.axes(projection = 'polar')
rads = np.arange(0, 2 * np.pi, 0.001)
for rad in rads:
    r = rad
    plt.polar(rad, r, 'g.')
plt.show()
```



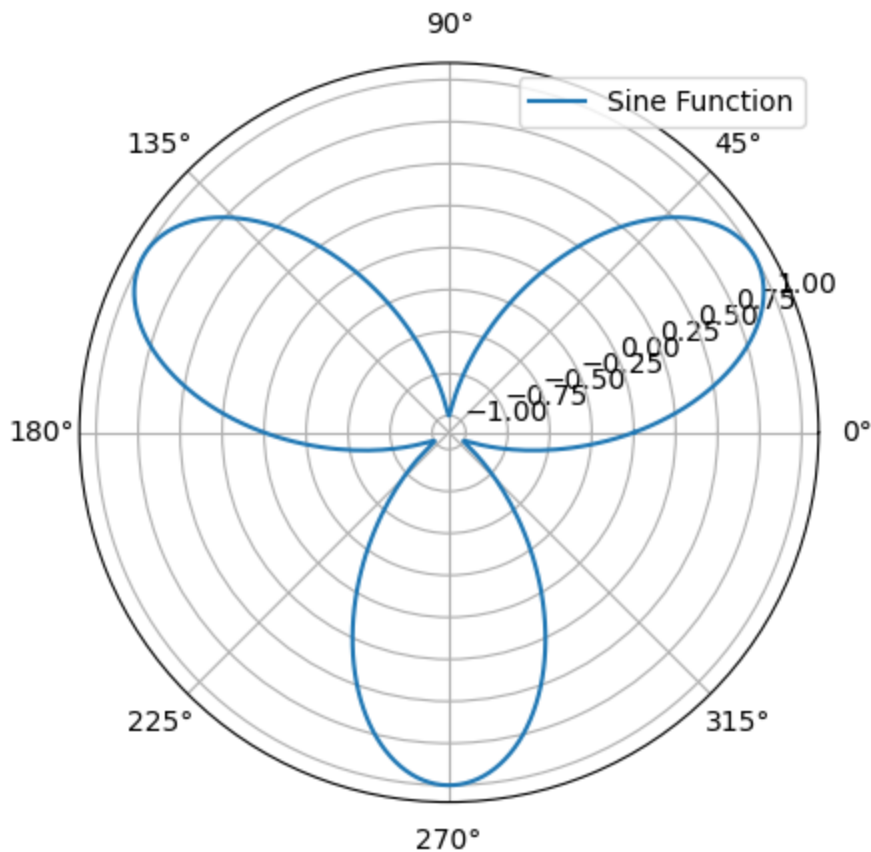
```
In [14]: import numpy as np
import matplotlib.pyplot as plt
plt.axes(projection='polar')
a = 1
n = 6
rads = np.arange(0, 2 * np.pi, 0.001)
for rad in rads:
    r = a * np.cos(n*rad)
    plt.polar(rad, r, 'g.')
plt.show()
```



```
In [15]: import matplotlib.pyplot as plt
import numpy as np

# Generating polar coordinates using a sine function
theta = np.linspace(0, 2*np.pi, 360)
r = np.sin(3 * theta)

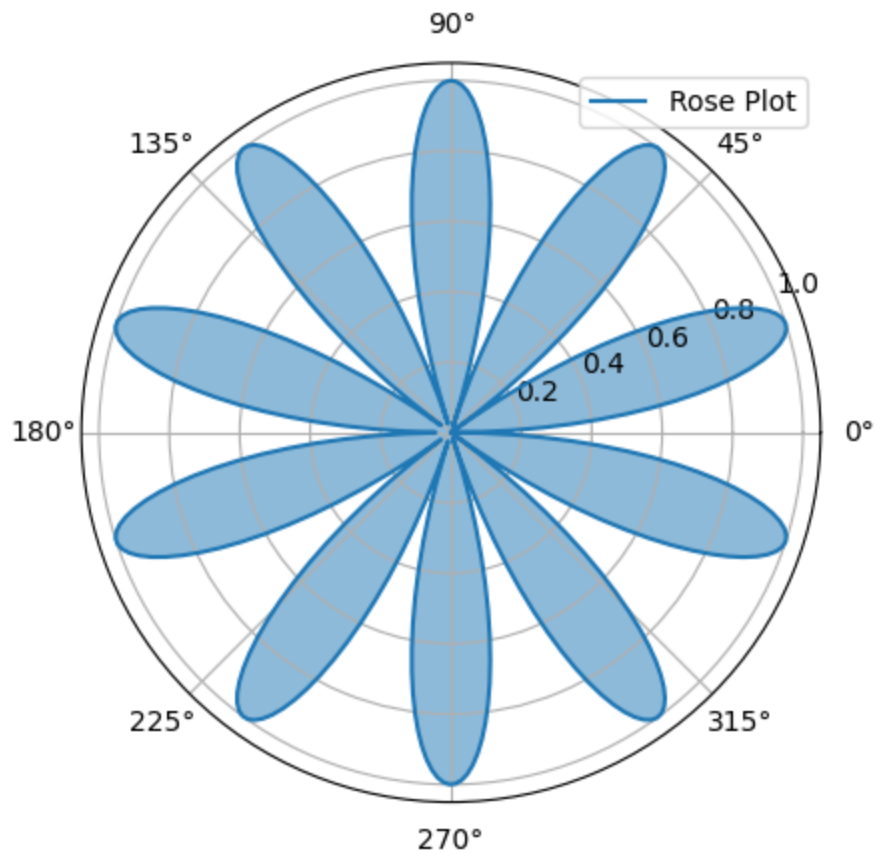
# Plotting the polar data and adding a legend
plt.polar(theta, r, label='Sine Function')
plt.legend()
plt.show()
```



```
In [16]: import matplotlib.pyplot as plt
import numpy as np

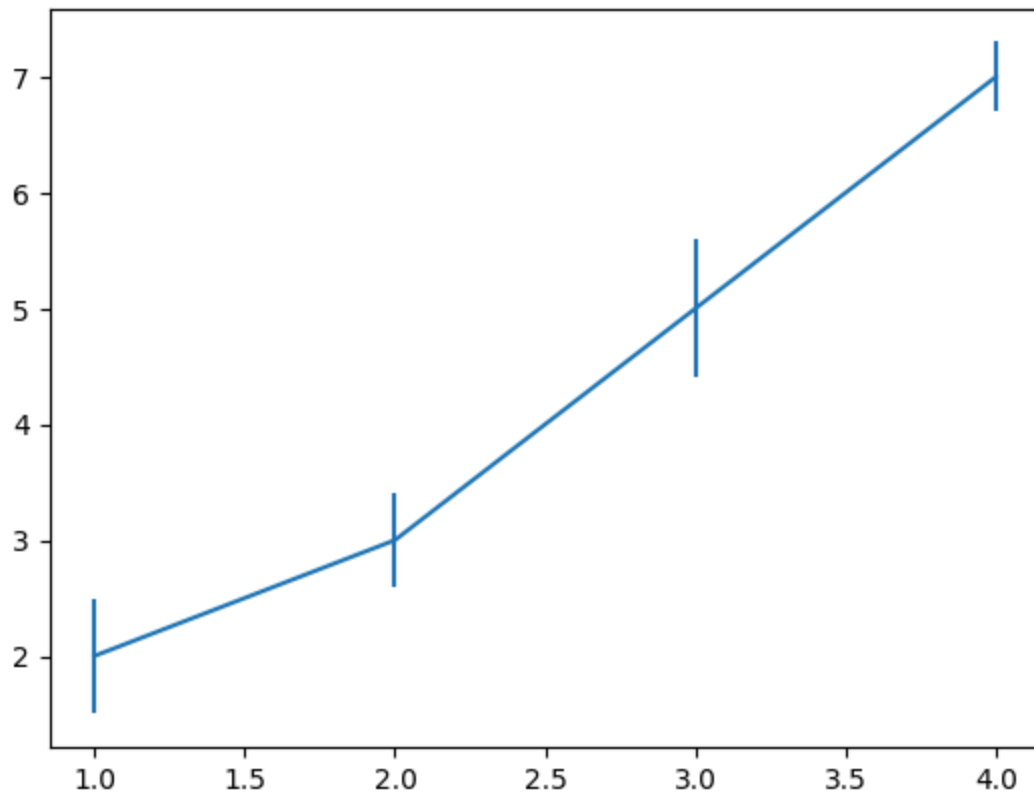
# Generating polar coordinates using the absolute value of a sine function
theta = np.linspace(0, 2*np.pi, 360)
r = np.abs(np.sin(5 * theta))

# Plotting a rose plot with a filled area under the curve
plt.polar(theta, r, label='Rose Plot')
# Filled area under the curve
plt.fill(theta, r, alpha=0.5)
plt.legend()
plt.show()
```

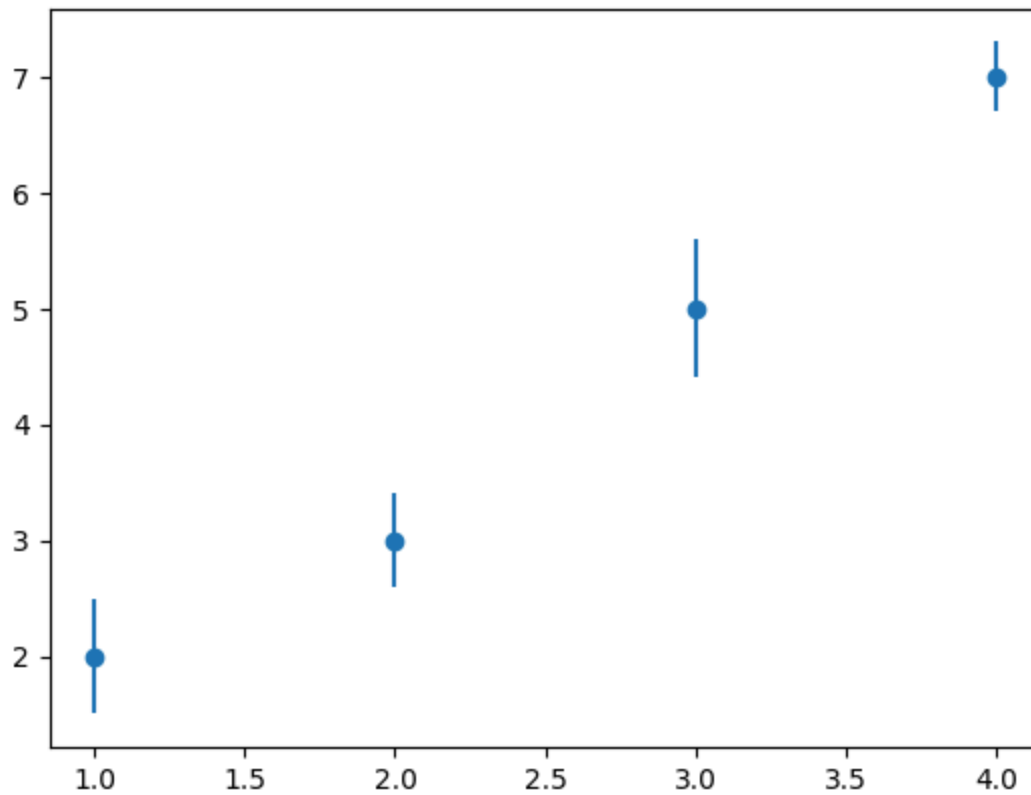


Error Bar Plot

```
In [17]: import matplotlib.pyplot as plt
x = [1, 2, 3, 4]
y = [2, 3, 5, 7]
yerr = [0.5, 0.4, 0.6, 0.3]
plt.errorbar(x, y, yerr=yerr) # Default behavior connects points with lines
plt.show()
```



```
In [18]: import matplotlib.pyplot as plt
x = [1, 2, 3, 4]
y = [2, 3, 5, 7]
yerr = [0.5, 0.4, 0.6, 0.3]
plt.errorbar(x, y, yerr=yerr, fmt='o', linestyle='none') # Removes connecting lines
plt.show()
```



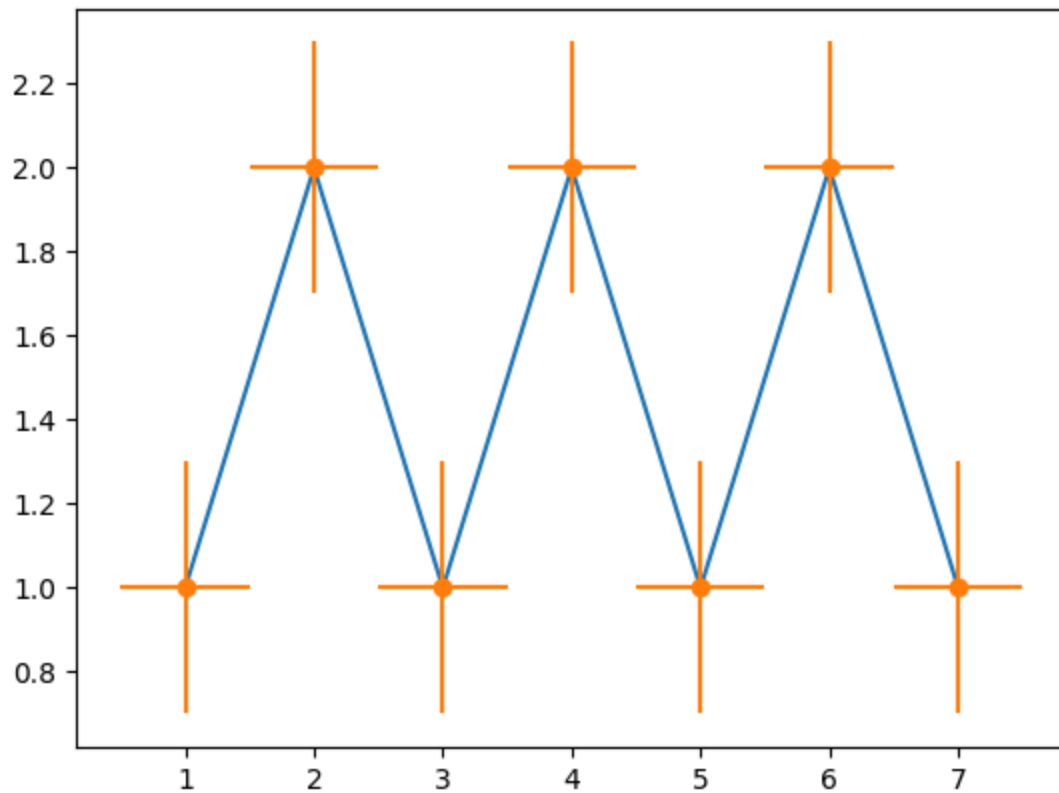
In [19]: `import matplotlib.pyplot as plt`

```
x =[1, 2, 3, 4, 5, 6, 7]
y =[1, 2, 1, 2, 1, 2, 1]

# creating error
x_error = 0.5
y_error = 0.3

# plotting graph
plt.plot(x, y)
plt.errorbar(x, y,
             yerr = y_error,
             xerr = x_error,
             fmt = 'o')
```

Out[19]: `<ErrorbarContainer object of 3 artists>`



In []:

joint plot

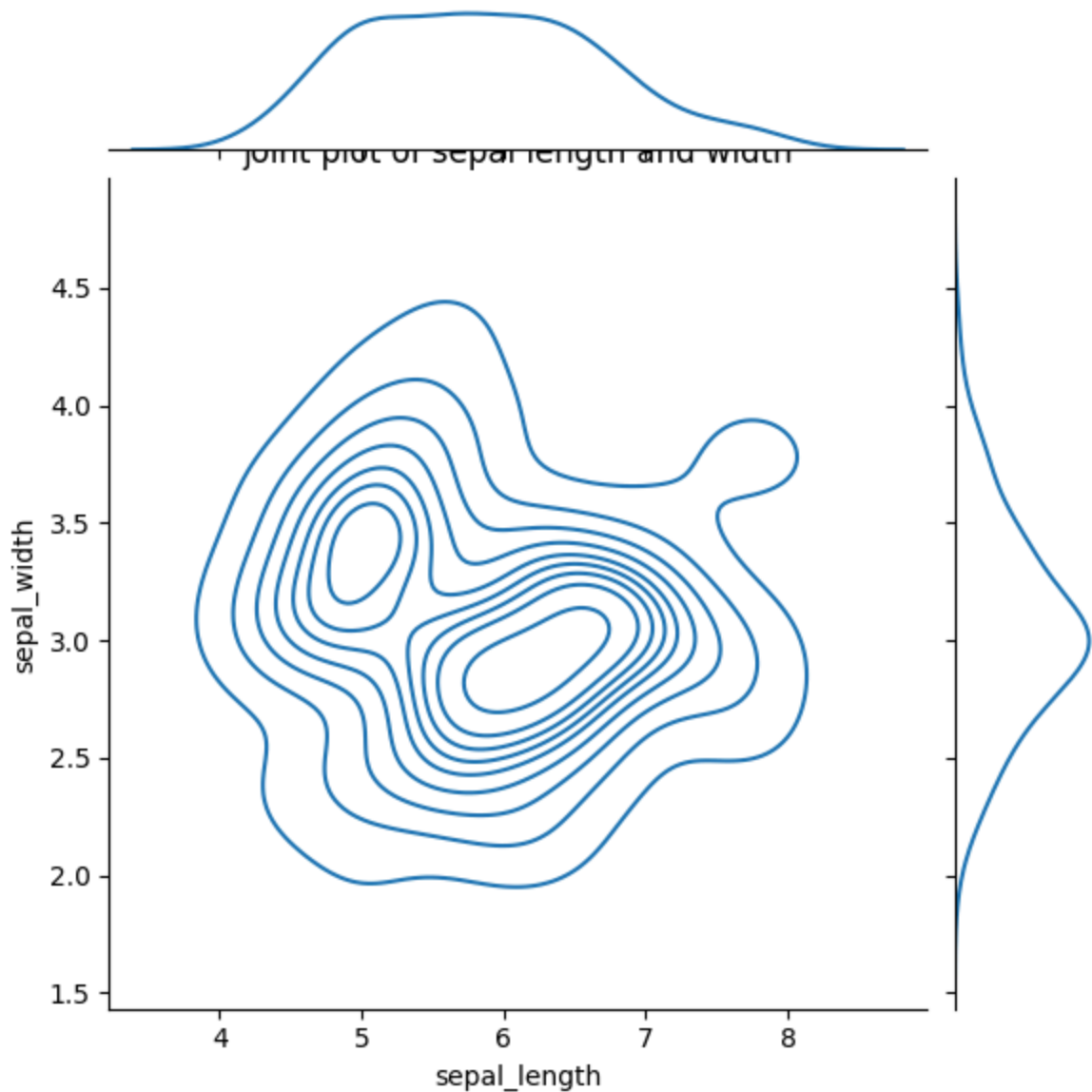
```
In [29]: import matplotlib.pyplot as plt
import seaborn as sns

#sample data(replace with your data )

data=sns.load_dataset('iris')
#create joint plot

sns.jointplot(x='sepal_length',y='sepal_width',data=data,
              kind='kde')

plt.title('joint plot of sepal length and width')
plt.show()
```



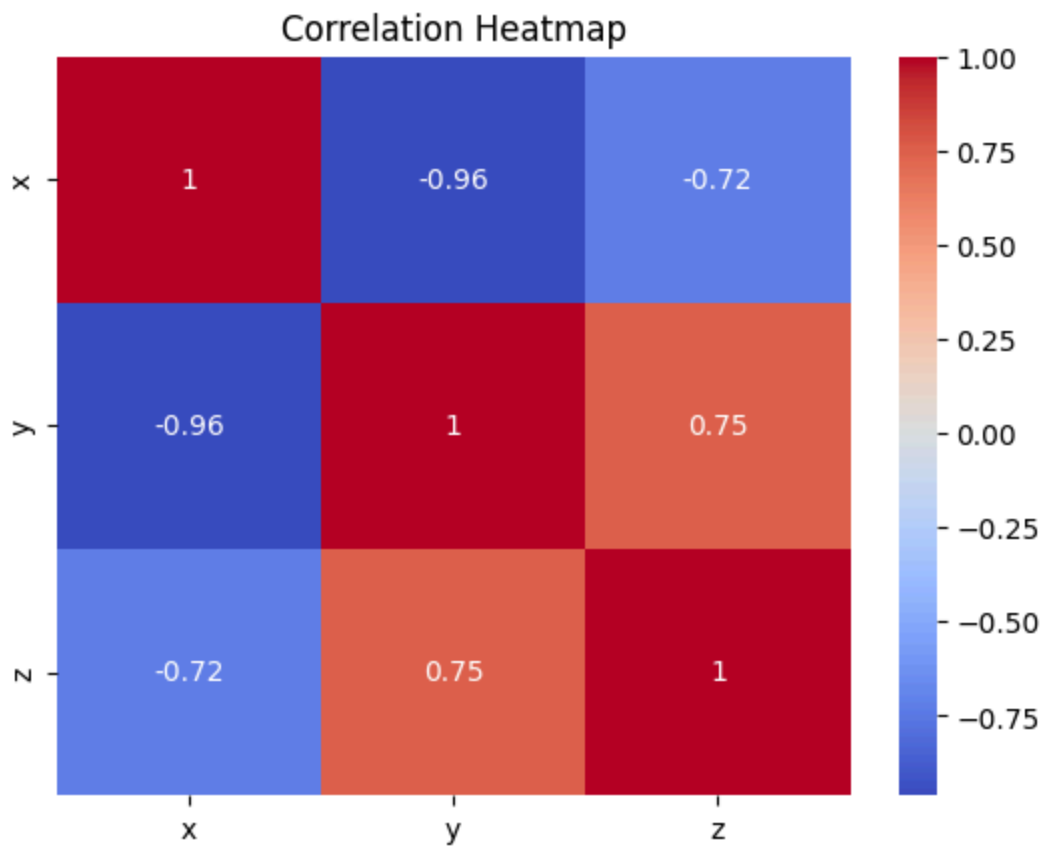
Correlation Heatmap

```
In [30]: import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
import pandas as pd
data = pd.DataFrame({
    "x": [1, 2, 3, 4, 5],
    "y": [66, 44, 39, 2, 1],
    "z": [244, 29, 35, 3, 33],
})

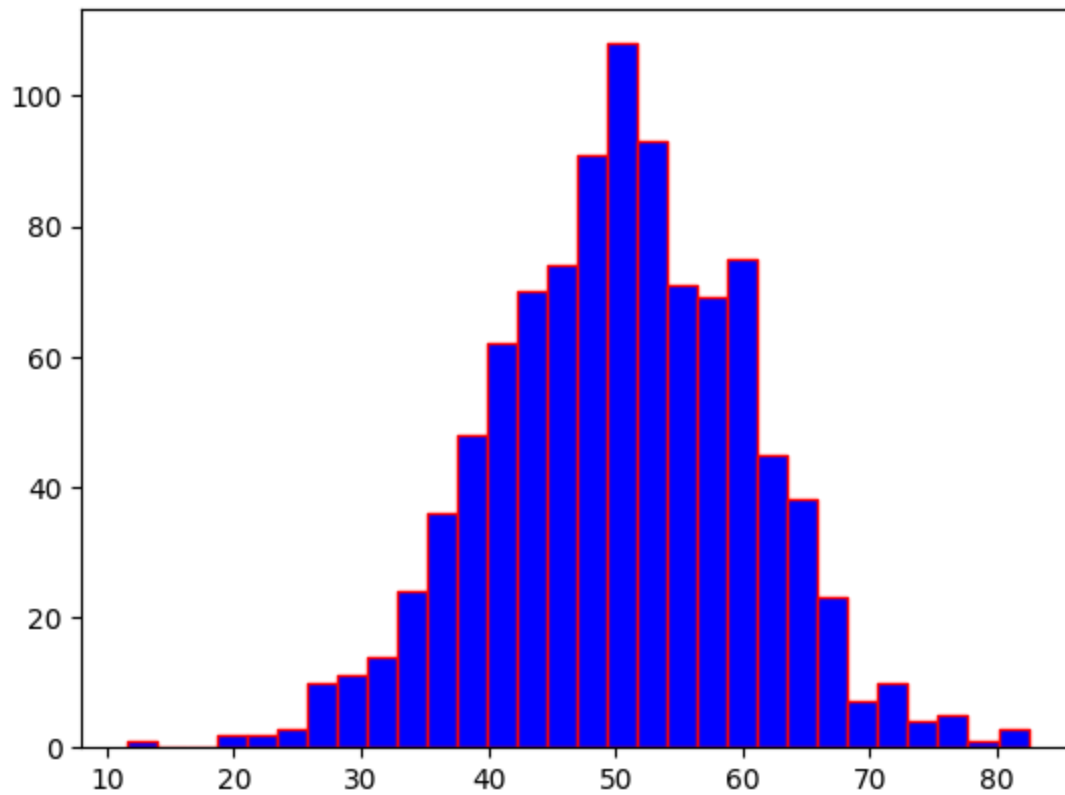
# Correlation matrix
corr = data.corr()

# Heatmap
sns.heatmap(corr,
            annot=True,
```

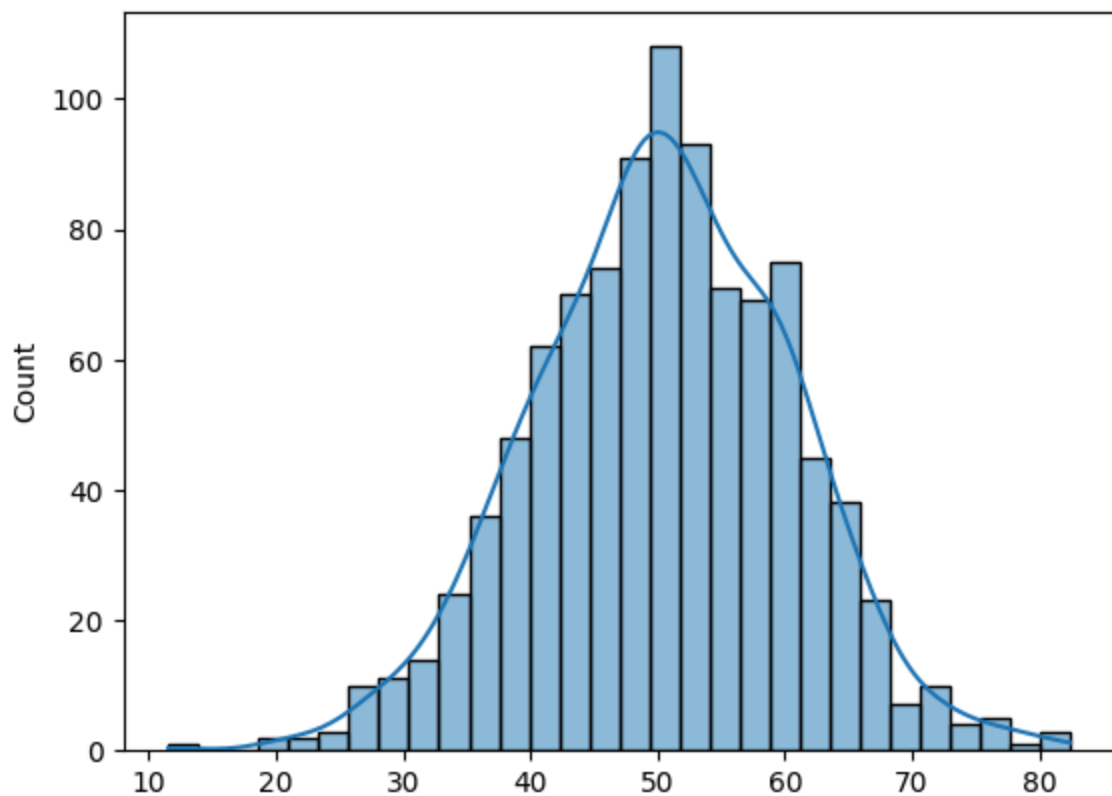
```
cmap="coolwarm")  
plt.title("Correlation Heatmap")  
plt.show()
```



```
In [31]: data = np.random.normal(loc=50, scale=10, size=1000)  
  
plt.hist(data, bins=30, color='blue', edgecolor='red' )  
plt.show()
```



```
In [32]: sns.histplot(data, bins=30, kde=True)
plt.show()
```



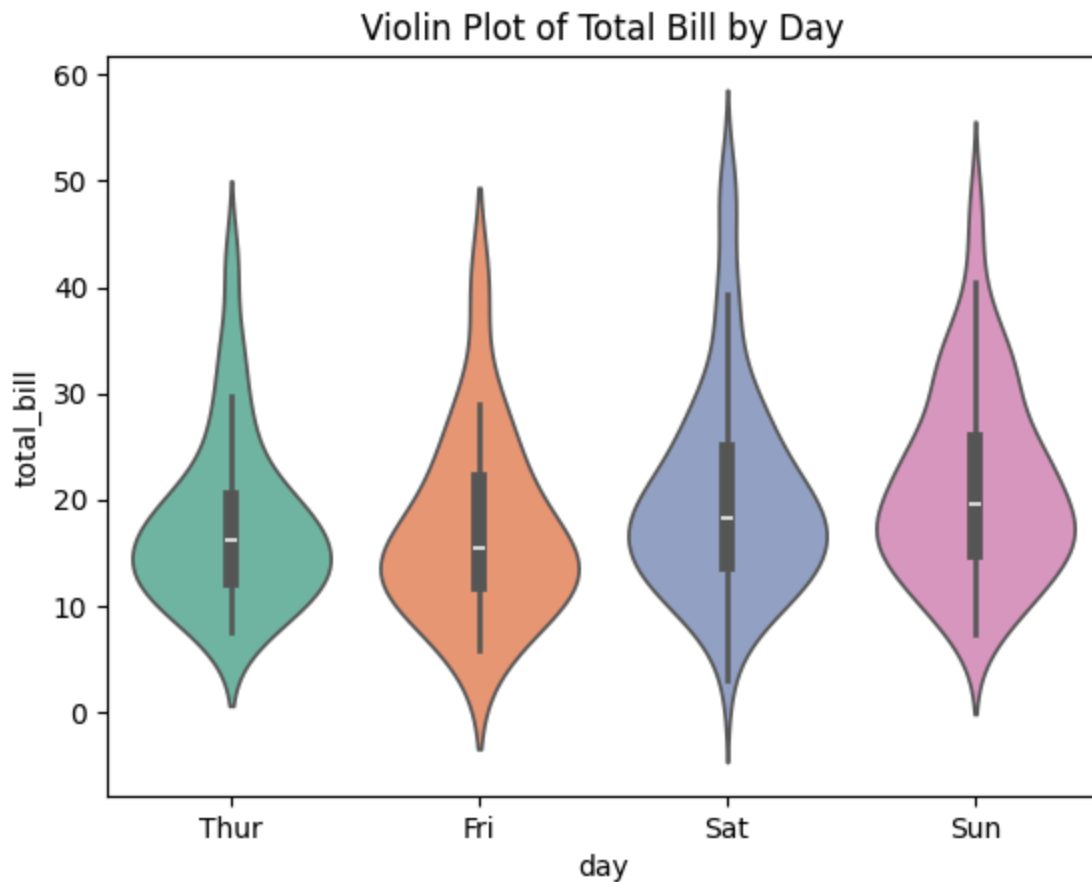
Voilin Chart

```
In [33]: import seaborn as sns
import matplotlib.pyplot as plt

# Load the example dataset
tips = sns.load_dataset("tips")

# Violin plot with hue set to x-variable and legend disabled
sns.violinplot(x="day", y="total_bill", data=tips, inner="box", palette="Set2", hue

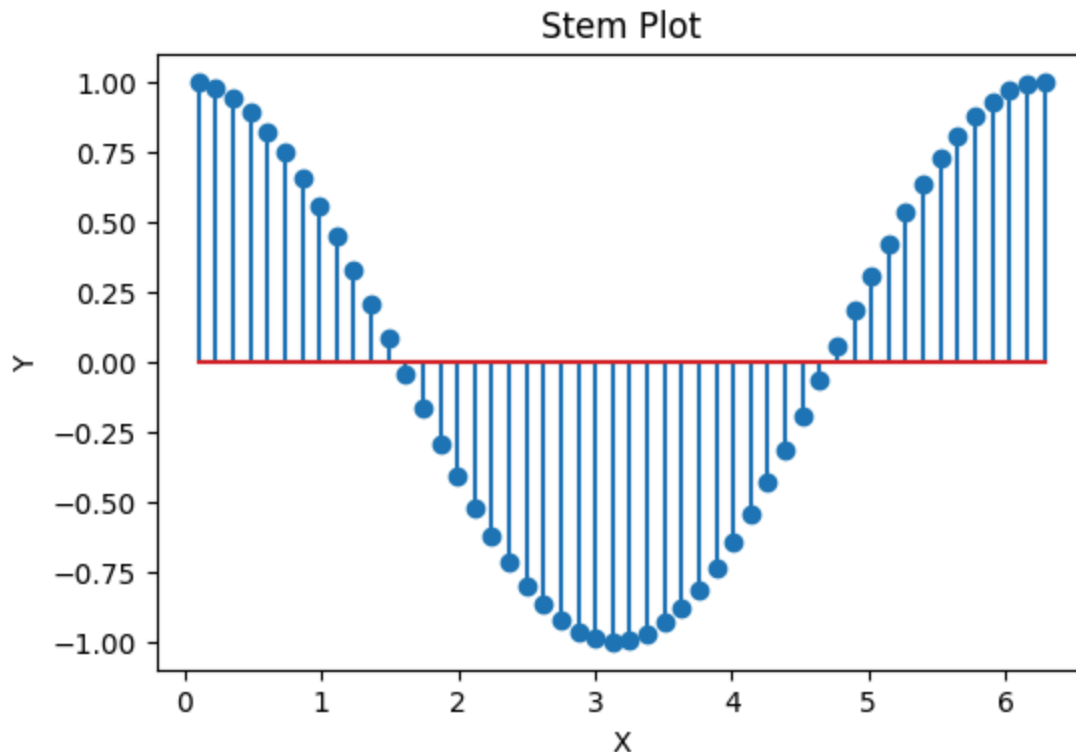
plt.title("Violin Plot of Total Bill by Day")
plt.show()
```



Stem plot

```
In [34]: import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import numpy as np
x = np.linspace(0.1, 2*np.pi, 50)
y = np.cos(x)
plt.figure(figsize=(6,4))
plt.stem(x, y)
plt.xlabel("X")
plt.ylabel("Y")
```

```
plt.title("Stem Plot")
plt.show()
```



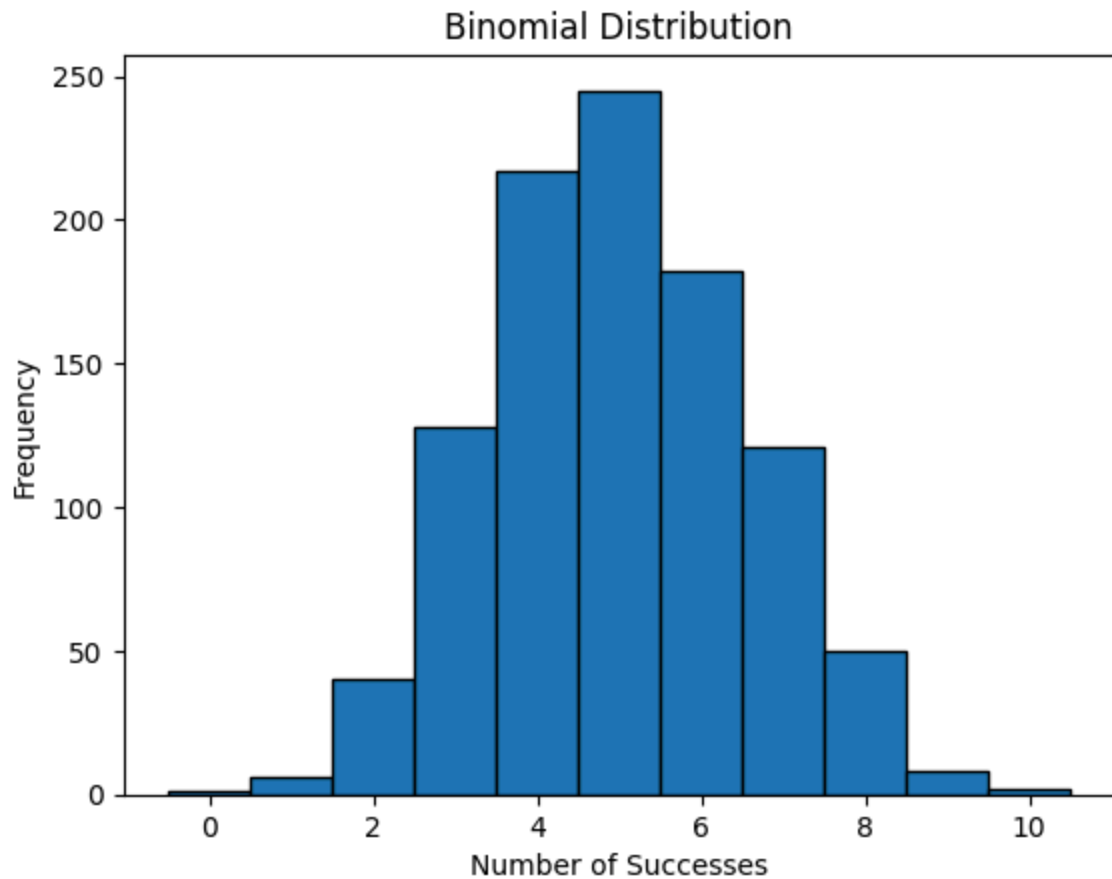
Binomial Distribution

```
In [36]: import numpy as np

# Parameters: n = number of trials, p = probability of success, size = number of ex
n = 10          # number of trials
p = 0.5         # probability of success
size = 1000     # simulate 1000 experiments

# Generate binomial distribution samples
data = np.random.binomial(n, p, size)

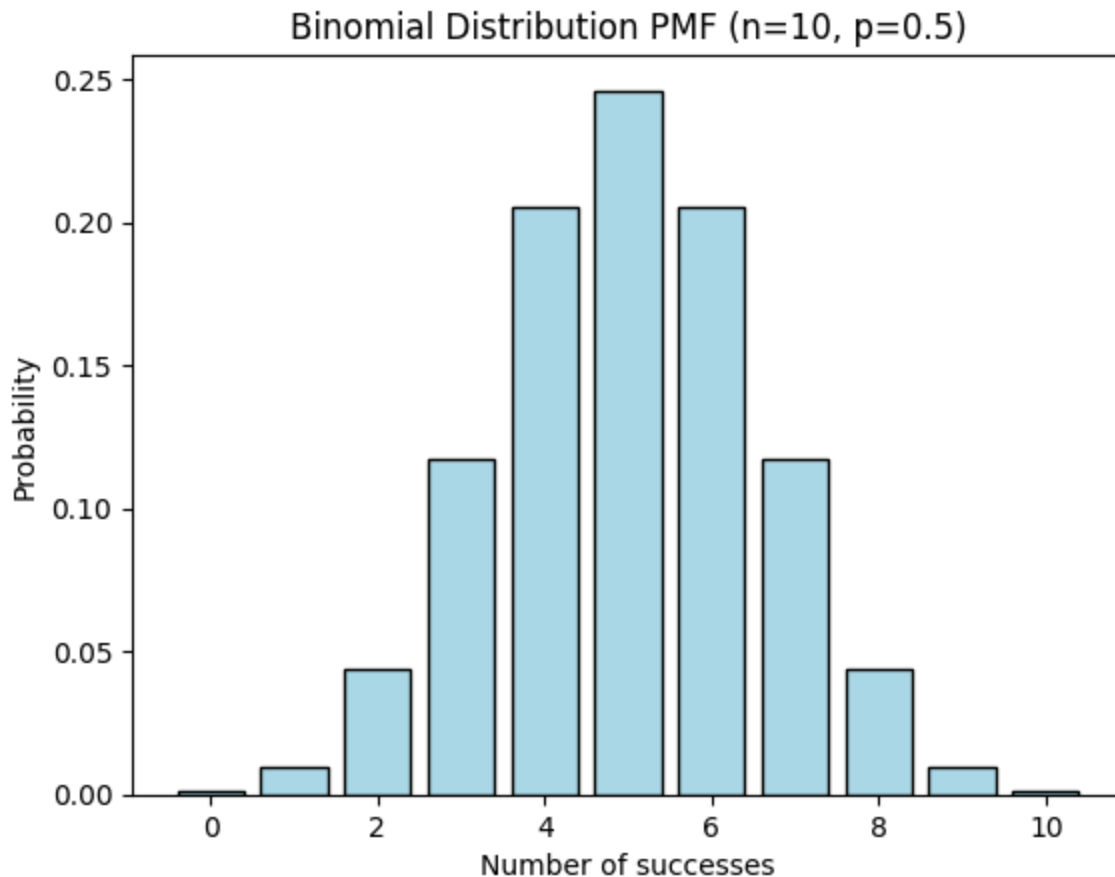
# Optional: visualize
import matplotlib.pyplot as plt
plt.hist(data, bins=range(n+2), edgecolor='black', align='left')
plt.title("Binomial Distribution")
plt.xlabel("Number of Successes")
plt.ylabel("Frequency")
plt.show()
```



```
In [37]: import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
import pandas as pd
from scipy.stats import binom
n = 10
p = 0.5

x = np.arange(0, n+1)
y = binom.pmf(x, n, p)

plt.bar(x,y,color='lightblue', edgecolor='black')
plt.title('Binomial Distribution PMF (n=10, p=0.5)')
plt.xlabel('Number of successes')
plt.ylabel('Probability')
plt.show()
```

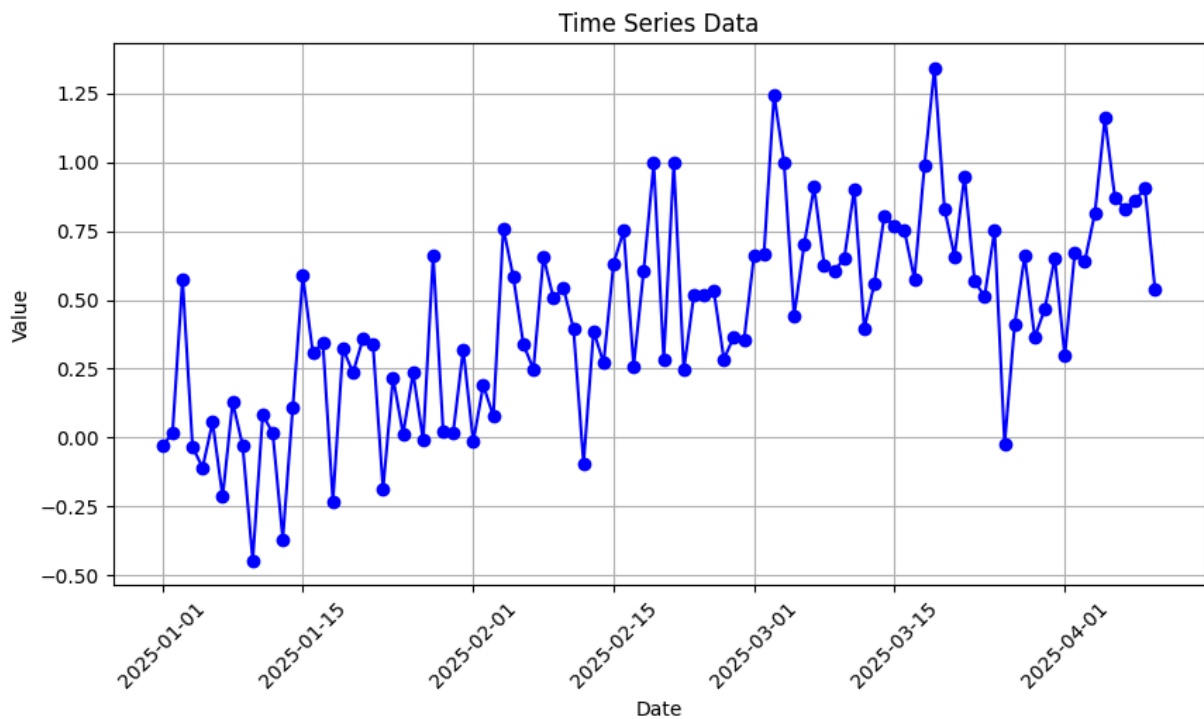


Time Series Data

```
In [38]: dates = pd.date_range('2025-01-01', periods=100, freq='D')
values = np.sin(np.linspace(0,1,100)) + np.random.normal(scale=0.3, size=100)
df = pd.DataFrame({'Date': dates, 'Value': values})

plt.figure(figsize=(10, 5))

plt.plot(df['Date'], df['Value'], marker='o', linestyle='-', color='blue')
plt.title('Time Series Data')
plt.xlabel('Date')
plt.ylabel('Value')
plt.xticks(rotation=45)
plt.grid(True)
plt.show()
```



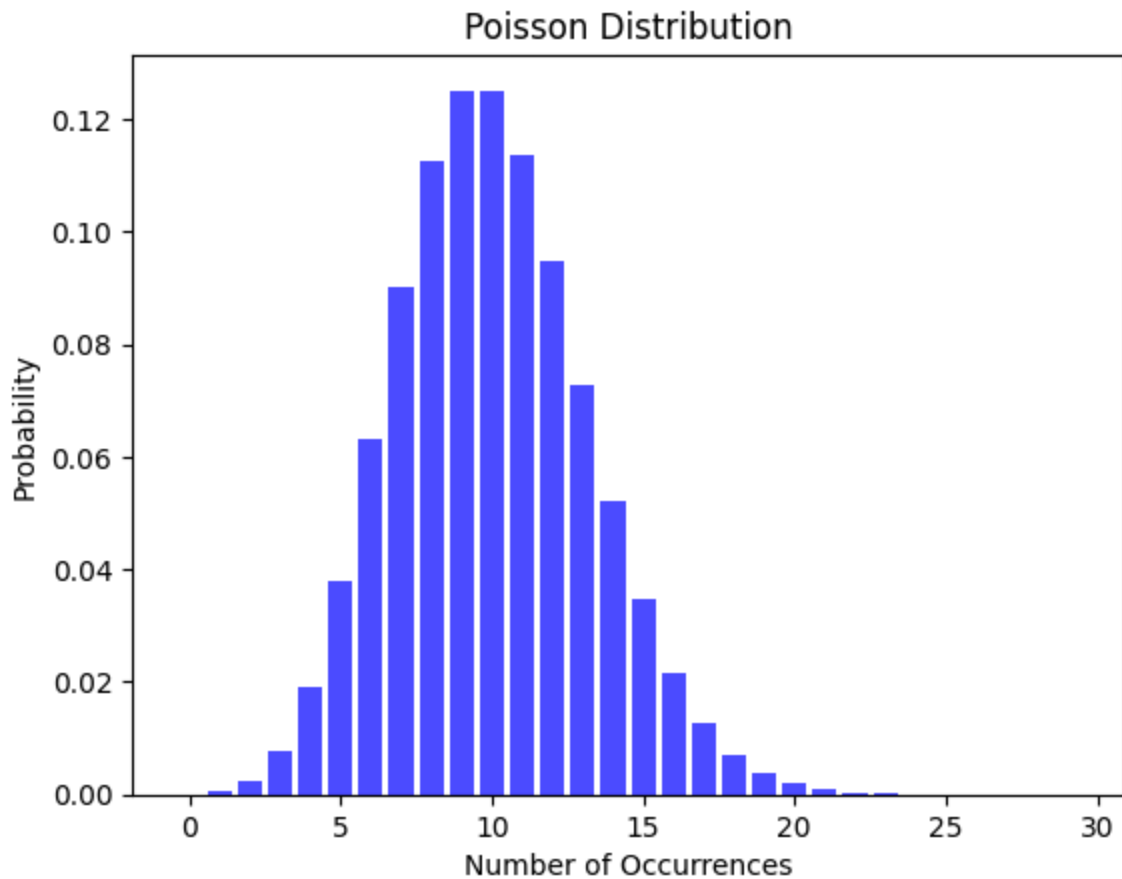
poission distribution

```
In [39]: import numpy as np
import matplotlib.pyplot as plt
import scipy.stats as stats

# Parameters
mu = 10
x = np.arange(0, 30)

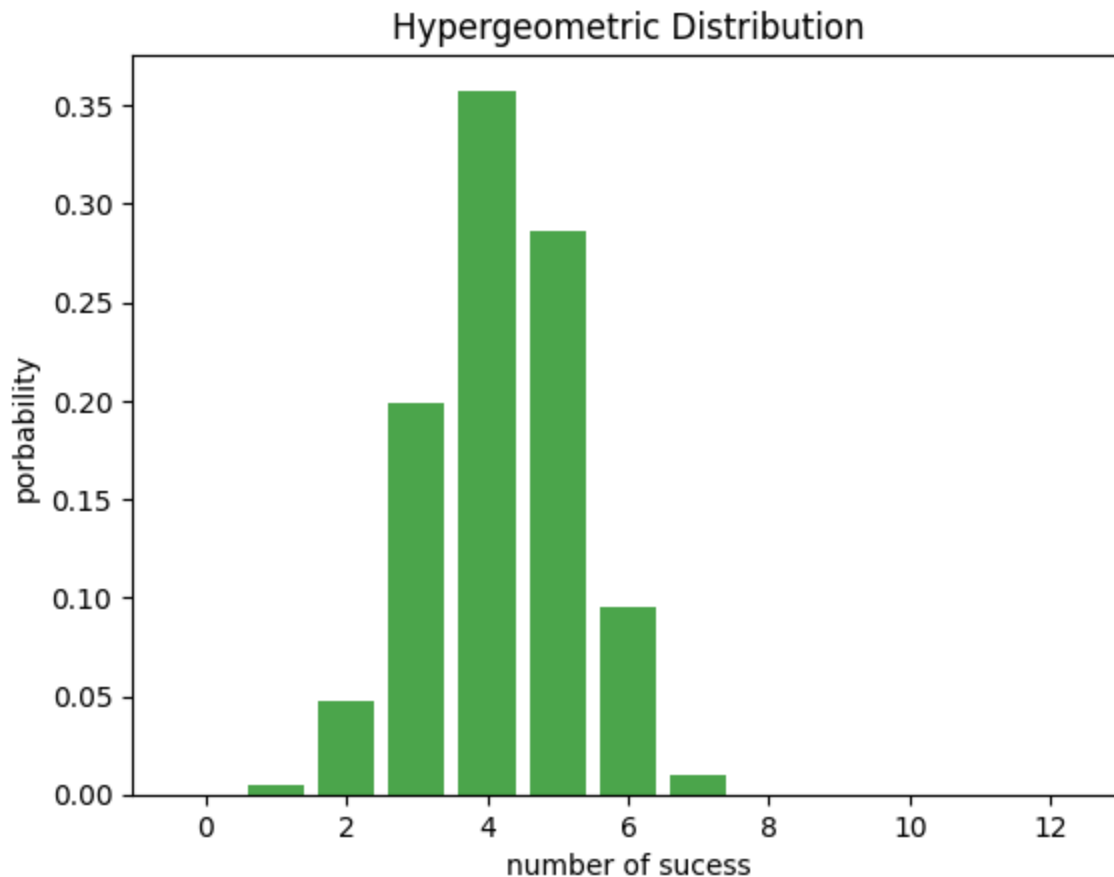
# Poisson PMF
pmf = stats.poisson.pmf(x, mu)

# Plot
plt.bar(x, pmf, color="blue", alpha=0.7)
plt.xlabel("Number of Occurrences")
plt.ylabel("Probability")
plt.title("Poisson Distribution")
plt.show()
```



Hypergeometric Distribution

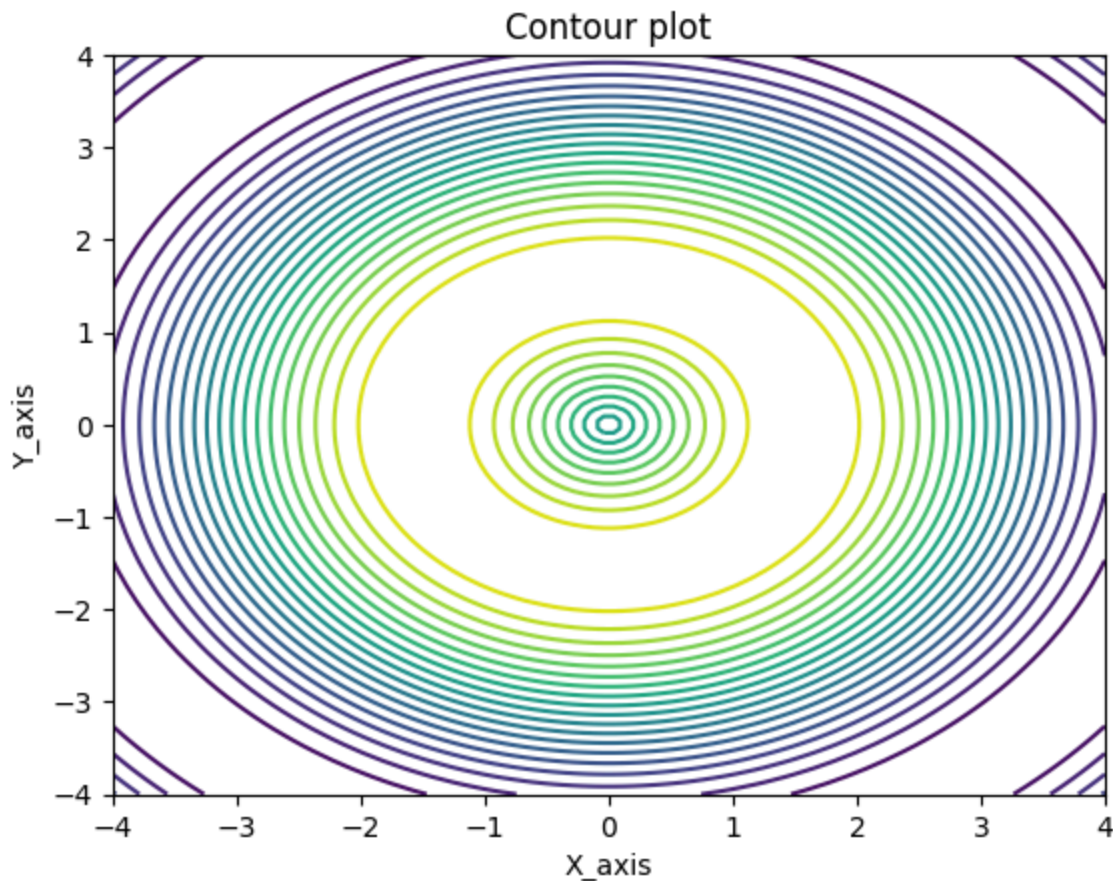
```
In [40]: import numpy as np
import matplotlib.pyplot as plt
import scipy.stats as stats
N = 20 # population size
K = 7 # number of sucess stats in the populatin
n = 12 # number of draws
x = np.arange(0, n+1)
pmf = stats.hypergeom.pmf(x, N, K, n)
plt.bar(x,pmf, color="green",alpha=0.7)
plt.xlabel("number of sucess")
plt.ylabel("porbability")
plt.title("Hypergeometric Distribution")
plt.show()
```



contoure plot

```
In [41]: import numpy as np
import matplotlib.pyplot as plt # Fixed: matplotlib → matplotlib
x = np.linspace(-4, 4, 100)     # Fixed: Linespace → linspace, corrected syntax
y = np.linspace(-4, 4, 100)
X, Y = np.meshgrid(x, y)
Z = np.sin(np.sqrt(X**2 + Y**2))

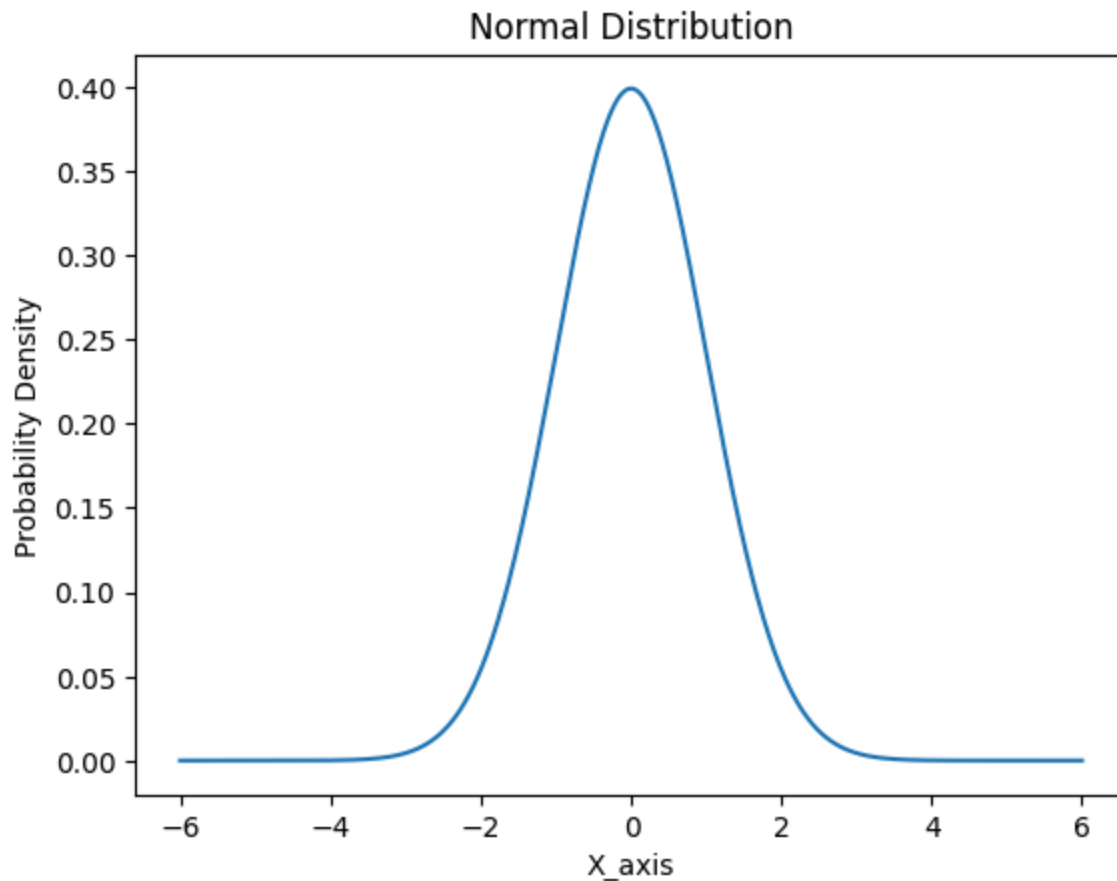
plt.contour(X, Y, Z, levels=20, cmap='viridis') # Fixed: camp → cmap
plt.title("Contour plot")                      # Removed duplicate plt.contour()
plt.xlabel("X_axis")
plt.ylabel("Y_axis")
plt.show()
```



Normal Distribution

```
In [42]: import numpy as np
import matplotlib.pyplot as plt
import scipy.stats as stats

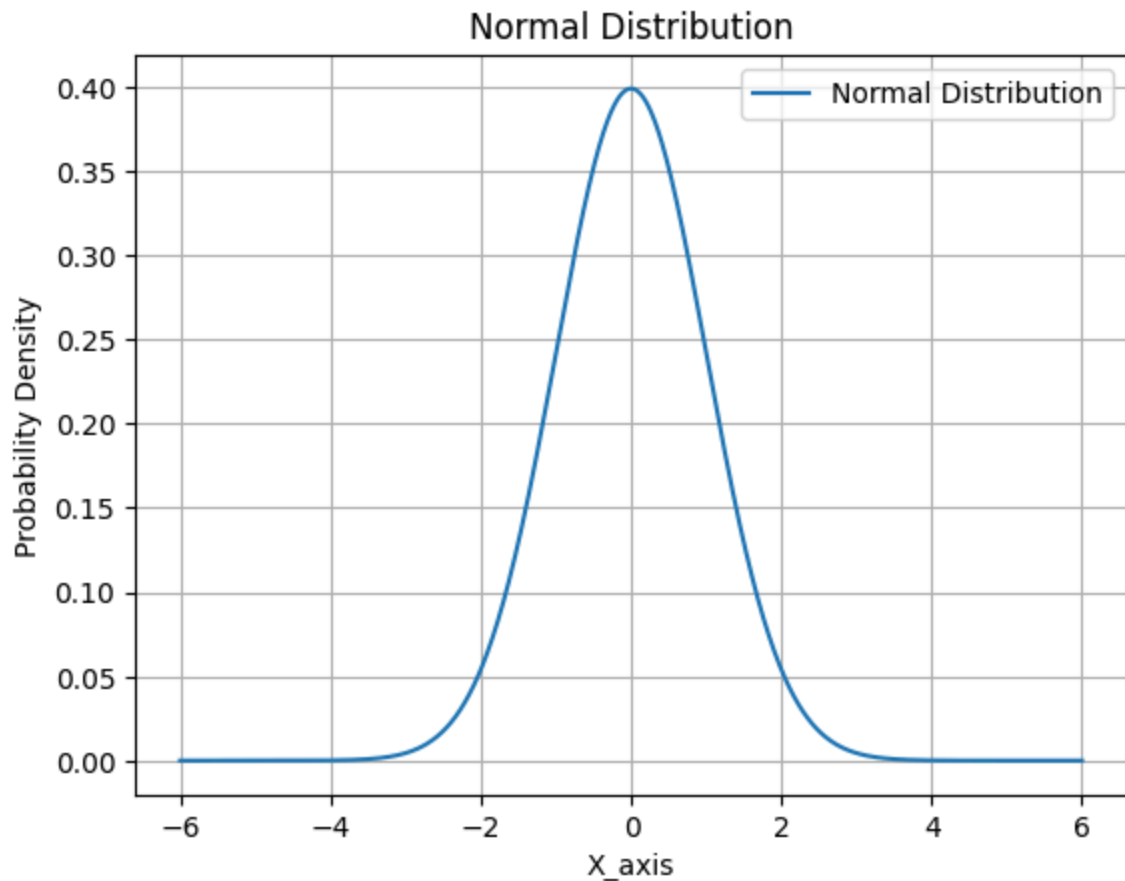
x = np.linspace(-6, 6, 1000)
y = stats.norm.pdf(x, loc=0, scale=1.0)
plt.plot(x, y, label="Normal Distribution")
plt.xlabel("X_axis")
plt.ylabel("Probability Density")
plt.title("Normal Distribution")
plt.show()
```



```
In [43]: import numpy as np
import matplotlib.pyplot as plt
import scipy.stats as stats

x = np.linspace(-6, 6, 1000) # Fixed: 'linespace' -> 'linspace' and proper syntax
y = stats.norm.pdf(x, loc=0, scale=1.0) # Fixed: added 'stats.'

plt.plot(x, y, label="Normal Distribution")
plt.xlabel("X_axis")
plt.ylabel("Probability Density")
plt.title("Normal Distribution")
plt.legend() # Optional but recommended if using label
plt.grid(True) # Optional: adds grid to the plot
plt.show()
```



3D response surface plot

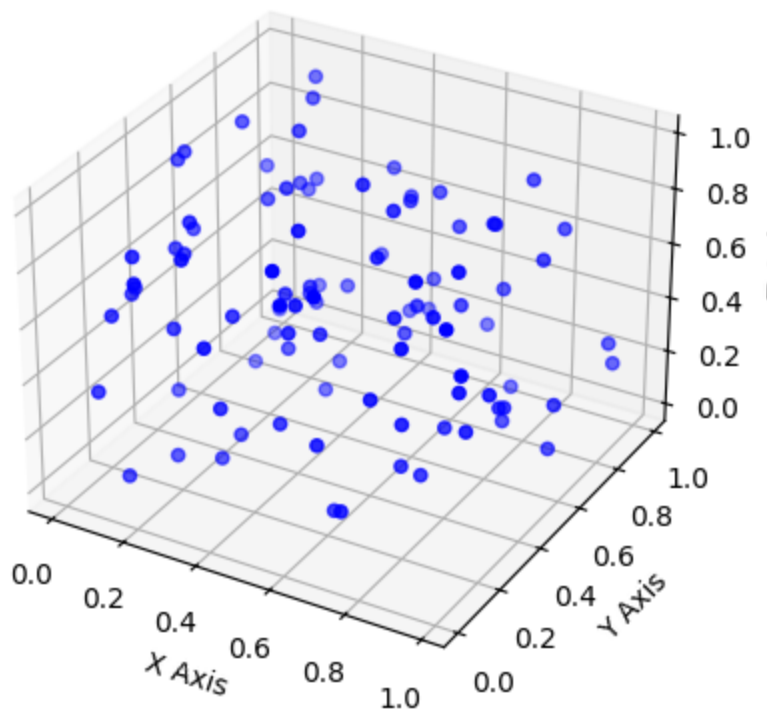
```
In [21]: import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import numpy as np

# Generating random 3D data
np.random.seed(42)
n_points = 100
x = np.random.rand(n_points)
y = np.random.rand(n_points)
z = np.random.rand(n_points)

# Creating a 3D scatter plot
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.scatter(x, y, z, c='blue', marker='o')

ax.set_xlabel('X Axis')
ax.set_ylabel('Y Axis')
ax.set_zlabel('Z Axis')
ax.set_title('3D Scatter Plot')
plt.show()
```

3D Scatter Plot



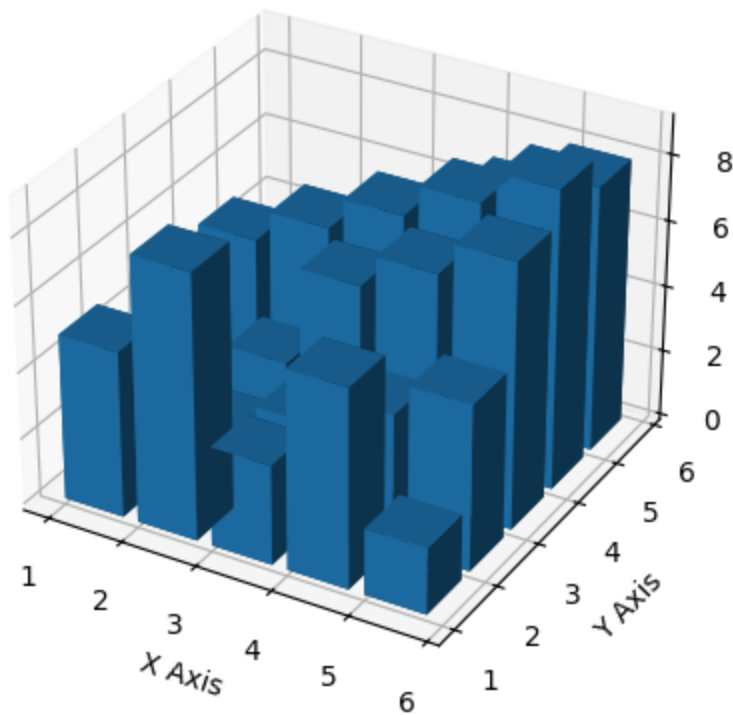
```
In [22]: import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import numpy as np

# Generating data for a 3D bar plot
x_pos = np.arange(1, 6)
y_pos = np.arange(1, 6)
x_pos, y_pos = np.meshgrid(x_pos, y_pos)
z_pos = np.zeros_like(x_pos)
z = np.array([[5, 8, 3, 6, 2],
              [1, 2, 3, 4, 5],
              [2, 3, 6, 7, 8],
              [5, 6, 7, 8, 9],
              [3, 4, 5, 7, 8]])

# Creating a 3D bar plot
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.bar3d(x_pos.flatten(), y_pos.flatten(), z_pos.flatten(), 0.8, 0.8, z.flatten(),
         color='blue')

ax.set_xlabel('X Axis')
ax.set_ylabel('Y Axis')
ax.set_zlabel('Z Axis')
ax.set_title('3D Bar Plot')
plt.show()
```

3D Bar Plot

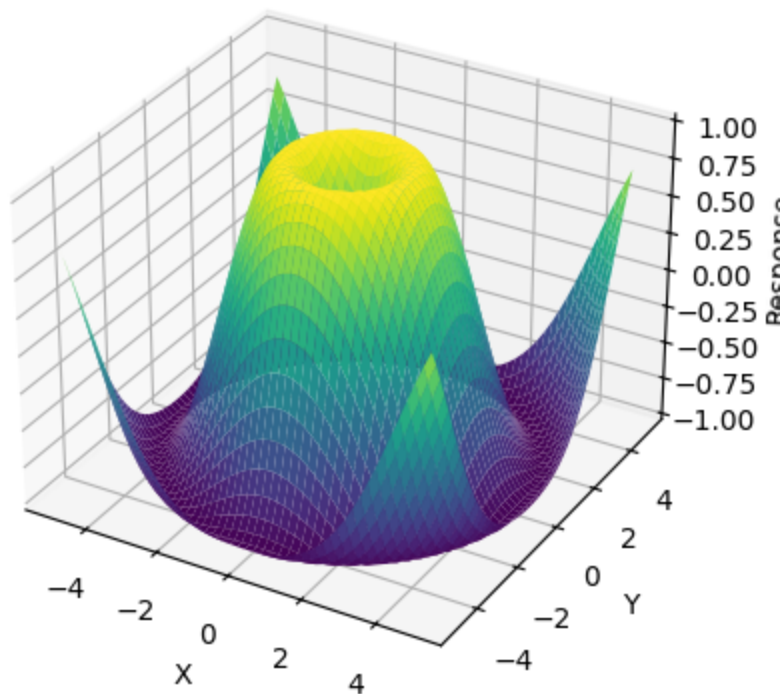


```
In [44]: import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

X = np.linspace(-5, 5, 50)
Y = np.linspace(-5, 5, 50)
X, Y = np.meshgrid(X, Y)
Z = np.sin(np.sqrt(X**2 + Y**2))

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.plot_surface(X, Y, Z, cmap='viridis') # fixed 'camp' -> 'cmap'
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Response')
ax.set_title("3D Response surface plot")
plt.show()
```

3D Response surface plot



```
In [20]: import matplotlib.pyplot as plt
import numpy as np
from mpl_toolkits.mplot3d import Axes3D

# Parametric equations for a torus
def torus_parametric(u, v, R=1, r=0.3):
    x = (R + r * np.cos(v)) * np.cos(u)
    y = (R + r * np.cos(v)) * np.sin(u)
    z = r * np.sin(v)
    return x, y, z

# Generating data
u = np.linspace(0, 2 * np.pi, 100)
v = np.linspace(0, 2 * np.pi, 100)
U, V = np.meshgrid(u, v)
X, Y, Z = torus_parametric(U, V)

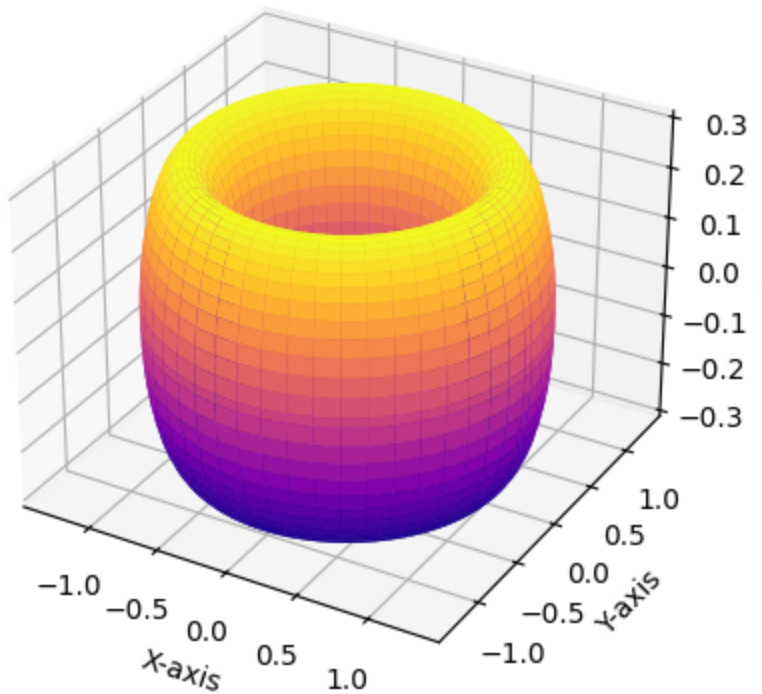
# Creating a 3D plot
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

# Plotting the parametric 3D surface
ax.plot_surface(X, Y, Z, cmap='plasma')

# Customizing the plot
ax.set_xlabel('X-axis')
ax.set_ylabel('Y-axis')
ax.set_zlabel('Z-axis')
ax.set_title('Parametric 3D Surface Plot (Torus)')
```

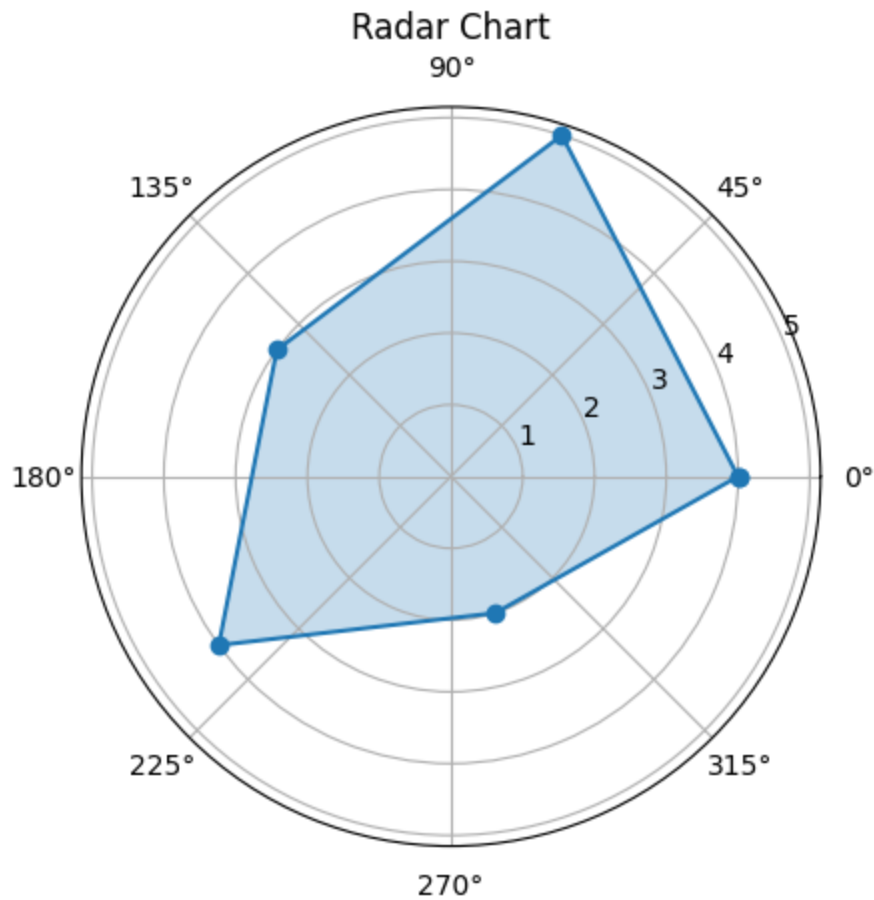
```
# Displaying the plot
plt.show()
```

Parametric 3D Surface Plot (Torus)



In []:

```
In [45]: import matplotlib.pyplot as plt
import numpy as np
labels=np.array(["A","B","C","D","E"])
data=np.array([4,5,3,4,2])
angles = np.linspace(0,2*np.pi,len(labels),endpoint=False)
data=np.concatenate((data,[data[0]]))
angles=np.concatenate((angles,[angles[0]]))
plt.polar(angles,data,marker="o")
plt.fill(angles,data,alpha=0.25)
plt.title("Radar Chart")
plt.show()
```



```
In [46]: import turtle

# Screen setup
screen = turtle.Screen()
screen.bgcolor("white")

# Turtle setup
t = turtle.Turtle()
t.speed(0)      # Fastest speed
t.width(2)     # Pen width

# Rainbow colors
colors = ["red", "orange", "yellow", "green", "blue", "indigo", "violet"]

# Drawing loop
for i in range(360):
    t.color(colors[i % len(colors)]) # Cycle through colors
    t.forward(i * 2 / len(colors) + i)
    t.left(120)
    t.left(1)

# Finish
turtle.done()
```